A TEXTBOOK OF

DIGITAL LOGIC DESIGN

MTR-272

D.A.E. 2nd Year

Mechatronics Technology

Authors

ENGR MUHAMMAD WASEEM

Instructor (Electronics), GCT TAXILA

BILAL AHMED

Jr.Instructor (Mechatronics), GCT CHAKWAL

Published by:

TEVTA PUNJAB (ACADEMICS WING)

PREFACE

We are very thankful to Almighty ALLAH who bestowed us the capability to complete this textbook, named: "*DIGITAL LOGIC DESIGN"* for the students of D.A.E 2ND year Mechatronics; and millions of Darood o Salaam (درودوسلام) on the Last Prophet حضرت ﷺ **محمد رسول اللہ .**

In this textbook all topics related to Digital Logic Design as directed by new syllabus of Punjab Board of Technical Education, are covered with all the necessary theoretical information and detailed examples to understand the basic of logic design, logic gates and logic ideas in true letter and spirit with the supporting circuit diagrams. At the end of each chapter, numerical problems and short questions are given to enhance the concepts of students.

We hope the presented textbook will be beneficial for both the students and other people having interest in *"Digital Logic Design".*

We are thankful to all our colleagues especially in "*Telecom Technology, GCTTAXILA and Mechatronics Technology in GCT CHAKWAL*". We dedicate this textbook to "*Our Honorable Parents."* with prayers that:

رَبِّ ارْحَمْهُمَا كَمَا رَبَّيٰنِىْ صَغِيْرًا ۝

Finally, as a human being the chance of mistake in this edition is possible, so all the professional readers are requested to highlight the mistakes found, if any. Your suggestions will always be welcome with open heart and acknowledged.

AUTHORS

CONTENTS INDEX

CONTENTS		Page No.
Preface		2
Course Outline		7-12
Chapter No. 01 INTRODUCTION		13-32
1.1	Digital and Analog Quantities	
1.2	Number systems, BIT, BYTE, NIBBLE	
1.3	Elements of digital electronics	
Chapter No. 02 LOGIC GATES		33-56
2.1	And Gate	
2.2	OR Gate	
2.3	NOT Gate	
2.4	NAND Gate	
2.5	NOR Gate	
2.6	X-OR Gate	
Chapter No. 03 BOOLEAN ALGEBRA		57-68
3.1	Laws of Boolean algebra	
3.2	De Morgan's Laws	
3.3	Boolean Function Generation from truth tables	

Chapter No. 04 ARITHMETIC LOGIC CIRCUITS		
4.1	Half Adder	**69-79**
4.2	Full Adder	
4.3	Adder-Subtractor	
4.4	Comparator	
Chapter No. 05 COMBINATIONAL LOGIC AND DATA PROCESSING CIRCUITS		
5.1	Multiplexers & logic (IC's such as 74150)	**80-104**
5.2	De-multiplexers (IC's such as 74154)	
5.3	Decoders	
5.4	BCD-to-Decimal Decoders (such as 7445)	
5.5	Seven-Segment Decoders & Displays	
5.6	Encoders (IC's such as 74147)	
5.7	Read-only memories	
5.8	Diode ROM	
5.9	Commercially available ROM	
Chapter No. 06 Flip Flops		
6.1	RS Flip-Flop	**105-121**
6.2	Clocked RS Flip-Flop	
6.3	D Flip-Flop	

6.4	Edge-triggered D Flip-Flop	
6.5	JK Flip-Flop	
Chapter No. 07	**CLOCKS & TIMERS**	**122-136**
7.1	TTL clock	
7.2	555–astable, monostable	
7.3	Applications	
Chapter No. 08	**SHIFT REGISTERS**	**137-145**
8.1	Introduction, shift-Right & shift-Left	
8.2	Serial in-serial out	
8.3	Serial-in parallel-out	
8.4	Parallel-on serial-out	
Chapter No. 09	**COUNTERS**	**146-158**
9.1	Introduction, types	
9.2	Ripple counter	
9.3	Synchronous counter	
9.4	Presentable counter	
9.5	Digital clock	
Chapter No. 10	**MEMORIES**	**159-171**
10.1	Introduction, volatile, non-volatile	

10.2	Memory Addressing	
10.3	ROM, PROM, EPROM, capacity	
10.4	RAM, Static & dynamic, capacity	
Chapter No. 11	**APPLICATIONS**	**172-184**
11.1	Digital Clock	
11.2	Frequency and Time Period counter	
11.3	Moving Message Display	

MT-272DIGITAL LOGIC DESIGN

TOTAL CONTACT HOURS	**128**	**T**	**P**	**C**
THEORY	**32**	**1**	**3**	**2**
PRACTICAL	**96**			

AIMS: At the end of this course, the students will be able to:-

1. To understand the fundamentals of digital electronics
2. To understand the applications of digital electronics devices in the industry.

COURSE CONTENTS

1. INTRODUCTION

1.1 Digital & analogy quantities

1.2 Number systems, BIT, BYTE, NIBBLE

1.3 Elements of digital electronics 2 Hours

2. LOGIC GATES

2.1 AND Gate

2.2 OR Gate

2.3 NOT Gate

2.4 NAND Gate

2.5 NOR Gate

2.6 XOR Gates 3 Hours

3. BOOLEAN ALGEBRA

3.1 Laws of Boolean algebra

3.2 De Morgans Laws

3.3 Boolean Function Generation from truth tables 2 Hours

4. ARITHMETIC LOGIC CIRCUITS

4.1 Half adders

4.2 Full adders

4.3 Adder-subtractor

4.4 Comparators 3 Hours

5. COMBINATIONAL LOGIC AND DATA PROCESSING CIRCUITS

5.1 Multiplexers & logic (IC's such as 74150)
5.2 DE multiplexers (IC's such as 74154) 3 Hours
5.3 Decoders
5.4 BCD-to-Decimal Decoders (such as 7445)
5.5 Seven-Segment Decoders & Displays
5.6 Encoders (IC's such as 74147)
5.7 Read-only memories
5.8 Diode ROM
5.9 Commercially available ROM

6. FLIP-FLOPS

6.1 RS Flip-Flop
6.2 Clocked RS Flip-Flop
6.3 D Flip-Flop
6.4 Edge-triggered D Flip-Flop
6.5 JK Flip-Flop 4 Hours

7. CLOCKS & TIMERS

7.1 TTL clock
7.2 555–astable, monostable
7.3 Applications 4 Hours

8. SHIFT REGISTERS

8.1 Introduction, shift-Right & shift-Left
8.2 Serial in-serial out
8.3 Serial-in parallel-out
8.4 Parallel-on serial-out 3 Hours

9. COUNTERS

9.1 Introduction, types
9.2 Ripple counter
9.3 Synchronous counter
9.4 Presentable counter 9.5 Digital clock 2 Hours

10. MEMORIES

10.1 Introduction, volatile, non-volatile

10.2 Memory Addressing

10.3 ROM, PROM, EPROM, capacity

10.4 RAM, Static & dynamic, capacity 3 Hours

11. APPLICATION OF DIGITAL ELECTRONICS

11.1 Digital clock

11.2 Frequency & time period counter

11.3 Moving Message Display 2 Hours

RECOMMENDED BOOKS

1. Industrial Electronics: James A. Rehg and Glenn J. Sartori (2005)

2. Digital Electronics: A Practical Approach William Kleitz (8th Edition) (2007)

3. Power Electronics: B.W. Williams

MT-272 DIGITAL LOGIC DESIGN

INSTRUCTIONAL OBJECTIVE

1. KNOW THE BASIC TERMS RELATED TO DIGITAL ELECTRONICS.

1.1 State in a tabulated form the merits & demerits of analog & digital quantities

1.2 Define basic terms related to digital electronics

2. UNDERSTAND THE WORKING OF VARIOUS LOGIC GATES

2.1 Describe the working of AND Gate

2.2 Describe the working of OR Gate

2.3 Describe the working of NOT Gate

2.4 Describe the working of NAND Gate

2.5 Describe the working of NOR Gate

2.6 Describe the working of XOR Gates

2.7 Understand the configurations formed by combining gates

3. INTRODUCTION TO BOOLEAN ALGEBRA & ITS LAWS

3.1 Study the laws of Boolean algebra

3.2 Study the De Morgan's Laws

3.3 Understand the Boolean function generation from truth tables

4. DETAILED DESCRIPTION OF ARITHMETIC LOGIC CIRCUITS

4.1 Describe the working of half adders

4.2 Describe the working of Full adders

4.3 Describe the working of Adder-subtractor

4.4 Describe the working of comparators

5. UNDERSTAND THE WORKING AND USES OF COMBINATIONAL LOGIC CIRCUITS, INCLUDING DATA PROCESSING CIRCUITS

5.1 Define the terms multiplexer, DE multiplexers, decoder, encoders, BDC converters

5.2 Explain multiplexers, using logic circuits & block-diagrams (multiplexers using IC's such as 74150)

5.3 Explain DE multiplexers, using block-diagrams (using IC's such as 74154)

5.4 Explain using block-diagram<BDC& its conversion to Decimals, using IC's, such as7445

5.5 Explain seven segments decoders, showing block diagrams, giving examples for letters & digits.

5.6 Describe the operation of combinational logic circuits as applied to data processing circuits

5.7 Explain the working of 7-segment display circuit

6. DETAILED DESCRIPTION OF FLIP-FLOPS

6.1 Describe the working of RS Flip-Flop

6.2 Describe the working of Clocked RS Flip-Flop

6.3 Describe the working of D Flip-Flop

6.4 Describe the working of Edge-triggered D Flip-Flop

6.5 Describe the working of JK Flip-Flop

7. UNDERSTAND THE WORKING OF CLOCKS & TIMERS

7.1 Study the function of a TTL clock

7.2 Study the Timer 555 – a stable, monostable

7.3 Identify the applications of clock and timers

8. KNOW ABOUT SHIFT REGISTERS

8.1 Study the basic concepts of shift-right & shift-left registers

8.2 Understanding the serial in-serial out configuration

8.3 Understand the serial-in parallel-out configuration

8.4 Understand the parallel-on serial-out configuration

9. UNDERSTAND THE WORKING OF COUNTERS

9.1 Study the basics and the types of counters

9.2 Describe the working of the ripple counter

9.3 Describe the working of synchronous counter

9.4 Describe the working of presentable counter

9.5Understand the working of digital clock

10 KNOWS VARIOUS TYPES OF MEMORIES

10.1 Define various types of memories: ROM, PRM, EPROM, and RAM

10.2 State memory of common memory devices in KB, MB

10.3 Know the system of memory addressing

11 APPLY THE CONCEPTS OF DIGITAL ELECTRONICS FOR PROJECT(S)

11.1 Explain the use of digital electronic circuits as elements of system

11.2 Describe the use of digital electronic circuits for the following:-

11.2.1 Moving display

11.2.2 Frequency counters

11.2.3 Digital clock

DIGITAL LOGIC DESIGN

LIST OF PRACTICALS

1. Identify and verify truth tables for AND, OR, NOT Gates ICs
2. Identify and verify truth tables for NOR, NAND, XOR Gates ICs
3. Construct and verify truth tables of half adder, full adder
4. Connect and study an adder &sub tractor circuits
5. To perform and study the function of Decoder &DE multiplexer through related IC
6. To perform and study the function of Encoder & Multiplexer through related IC
7. Identify and verify the functions of RS Flip-Flop
8. Identify and verify the functions of Clocked RS Flip-Flop
9. Identify and verify the functions of D Flip-Flop
10. Identify and verify the functions of JK Flip-Flop
11. Connect a 555 IC as
    A-stablemultivibrator
    Mono-stablemultivibrator
    Bi-stablemultivibrator
12. Connect and observe the working of shift registers (SISO, SIP, PISO, PIPO)
13. Identify, connect and observe working of ripple and synchronous counter
14. Select components, assemble and observe working of a digital clock
15. Study the working of memory devices
16. Assemble and observe working of frequency counter (Project – 2)
17. Assemble and observe working of moving message display (Project – 3)

CHAPTER NO. 01

INTRODUCTION

1.1 Introduction

. At some point in our education or even our daily lives, we have all heard the terms “analog devices” and “digital devices”. Where analog devices use analog signals, digital devices use digital logic design to enter, interpret, and display data. In a digital system, values are discrete and there are only two states- it is either 1 or 0, or it is either ‘on’ or ‘off’. This digital, binary representation of input and output data is basically what digital logic design is.

1.2 Chapter Objectives

1. Understand the basics of analog and digital.
2. Understand the basics of bit, byte and also binary number systems

1.3 Analog Quantities

By definition, “Analog signal is any continuous signal for which the time-varying feature (variable) of the signal is a representation of some other time wearing quantity, that is, analogous to another time wearing a signal.” To quote an example, in an analog audio signal, the instantaneous voltage of the signal fluctuates in harmony with the pressure of the sound waves.

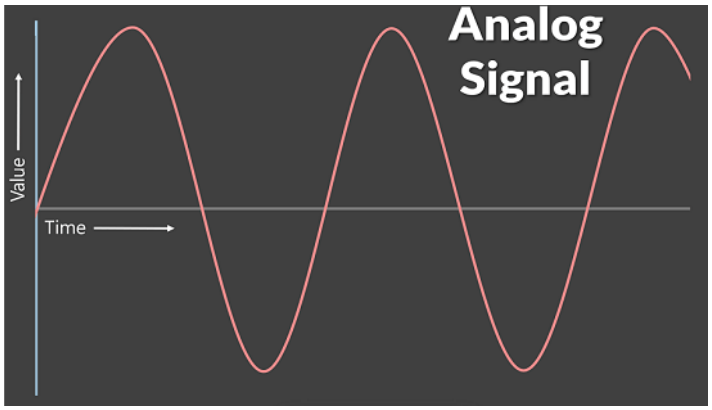


Figure 1.1:Analog representation of signal

By looking at the graph of a signal, it is easy to identify whether the signal is analog or digital. In the case of an Analog signal, the graph is generally smooth and continuous and very much similar to the sine graph. As we can see in a continuous graph, the range is specified for maximum and minimum values. However, it is to one who has to keep in mind that there are still an infinite number of possible values within that particular range.

1.3.1 Uses of Analog system

Almost all types of measurements use specialized Analog signal Processing. For example, the electric field meters, precision micro voltmeters, Analog watches thermometer, capacitance, etc. They are also used to broadcast radio frequency and widely used in mobile applications too.

1.3.2 Advantages of the Analog system

1. One of the significant benefits of the Analog signal is that it can store up an infinite amount of data.
2. As compared to digital signals, the density of messages is much higher.
3. It is effortless to understand the use of Analog signals as compared to digital signals.
4. The Analog system and their use is very less expensive as compared to the digital system.
5. The accuracy of the Analog system is also more than the digital system.

1.3.3 Disadvantages of Analog system

1. One of the significant drawbacks in the Analog signal is that along with the transmission of messages; it carries a lot of unwanted noise, which is transmitted during recording.
2. While transferring data at long distances, there is a high chance of having unwanted disturbances.
3. If the operator of an Analog system takes months to upgrade the order to the digital interface, it is not possible.

1.3.4 Digital Signal

By definition, “a digital signal is a signal that is being used to represent as a series of discrete values, at any given time it can only take on one of a finite number of values.” The fact that the digital signal can only take a finite number of values is the fundamental difference between the digital signal

and The Analog signal. The digital signals can only have a limited number of values, whereas the Analog signals have an almost infinite number of values.

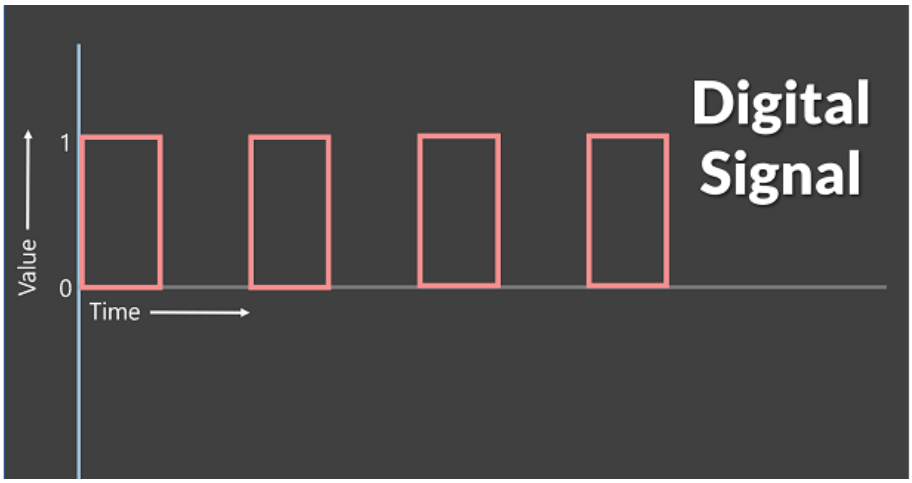


Figure 1.2:Digital representation of signal

In most of the digital circuits of the systems, the signals can have only two possible values, and they are also called binary signals or logic signals. The digital signals are represented by two voltage pants, one band which is near a reference value (generally 0), and the other band lies near the supply voltage. This is similar to the values, '0' and '1' or 'false' and 'true' of the Boolean domain. This means that at any particular time, a digital signal can represent only one binary digit.

One significant advantage of digital signals is that they are immune to outside noise and electronic noise, provided that it is not very loud and does not affect the digital circuits. In the case of the Analog signal, the noise can always degrade the transmission quality.

1.3.4.1 Uses of digital system

Nowadays, the digital system is used almost everywhere. Thermometer use, photocopies, landline telephones, audiotape recorders, television, computers, laptops, mobile phones, wristwatches, wall clocks, are all becoming digital nowadays. It increases the accuracy of the message as well as makes it easy to read.

1.3.4.2 Advantages of Digital system or signals

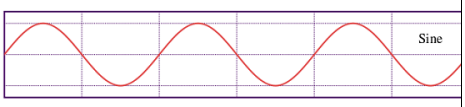
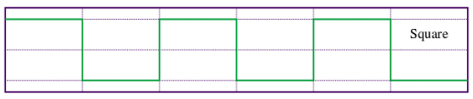
1. Because of the digital nature, the signals in the digital systems can travel significantly faster over digital lines as compared to the Analog signals
2. As compared to Analog signals, digital signals can transfer more data.

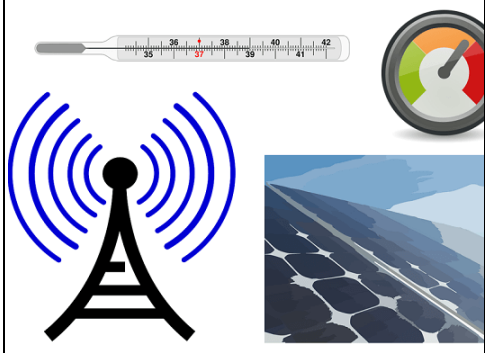
3. The digital systems are less expensive, more reliable, easy to manipulate, and more flexible as compared to the Analog system.
4. A digital system can be made compatible with other digital systems to which are not possible in the Analog system.

1.3.4.3 Disadvantages of digital system or signals

1. Higher bandwidth in the case of digital signal transmission is essential.
2. the systems and processing of the signals are more complicated as compared to the Analog system
3. There is always a very big chance of sampling errors in case of digital system
4. In order to detect the digital signals, the communications system of the digital systems needs to be appropriately synchronized.

Difference between Analog signals and Digital signals

Analog Signal	Digital Signal
The analog signal is a continuous signal that helps to measure physical quantities.	Digital signals are time separated signals, and they are generated with the help of digital modulation.
The analog signals can be denoted with the sine curves. 	The digital signals can be denoted with the square curves. 
It can use a continuous range of values that facilitate you to understand or send information.	The Digital signal uses distinct 0 and 1 to represent information.
Temperature sensors, Resistive touch screen, FM radio signals,	Computers, Laptops, Pen drive, CDs, DVDs

Light sensor; Photocells are examples of Analog signals. 	are some examples of Digital signal. 
The bandwidth of the analog signal is low.	The bandwidth of digital signal bandwidth is high.
Noise can easily damage the analog signals during transmission, as well as the write/read cycle.	Digital signals are immune to deterioration during the transmission process and write/read cycle.
Analog hardware can never offer flexible execution.	Digital hardware offers flexibility in execution.
Analog signals are preferable for audio and video transmission.	Digital signals are preferable for Computing and digital electronics.
Analog instruments generally have a scale, which is situated at the lower end and gives significant observational errors.	Digital instruments do not give any kind of observational errors.
Analog signals have no fixed range.	Digital signals have a finite range, that is, 0 and 1.

The analog systems record waveforms as it is.	The digital systems collect analog waveforms into a limited set of numbers and then record them.
The analog systems can store data in the form of wave signals.	The digital systems can store data in the form of a binary bit.
As compared to digital signals, analog signals are generally very cheap.	As compared to analog signals, digital signals are costly.

1.4 Basic Terminologies

Some of the basic terminologies that are used in digital system are given below

BIT:

The bit is a basic unit of information in computing and digital communications. ... The bit represents a logical state with one of two possible values. These values are most commonly represented as either "1"or"0", but other representations such as true/false, yes/no, +/−, or on/off are common.

BYTE:

The term "**byte**" is a respelling of the word "bite," in order to avoid the problem of a typo causing the two words to be mistaken for one another. If you have ever heard of an eight bit computer, what that means is that the **digital circuits** within the computer are **designed** to primarily handle single **bytes**.

NIBBLE:

A **nibble** (occasionally nibbles or nibble to match the spelling of **byte**) is a four-bit aggregation, or half an octet. It is also known as half-**byte** or tetrade. ... A **nibble** has sixteen (2^4) possible values. A **nibble** can be represented by a single hexadecimal digit and called a hex digit.

WORD:

Combination of few bits or set of information or binary data in whole unit is known as word.

i.e 1011, 1111, 1010 etc…

CODE:

Groups of bits are known as Code however figures not matter. These codes also representation of numbers, letters, symbols and instructions etc…

Number Systems

Most number systems follow a common pattern for writing down the value of a number: A fixed number of values can be written with a single numerical character, then a new column is used to count how many times the highest value in the counting system has been reached. The number of numerical values the system uses is called the base of the system. For example, the decimal system has 10 numerical characters and so has a base of 10:

0 1 2 3 4 5 6 7 8 9

For writing numbers greater than 9 a second column is added to the left, and this column has 10 times the value of the column immediately to its right.

Because number systems commonly used in digital electronics have different base values to the decimal system, they look less familiar, but work in essentially the same way.

Decimal, (base 10)

Decimal has ten values 0 to 9. If larger values than 9 are needed, extra columns are added to the left. Each column value is ten times the value of the column to its right. For example the decimal value twenty-two is written 22 (2 tens + 2 ones).

Binary, (base 2)

Binary has only two values 0 and 1. If larger values than 1 are needed, extra columns are added to the left. Each column value is now twice the value of the column to its right. For example the decimal value three is written 11 in binary (1 two + 1 one).

Octal, (base 8)

Octal has eight values 0 to 7. If larger values than 7 are needed, extra columns are added to the left. Each column value is now 8 times the value of the column to its right. For example the decimal value twenty-seven is written 33 in octal (3 eights + 3 ones).

Hexadecimal, (base 16)

Hexadecimal has sixteen values 0 to 15, but to keep all these values in a single column, the 16 values (0 to 15) are written as 0 to F, using the letters A to F to represent numbers 10 to 15, so avoiding the use of a second column. Again, if higher values than 15 (F in hexadecimal) are needed, extra columns to the left are used. Each column value is sixteen times that of the column to its right. For example the decimal value sixty-eight is written as 44 in hexadecimal (4 sixteens + 4 ones).

Some column values of different number systems				
Decimal	1000	100	10	1
Binary	8	4	2	1
Octal	512	64	8	1
Hexadecimal	4096	256	16	1

Each of these different number systems works in the same way, it is just that each system has a different base, and the column values in each system increase by multiples of the base number as columns are added to the left.

Because this module describes several different number systems, it is important to know which system is being described. Therefore if there is some doubt which system a number is in, the base of the system, written as a subscript immediately after the value, is used to identify the number system.

For example:

1010 represents the decimal value ten. (1 ten + 0 units)

102 represent the binary value two. (1 two + 0 units)

108 represent the octal value eight. (1 eight + 0 units)

1016 represents the hexadecimal value sixteen. (1 sixteen + 0 units)

Decimal (Radix 10)	Binary (Radix 2)	Octal (Radix 8)	Hexadecimal (Radix 16)
0	0	0	0
1	1	1	1
2		2	2
3		3	3
4		4	4
5		5	5
6		6	6
7		7	7
8			8
9			9
			A
			B
			C
			D
			E
			F

The System Radix

The base of a system, more properly called the RADIX, is the number of different values that can be expressed using a single digit. Therefore the decimal system has a radix of 10, the octal system has a radix of 8, hexadecimal is radix 16, and binary radix 2.

The range of number values in different number systems is shown in Table 1.1.2, Notice that because the hexadecimal system must express 16 values using only one column, it uses the letters A B C D E & F to represent the numbers 10 to 15.

The Radix Point.

When writing a number, the digits used give its value, but the number is 'scaled' by its RADIX POINT.

For example, 456.210 is ten times bigger than 45.6210 although the digits are the same.

Notice also that when using multiple number systems, the term 'RADIX point' instead of 'DECIMAL point' is used. When using decimal numbers, a decimal point is used, but if a different system is used, it would be wrong to call the

point a decimal point, it would need to be called "Binary point" or "Octal point" etc. The simplest way around this is to refer to the point in any system (which will of course have its value labeled with its radix) as the RADIX POINT.

Exponents

A decimal number such as 456.210 can be considered as the sum of the values of its individual digits, where each digit has a value dependent on its position within the number (the value of the column):

= 456.210

Col 2	Col 1	Col 0	Col -1
4 hundreds	+ 5 tens	+ 6 units	+ 2 tenths
(4×10^{2})	$+ (5 \times 10^{1})$	$+ (6 \times 10^{0})$	$+ (2 \times 10^{-1})$
400	+ 50	+ 6	+ 0.2

Each digit in the number is multiplied by the system radix raised to a power depending on its position relative to the radix point. This is called the EXPONENT. The digit immediately to the left of the radix point has the exponent 0 applied to its radix, and for each place to the left, the exponent increases by one. The first place to the right of the radix point has the exponent -1 and so on, positive exponents to the left of the radix point and negative exponents to the right.

This method of writing numbers is widely used in electronics with decimal numbers, but can be used with any number system. Only the radix is different.

Hexadecimal exponents 98.216 = (9 x 161) + (8 x 160) + (2 x 16-1)

Octal exponents 56.28 = (5 x 81) + (6 x 80) + (2 x 8-1)

Binary Exponents 10.12 = (1 x 21) + (0 x 20) + (1 x 2-1)

When using your calculator for the above examples you may find that it does not like radix points in anything other than decimal mode. This is common with many electronic calculators.

Floating Point Notation

If electronic calculators cannot use radix points other than in decimal, this could be a problem. Fortunately for every problem there is a solution. The radix exponent can also be used to eliminate the radix point, without altering the value of the number. In the example below, see how the value remains

the same while the radix point moves. It is all done by changing the radix exponent.

102.610 = 102.6 x 100 = 10.26 x 101 = 1.026 x 102 = .1026 x 103

The radix point is moved one place to the left by increasing the exponent by one.

It is also possible to move the radix point to the right by decreasing the exponent. In this way the radix point can be positioned wherever it is required, in any number system, simply by changing the exponent. This is called FLOATING POINT NOTATION and it is how calculators handle decimal points in calculations.

Normalized Form

By putting the radix point at the front of the number, and keeping it there by changing the exponent, calculations become easier to do electronically, in any radix.

Electronic storage of numbers.

A number written (or stored) in this way, with the radix point at the left of the most significant digit is said to be in NORMALISED FORM. For example .110112 x 23 is the normalized form of the binary number 110.112. Because numbers in electronic systems are stored as binary digits, and a binary digit can only be 1 or 0, it is not possible to store the radix point within the number. Therefore the number is stored in its normalized form and the exponent is stored separately. The exponent is then reused to restore the radix point to its correct position when the number is displayed.

In electronics systems a single binary digit is called a bit (short for Binary Digit), but as using a single digit would seriously limit the math's that could be performed, binary bits are normally used in groups.

4 bits = 1 nibble

8 bits = 1 byte

Multiple bytes, such as 16 bits, 32 bits, 64 bits are usually called 'words', e.g. a 32 bit word. The length of the word depends on how many bits can be physically handled or stored by the system at one time.

Decimal	MSB	4 Bit Binary		LSB
	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

4 Bit Binary Representation

When a number is stored in an electronic system, it is stored in a memory location having a fixed number of binary bits. Some of these memory locations are used for general storage whilst others, having some special function, are called registers. Wherever a number is stored, it will be held in some form of binary, and must always have a set number of bits. Therefore a decimal number such as 13, which can be expressed in four binary bits as 11012, becomes 000011012 when stored in an eight-bit register. This is achieved by adding four NON SIGNIFICANT ZEROS to the left of the most significant '1' digit.

Using this system, a binary register that is n bits wide can hold 2n values. Therefore an 8 bit register can hold 28 values = 256 values (0 to 255)

A 4 bit register can hold 24 values = 16 values (0 to 15)

1.3.6 Converting Between Number Systems

It is often necessary to convert values written in one number system to another. The simplest way is to reach for your calculator or use a conversion app. from the web. That is fine, but converting a number in this way does not help you to understand the way each number system works and how different systems are related. The purpose of this module is to explain just that, and to get you to carry out some simple conversions so that you can not only convert between number systems, but also understand how the conversion process works. There are various ways to tackle conversions without a calculator; once the conversion methods are learned, the only

skills needed are the ability to multiply and divide, and to add together a few numbers.

Conversion from any system to decimal

The number of values that can be expressed by a single digit in any number system is called the system radix, and any value can be expressed in terms of its system radix.

Octal to Decimal

For example the system radix of octal is 8, since any of the 8 values from 0 to 7 can be written as a single digit.

Convert 126_8 to decimal.

Using the values of each column, (which in an octal integer are powers of 8) the octal value 126_8 can also be written as:

$(1x8^2) + (2x8^1) + (6 x 8^0)$

As ($8^2 = 64$), ($8^1 = 8$) and ($8^0 =1$), this gives a multiplier value for each column.

Multiply the digit in each column by the column multiplier value for that column to give:

1x64 = 64 2x8 =16 6x1 = 6

Then simply add these results to give the decimal value.

$64 + 16 + 6 = 86_{10}$

Therefore $126_8 = 86_{10}$.

Binary to Decimal

Convert 1101_2 to decimal.

The same method can be used to convert binary number to decimal:

$= (1x2^3)+(1x2^2)+(0x2^1)+(1x2^0)$

$= 8 + 4 +0 +1$

$= 13_{10}$

Therefore $1101_2 = 13_{10}$.

Hexadecimal to Decimal

Convert $B2D_{16}$ to decimal.
Using the same method to convert hexadecimal to decimal.

$= (Bx16^2)+(2x16^1)+(Dx16^0)$

$= (11x16^2)+(2x16^1)+(13x16^0)$

$= 2816 + 32 +13$

$= 2861_{10}$

Therefore $B2D_{16} = 2861_{10}$.

The same method (multiplying each digit by it's column value) can be to convert any system to decimal.

Try these conversions to decimal WITHOUT USING YOUR CALCULATOR FOR THE ACTUAL CONVERSION.

110_2 67_8 AFC_{16} FC_{16}

How do you know if your answer is correct?

Convert your decimal answer back into its original format.

Converting from Decimal to any Radix

To convert a decimal integer number (a decimal number in which any fractional part is ignored) to any other radix, all that is needed is to continually divide the number by its radix, and with each division, write down the remainder. When read from bottom to top, the remainder will be the converted result.

Decimal to Octal

Example 1.2.2 Decimal to Octal Conversion

For example, to convert the decimal number 8610 to octal:

Divide 8610 by the system radix, which when converting to octal is 8. This gives the answer 10, with a remainder of 6.

Continue dividing the answer by 8 and writing down the remainder until the answer = 0

Now simply write out the remainders, starting from the bottom, to give 1268

Therefore 8610 = 1268

Decimal to Binary

Example 1.2.1 Decimal to Binary Conversion

This process also works to convert decimal to binary, but this time the system radix is 2:

For example, to convert the decimal number 1310 to binary:

Therefore 1310 = 11012

Decimal to Hexadecimal

Example 1.2.3 Decimal to Hexadecimal Conversion

It also works to convert decimal to hexadecimal, but now the radix is 16:

As some of the remainders may be greater than 9 (and so require their alphabetic replacement), you may find it easier to use Decimal for the remainders, and then convert them to Hex.

Therefore 286110 = B2D16

Numbers with Fractions

It is very common in the decimal system to use fractions; that is any decimal number that contains a decimal point, but how can decimal numbers, such as 34.62510 be converted to binary fractions?

In electronics this is not normally done, as binary does not work well with fractions. However as fractions do exist, there has to be a way for binary to deal with them. The method used is to get rid of the radix (decimal) point by normalizing the decimal fraction using floating point arithmetic. As long as the binary system keeps track of the number of places the radix point was moved during the normalization process, it can be restored to its correct position when the result of the binary calculation is converted back to decimal for display to the user.

However, for the sake of completeness, here is a method for converting decimal fractions to binary fractions. By carefully selecting the fraction to be converted, the system works, but with many numbers the conversion introduces inaccuracies; a good reason for not using binary fractions in electronic calculations.

Converting the Decimal Integer to Binary

Example 1.2.4 Converting the Integer to Binary

The radix point splits the number into two parts; the part to the left of the radix point is called the INTEGER. The part to the right of the radix point is the FRACTION. A number such as 34.62510 is therefore split into 3410 (the integer), and .62510 (the fraction).

To convert such a fractional decimal number to any other radix, the method described above is used to convert the integer.

So 3410 = 1000102

Converting the Decimal Fraction to Binary

Example 1.2.5 Converting the Fraction to Binary

To convert the fraction, this must be MULTIPLIED by the radix (in this case 2 to convert to binary). Notice that with each multiplication a CARRY is generated from the third column. The Carry will be either 1 or 0 and these are written down at the left hand side of the result. However when each result is multiplied the carry is ignored (don't multiply the carry). Each result is multiplied in this way until the result (ignoring the carry) is 000. Conversion is now complete.

For the converted value just read the carry column from top to bottom.

So 0.62510 = .1012

Therefore the complete conversion shows that 34.62510 = 100010.1012

However, with binary, there is a problem in using this method, .625 converted easily but many fractions will not. For example if you try to convert .626 using this method you would find that the binary fraction produced goes on to many, many places without a result of exactly 000 being reached.

With some decimal fractions, using the above method will produce carries with a repeating pattern of ones and zeros, indicating that the binary fraction will carry on infinitely. Many decimal fractions can therefore only be converted to binary with limited accuracy. The number of places after the radix point must be limited, to produce as accurate an approximation as required.

Quick Conversions

The most commonly encountered number systems are binary and hexadecimal, and a quick method for converting to decimal is to use a simple table showing the column weights, as shown in Tables 1.2.1a and 1.2.1b.

Converting Binary to Decimal

To convert from binary to decimal, write down the binary number giving each bit its correct 'weighting' i.e. the value of the columns, starting with a value of one for the right hand (least significant) bit. Giving each bit twice the value of the previous bit as you move left.

Bit	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value (weighting) of each bit	128	64	32	16	8	4	2	1
8 Bit Binary	0	1	0	0	0	0	1	1

Example:

To convert the binary number 010000112 to decimal. Write down the binary number and assign a 'weighting' to each bit as in Table 1.2.1a

Now simply add up the values of each column containing a 1 bit, ignoring any columns containing 0.

Applying the appropriate weighting to 01000011 gives 64 + 2 + 1 = 67

Therefore: 010000112 = 6710

Converting Hexadecimal to Decimal

A similar method can be used to quickly convert hexadecimal to decimal, using Table 1.2.1b

The hexadecimal digits are entered in the bottom row and then multiplied by the weighting value for that column.

Adding the values for each column gives the decimal value.

Therefore: 25CB16 = 967510

Column	16^3	16^2	16^1	16^0
Value (weighting) of each column	4096	256	16	1
Hex value	2	5	C	B

2 x 4096 = 8192
5 x 256 = 1280
C (12_{10}) x 16 = 192
B (11_{10}) x 1 = 11
9675

Binary and Hexadecimal

Converting between binary and hexadecimal is a much simpler process; hexadecimal is really just a system for displaying binary in a more readable form.

Binary is normally divided into Bytes (of 8 bits) it is convenient for machines but quite difficult for humans to read accurately. Hexadecimal groups each 8-bit byte into two 4-bit nibbles, and assigns a value of between 0 and 15 to each nibble. Therefore each hexadecimal digit (also worth 0 to 15) can directly represent one binary nibble. This reduces the eight bits of binary to just two hexadecimal characters.

Binary	Hex.
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Example:

111010012 is split into 2 nibbles 11102 and 10012 then each nibble is assigned a hexadecimal value between 0 and F.

The bits in the most significant nibble (11102) add up to 8+4+2+0 = 1410 = E16. The bits in the least significant nibble (10012) add up to 8+0+0+1 = 910 = 916

Therefore 111010012 = E916

Converting hexadecimal to binary of course simply reverses this process.

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. Electronics that operate digital signal is:
(a) Analog Electronics (b) Digital Electronics
(c) DC Electronics (d) AC Electronics

2. Which number system has a base 16?
(a) Binary (b) Hexa (c) Octal (d) decimal

3. The following hexadecimal number (1E.43)16 is equivalent to:
(a)(36.506)8 (b)(36.206)8 (c)$(35.506)_8$ d) (35.206)8

4. The value of radix in binary number system is:
(a) 2 (b) 8 (c) 10 (d) 1

5. The value of radix in decimal number system is:
(a) 10 (b) 2 (c) 3 (d) 4

6. The value of radix in hexa number system is:
(a) 2 (b) 8 (c) 10 (d) 16

7. The value of radix in octal number system is :
(a) 2 (b) 8 (c) 10 (d) 1

8. The binary equivalent of the decimal number 10 is:
(a) 0010 (b) 10 (c) 1010 (d) 010

9. The octal equivalent of 1100101.001010 is:
(a) 624.12 (b) 145.12 (c) 154.12 (d) 145.21

10. The input hexadecimal representation of 1110 is:
(a) 0111 (b)E (c)15 (d) 14

ANSWER KEY

1.	b	2.	b	3.	b	4.	a	5.	A
6.	d	7.	b	8.	c	9.	b	10.	B

PART-II SAMPLE SHORT QUESTIONS

1. What are analog quantities?
2. What are digital quantities?
3. What are the advantages of analog quantities?
4. What are the advantages of digital quantities?
5. What is the difference between analog and digital signal?
6. Define the term bit?
7. Define the term byte?

8. Define the term nibble?
9. Define the term word?
10. What is number system and also write their types?
11. What is decimal number system?
12. What is binary number system?
13. What is octal number system?
14. What is hexa decimal number system?
15. What is radix point?
16. What is exponent?
17. What is digital electronics?
18. What is floating point?
19. What is normalized form?
20. What are the dis advantages of analog signal?

PART-III SAMPLE LONG QUESTIONS

1. Explain the analog and digital signal with examples?
2. What are the main differences between analog and digital signal?
3. Explain how to convert binary number into octal and hexa decimal with example?
4. Explain how to convert hexa decimal number into decimal and binary number also explains it with examples?
5. How octal number is converted into binary and decimal?

CHAPTER NO. 02

LOGIC GATES

2.1 The AND Gate

An AND gate is a logic gate having two or more inputs and a single output. An AND gate operates on logical multiplication rules. In this gate, if either of the inputs is low (0), then the output is also low. If all of the inputs are high (1), then the output will also be high. An AND gate can have any number of inputs, although 2 input and 3 input AND gates are the most common.

There are many integrated circuits that work on this logic which we will revisit later on. Let us first get a better understanding of how the output of an AND Gate output behaves with respect to its inputs.

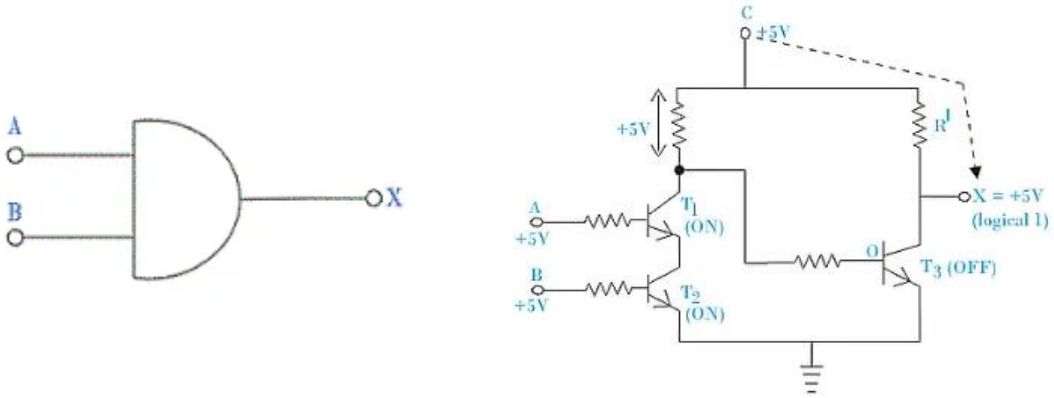
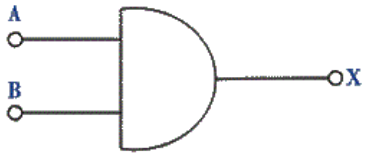


Figure 2.1:Symbol and circuit diagram of AND gate

There are two binary digits – 0 and 1. We just told that an AND gate performs a binary multiplication of binary digits. In multiplying 0 with 0 we will get 0, 1 with 0 or 0 with 1 we will get 0. Only we get 1 when 1 is multiplied by 1.

In other words, an AND gate is a digital device that produces high output only when all inputs are high and produces low output under all other inputs conditions. A high digital signal means logically 1 and a low digital signal means logically 0.An AND gate may have any number of input probes but

only one output probe. The symbol for a two-input AND gate is logically represented as:



Where A and B represent the input of the gate and X represents the output. A, B, and X can either be 0 (low) or 1 (high) logically.

The logical operation of AND gate hence can be represented as:

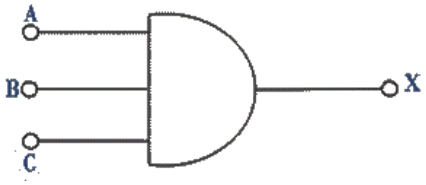
AB=X

All multiplication combinations of A and B can be represented in tabular form in a truth table. Truth tables list the output of a particular digital logic circuit for all the possible combinations of its inputs. The truth table of a 2 input AND gate can be represented as:

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

3 Inputs AND Gate

If instead of two inputs there are three inputs, this changes the logical symbol and truth table of the AND gate. The logical symbol of a 3-input AND gate is represented as:



The truth table of a 3 input AND gate is:

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

AND Gate Circuit Diagram

AND Gate Diode Circuit Diagram

Normally an AND gate is designed by either diodes or transistors.

When diodes are used to design AND gate, it is called a diode AND gate. The basic circuit of a diode AND gate is shown below

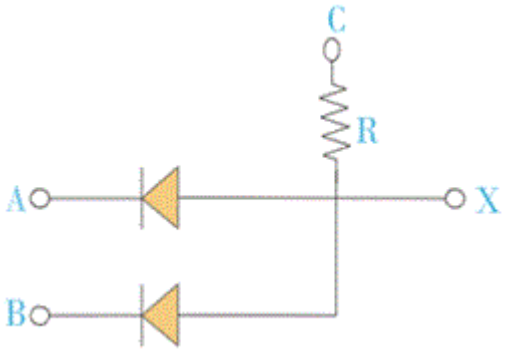


Figure 2.2:And gate circuit diagram using diode

In the above diode circuit, we first apply +5V at C. Now if we apply +5V at A and B, both of the diodes are reversed biased and hence behave both diodes as OFF or open circuit.

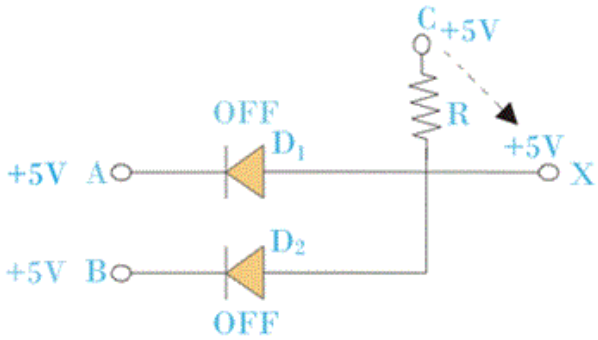


Figure 2.3:operation of And gate

In this situation as both diodes are OFF, no current will flow through resistor R and voltage of C (+5V) will also appear at X. As the supply voltage +5V appear at X, the output of the circuit is considered as high or logical 1.

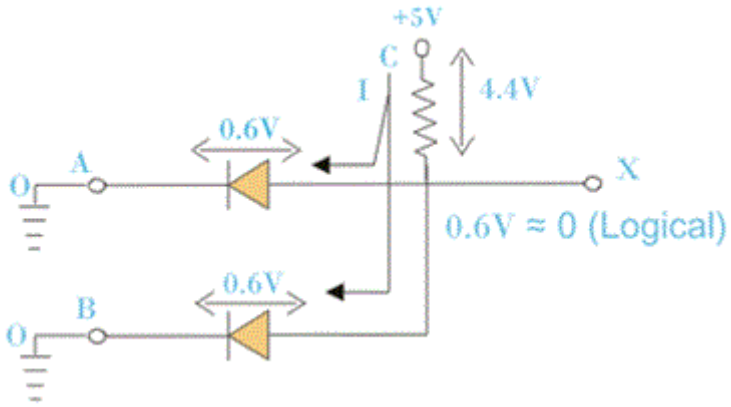


Figure 2.4:operation of and gate

Now, if either point A or B or both are applied with 0 Volt or they are grounded, the respective diode will become forward biased and hence behaves as 'ON' or short-circuited. In this condition, supply voltage +5V at point C will get a path through either of diodes or both to the ground potential.As the current flowing from C to the ground through resistor R, the entire 5V will be dropped across the resistor and hence the voltage at X will become low or logically zero.The diodes in the forwarding biased condition do not behave as an ideal short circuit; some voltage drop will be there across the forward biased diodes which are equal to forward bias voltage.

This voltage drop will appear at X during low output condition, so the practically low output will not be 0V it is rather 0.6 to 0.7V which is ideally considered as zero.

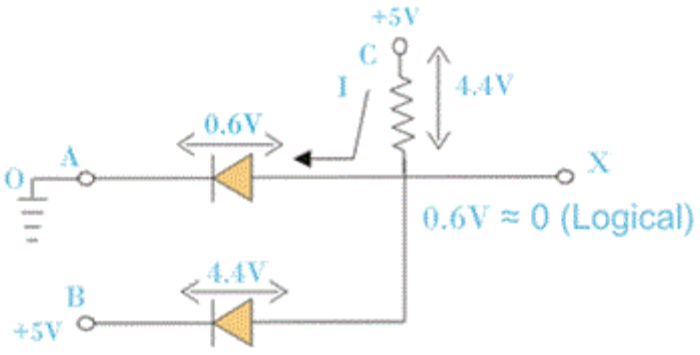


Figure 2.5;operation of and gate

AND Gate Transistor Circuit Diagram

An AND logic gate can also be realized from the transistor AND gate. The transistor circuit diagram for an AND gate is shown below:

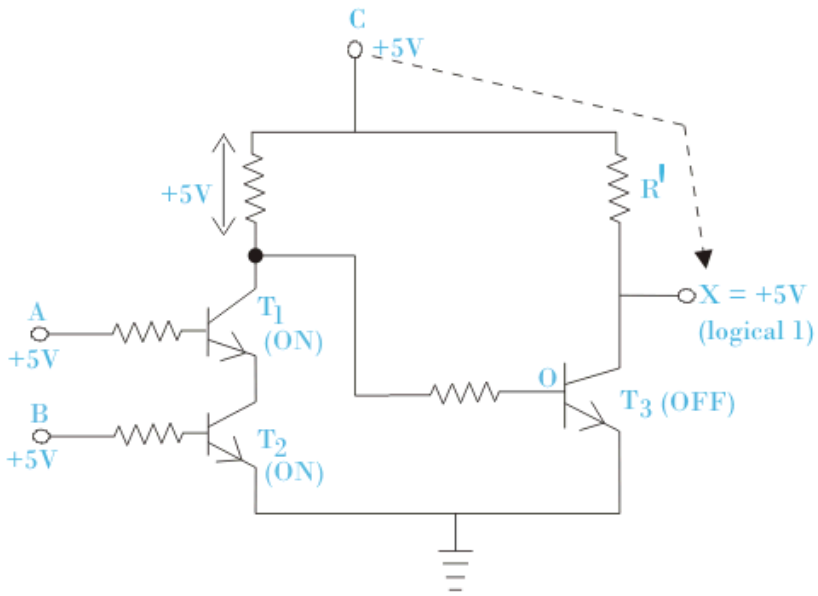


Figure 2.6:And gate transistor circuit

In the above circuit when A or B or both A and B are grounded or at 0V potential transistor T1 or T2 or both T1 and T2 are in OFF condition respectively.

This is because terminal A and B are base terminal of transistor T1 and T2 respectively. Zero base voltage makes a transistor OFF. As the path through T1and T2 is an open-circuited base of transistor T3 enough potential to makes T3 ON.

The current then starts flowing the supply to the ground through T3. As a result, the entire supply voltage will drop across R1, and the potential of terminal X will become low or logical zero.

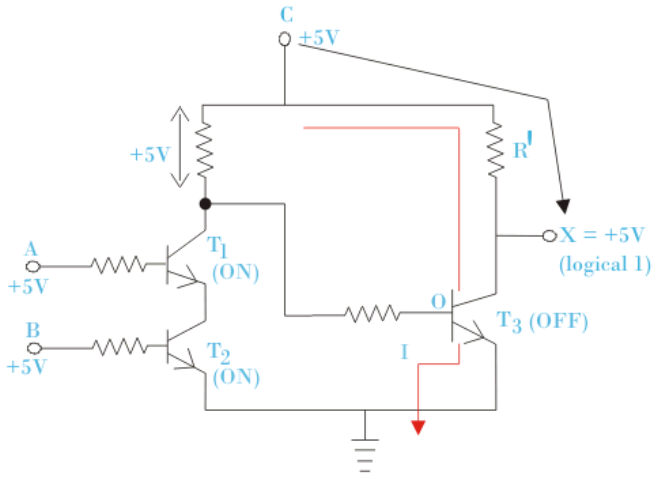


Figure 2.7:operation of and gate with transistor

If any of the transistors T1 and T2 are in OFF condition, the same result will come at output X as both the transistors are in series.

Now we will check what will be the logical value of X if both A and B are at high logical value. If we apply +5V at both A and B i.e. at the base of transistor T1 and T2 respectively.

This makes both the transistor T1 and T2 are in ON condition. Enter supply voltage will drop across R and the base potential of the transistor T3 will be zero and T3 becomes in OFF condition. As a result, the supply voltage +5V appears at X, and X will become logically 1 or high.

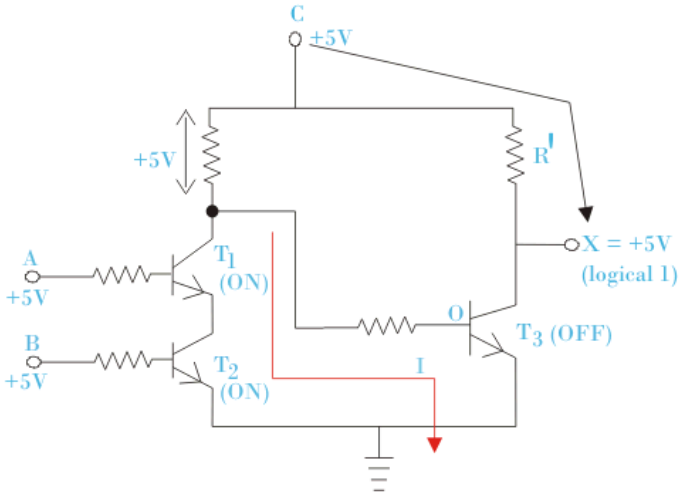


Figure 2.8:operation of transistor based and gate

AND Gate IC 7408

For AND Gate IC number in TTL is 7408. 7408 is Quad 2- input IC where four gates are present together. Let us have a look at the internal diagram of 7408.

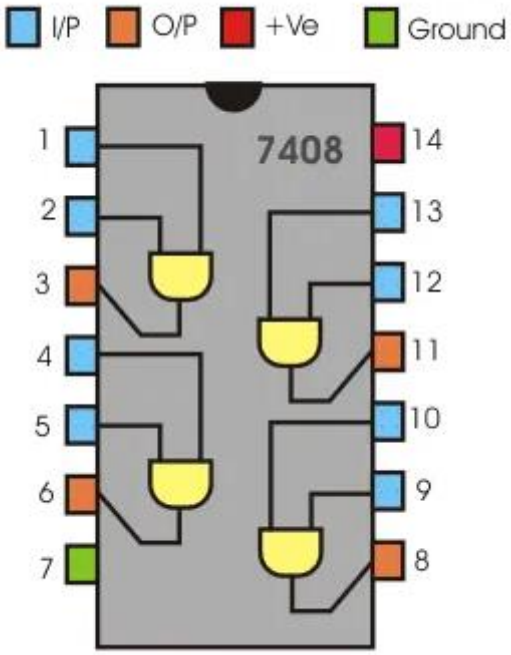


Figure 2.9:And gate IC

Here pins 1, 2 are the inputs of the first gate whose respective output is 3. Again 4 and 5 are the inputs of the second gate whose output is at pin 6. The inputs of the fourth are pin 12 and 13 and pin 11 is its output.Pin 14 is the supply input which can be a maximum of 5.2 volt D.C. if input voltage be more than this it may cause damage to the IC.

AND Gate IC 4081

In CMOS logic i.e complimentary MOSFET logic I.C number of AND Gate is 4081. This IC also has two inputs and one respective output. In this IC there are also 4 gates together. Now see the below internal diagram of this circuit to make it more clear.

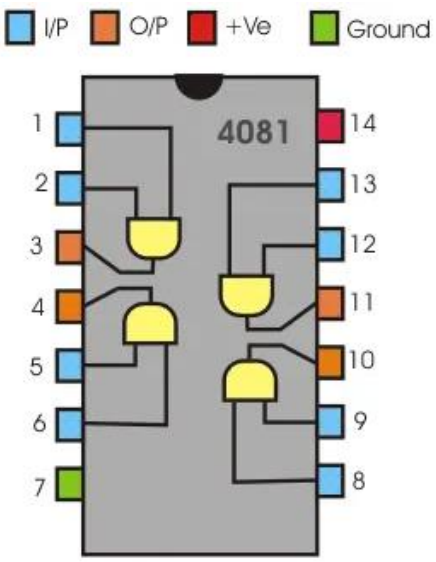


Figure 2.10 and gate ic 4081

Pin 1 and 2 are the inputs of the first gate whose output is in number 3. Again Pin 5 and 6 are the inputs of the second gate whose output is at Pin 4. pin number 7 is grounded. Pin 8 and 9 are the inputs of the third gate whose output is at pin 10.

Again pin 13 and 12 are the inputs of the fourth AND gate whose output is at pin number 11. Pin 13 and 12 are the inputs of fourth AND Gate whose output is at pin number 11. Pin number 14 is a power supply where a maximum 5.2-volt direct current supply can be given to activate the IC.

Here too if more voltage is given if may damage the IC. Inter circuit of CMOS and TTL differs from each other which must be noticed carefully.

2.2 OR GATE

An OR gate is a logic gate that performs logical OR operation. A logical OR operation has a high output (1) if one or both the inputs to the gate are high (1). If neither input is high, a low output (0) results.

Just like an AND gate, an OR gate may have any number of input probes but only one output probe.

The function of a logical OR gate effectively finds the maximum between two binary digits, just as the complementary AND function finds the minimum.



Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Figure 2.11: OR gate symbol and truth table

Truth tables list the output of a particular digital logic circuit for all the possible combinations of its inputs.

3 Input OR Gate

If instead of two inputs there are three inputs, this changes the logical symbol and truth table of the OR gate.

The logical symbol of a 3-input OR gate is represented as:



Inputs			Output
A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Figure 2.12:Symbol and truth table of 3-input or gate

OR Gate Circuit Diagram

Like AND gate and OR gate can also be realized by using a diode or transistor circuit. A simple two inputs OR gate can be realized by using a diode as follows,

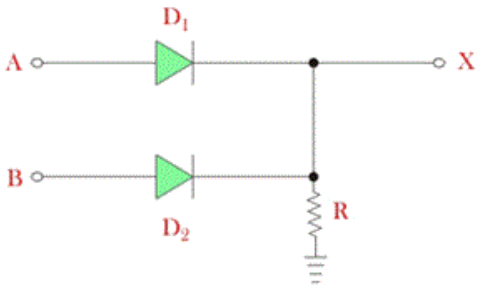


Figure 2.13:Circuit diagram of or gate

In the above circuit, if A and B are applied with 0V, there will be no voltage appears at X. When any of the inputs is given with +5V, the respective diode becomes forward biased and behaves as ideally short circuited hence this +5 V will appear at output X. +5 V means logical 1.

Actually entire 5V will not appear at X, around 0.6 to 0.7 V will drop across the diode as forwarding bios voltage, and the rest of the voltage i.e. 5 – 0.6 = 4.4 V or 5 – 0.7 = 4.3 V will appear at X. This 4.4 V or 4.3 V is practically considered as logical 1.

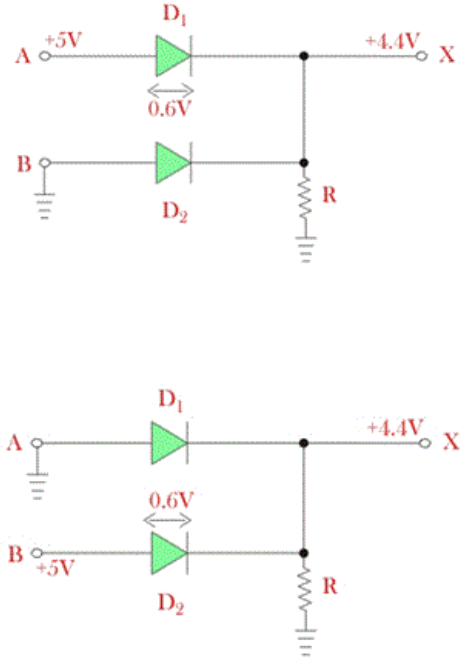


Figure 2.14:Analysis circuit of or gate

Now if both of the inputs are given with +5 V, both diodes will be forward biased. Hence, similarly, 4.4 V will appear at X.

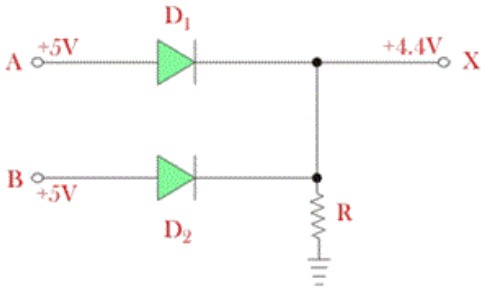


Figure 2.15:Analysis of or gate circuit

Now if both of the inputs A and B are grounded or given 0V, there will be no voltage appears at X and hence X is considered as logical 0.

OR Gate Transistor Circuit Diagram

The OR gate can also be realized by using a transistor. In this case, the OR gate is referred to as the transistor OR gate. Two inputs such OR gate is shown below,

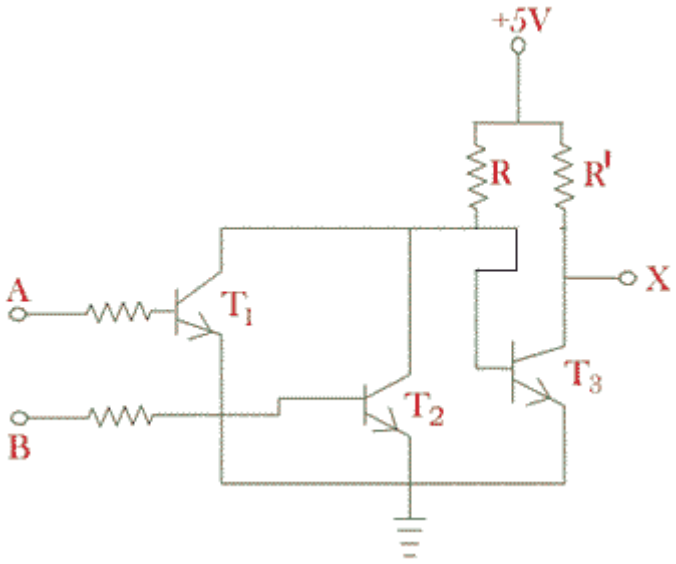


Figure 2.16:circuit diagram of transistor or gate

Now if A and B both are given with 0V, both of the transistors are in OFF condition, hence supply voltage + 5 V will not get the path to the ground through either of the transistors, T1 and T2. As a result base of the transistor T3 will get enough potential to make it ON. In that condition supply + 5 V will get the path to ground through resistor R' and transistor T3.

As the transistor T3 is in ON condition it will behave as ideally short circuited, hence the entire supply voltage + 5 V will drop across resistor R' and X terminal (Node) will get 0V. In practice, transistor T3 will not be ideal short circuited it will have some voltage drop across it which will be around 0.6 – 0.7 V. This voltage will appear at node X, and this 0.6 or 0.7 volt is considered as logical 0.

Now, if the base terminal either of the transistors T1 or T2 or both are given with + 5 V, the respective transistor as both will be in ON condition. In that case, supply voltage + 5 V will get the path to ground through either of the transistors or both.

As a result, current starts flowing to the ground from supply through this path, and the entire supply voltage will drop across resistor R. So, the base of transistor T3 will not get sufficient potential to make the transistor T3 ON. Hence entire supply voltage will appear at X and the X becomes at a high logical state or logical 1.

OR Gate Integrated Circuits

OR gate is available in TTL and CMOS ICS, Where TTL is Transistor – Transistor Logic ICs and CMOS are Complementary MOSFET ICS. We will now discuss the arrangement of the OR gate on both the ICs.

In CMOS ICs the 4000 series is available. In digital electronic 4071 is the CMOS IC number of inputs in each gate is two. 4075 is 3 – input OR gate and 4072 is 4 – input OR gate in CMOS ICs. An internal gate picture of 4071 can help you understand this IC.

OR Gate IC 4071

4071 is a 14 pin 1C as you can see where four or gates are fixed together having two inputs. Pin number 14 is tvcc where maximum SV DC supply is given which activates the IC. Pin number 7 is grounded. Pin 1 and 2 are the input for the first gate and 3 is the respective output for the first gate. Pin numbers 5 and 6 are inputs for the second gate whose output is pin number 4. For the third gate pin number 8 and 9 are the inputs whose output is pin number 10 for the last gate pin number 12 and 13 are the inputs whose respective output is at pin number 11. Thus how the IC looks internally for CMOS.

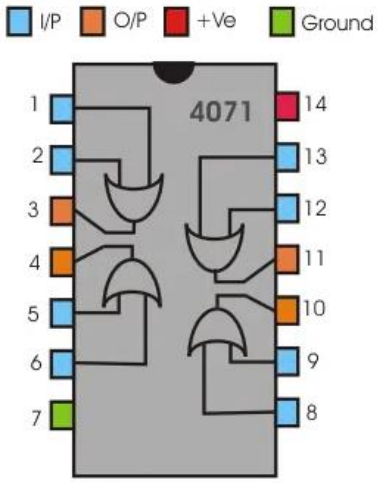


Figure 2.17:OR gate IC 4071

2.3 NOT Gate

A NOT gate is a logic gate that inverts the digital input signal. For this reason, a NOT gate is sometimes is referred to as an inverter (not to be confused with a power inverter). A NOT gate always has high (logical 1) output when its input is low (logical 0). Conversely, a logical NOT gate always has low (logical 0) output when the input is high (logical 1). The logical symbol for a NOT gate is shown below:

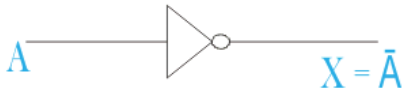


Figure 2.18:symbol of NOT gate

If the input binary variable of a NOT gate is considered as A, then the output binary variable of the gate will be Ā. As the symbol of not operation is (–) bar. If the value of A is 1.then Ā = 0 and in the opposite if the value of A is then Ā = 1.

NOT Gate Transistor Circuit Diagram

A NOT gate can easily be realized by using a simple bipolar transistor. The transistor circuit diagram of a NOT gate (also known as a transistor inverter) is shown below:

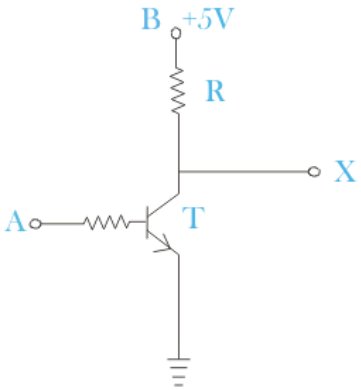
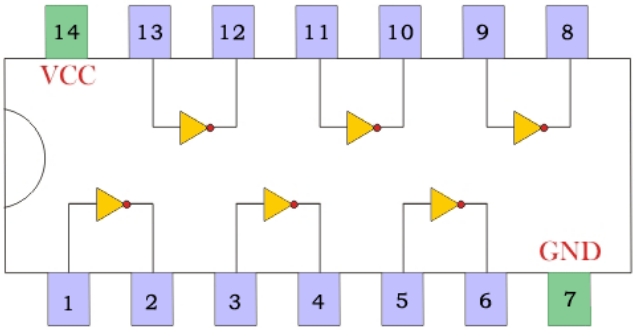


Figure 2.19:Transistor based NOT Gate

NOT Gate IC

The IC available at the market for NOT gate is IC 7404. One 7404 IC contains a total of six transistors inverter or simply six NOT gate



7404 Six Inverter

Figure 2.20:Not gate IC

2.4 NAND Gate

A NAND gate ("not AND gate") is a logic gate that produces a low output (0) only if all its inputs are true, and high output (1) otherwise. Hence the NAND gate is the inverse of an AND gate, and its circuit is produced by connecting an AND gate to a NOT gate. Just like an AND gate, a NAND gate may have any number of input probes but only one output probe.

The NAND gate performs the logical NAND operation. NAND gates are known as universal gates (along with NOR gates), which means they are a type of logic gate which can implement any Boolean function without the need to use any other gate type.

The basic logical construction of the NAND gate is shown below (you can see it is an AND gate followed by a NOT.

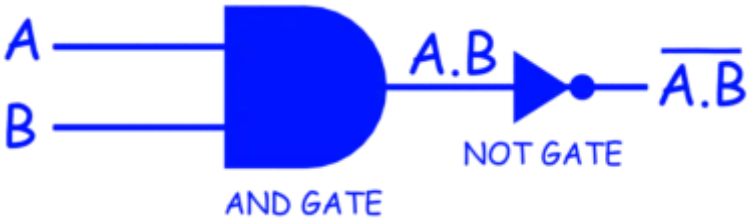


Figure 2.21:NAND gate symbol

The symbol of a NAND gate is similar to the AND gate, but a bubble is drawn at the output point of the AND gate.

NAND Gate Truth Table

NAND gate means "not AND gate", hence the output of this gate is just reverse of that of a similar AND gate.
We know that the output of the AND gate is only high or 1 when all the inputs are high or 1. In all other cases, the output of the AND gate is low or 0.
In the NAND, the fact is the opposite, here, the output is only logical 0 when and only when all inputs of the gate are 1s, and in all other cases, the output of the NAND gate is high or 1.
Hence, the truth table of a 2 input NAND gate can be represented as:

Inputs		Output
A	B	$X = \overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

Figure 2.22:Truth Table of Nand Gate

2.5 NOR Gate

A NOR gate ("not OR gate") is a logic gate that produces a high output (1) only if all its inputs are false, and low output (0) otherwise. Hence the NOR gate is the inverse of an OR gate, and its circuit is produced by connecting an OR gate to a NOT gate. Just like an OR gate, a NOR gate may have any number of input probes but only one output probe. A NOT gate followed by an OR gate makes a NOR gate. The basic logic construction of the NOR gate is shown below:



Figure 2.23:Symbol of NOR Gate

The symbol of the OR gate is similar to the OR gate but one bubble is drawn at the output point of the OR gate in the case of the OR gate.

NOR gate means “not an OR gate” hence the output of this logic gate is just the reverse of that of an OR gate.

We know that the output of an OR gate is 0 only when all inputs of the OR gate are 0. Hence in the case of the NOR gate, the output is 1 only when all inputs are 0. In all other cases, that is for all other combinations of inputs the output is 0.

Truth tables list the output of a particular digital logic circuit for all the possible combinations of its inputs. The truth table of a 2 input NOR gate can be represented as:

Inputs		Output
A	B	$X = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Inputs		Outputs
A	B	$X = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Figure 2.24:Truth Table of NOR and OR

Like the OR gate, the NOR gate may also have more than two inputs.

A NOR gate is also referred to as a universal gate. Because all binary operations can be realized by only using NOR gates.

As we know that there are only three basic operations AND, OR, and NOT. Also, we know that all complex binary operations can be realized by using these three basic operations.

If we can prove that AND, OR, and NOT operations can be realized by using only the NOR gate, then we can easily say that all binary operations can be realized by using only NOR gates.

OR Gate Using NOR Gates

NOR gate is an inverted OR gate, so the inverted NOR gate will be an OR gate. A NOT gate followed by a NOR gate will be equivalent to an OR gate. Here we will use the NOT gate which is realized from another NOR gate. The logic circuit of this OR gate will be as shown below,

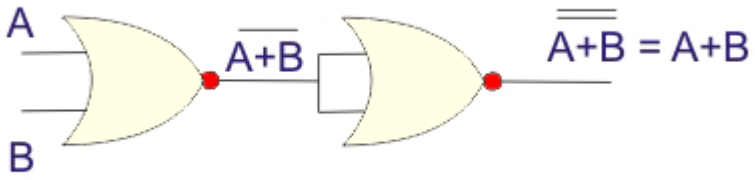


Figure 2.25:OR gate from NOR gate

AND Gate from NOR Gate

The NOR operation of two input NOR gate is, Now, according to D.Morgan's theorem,

$$\overline{A+B} = \overline{A}.\overline{B}$$

Now, if we put inverted A instead of A and inverted B instead of B in the above equation, we will get,

$$\overline{\overline{A}+\overline{B}} = \overline{\overline{A}}.\overline{\overline{B}} = A.B$$

This is an AND operation.

So from the above equation, it is clear that if inputs of a NOR gate are inverted then the output will be AND equivalent.

So, if we connect the NOT gate at both inputs of a NOR gate, we will get an AND gate equivalent. As we are only allowed to use the NOR gate, we will use here the NOT gates which are realized from the NOR gate. The logical equivalent circuit will be as shown below,

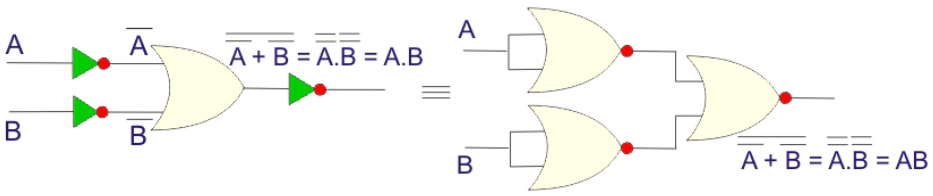


Figure 2.26:And Gate from NOR gate

Now we have proved that all three basic logical gates, NOT, OR, and AND gates can be realized by using only NOR gates. So any even much complex binary operation can be realized by using only a NOR gate hence it is quite justified to cal a NOR gate as a universal logic gate.

NOR Gate Integrated Circuits

There are many NOR gate Integrated Circuits (ICs) available in the market. A few of the most popular include:

IC 7402 contains four two inputs NOR gates.

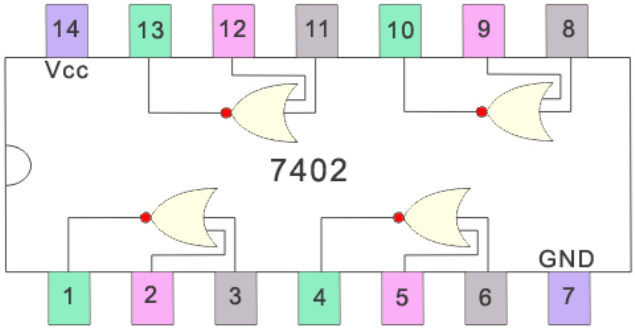


Figure 2.27:IC of NOR Gate

2.6 XOR Gate

An XOR gate (also known as an EOR, or EXOR gate) – pronounced as Exclusive OR gate – is a digital logic gate that gives a true (i.e. a HIGH or 1) output when the number of true inputs is odd. An XOR gate implements an exclusive OR, i.e., a true output result occurs if one – and only one – of the gate's inputs is true. If both inputs are false (i.e. LOW or 0) or both inputs are true, the output is false.

XOR represents the inequality function, i.e. the output is true if the inputs are not alike; otherwise, and the output is Another way to look at an XOR gate: a modulo sum of two variables in a binary system looks like this:

The logic gate performs this modulo sum operation without including carry is known as XOR gate. An XOR gate is normally two inputs logic gate where the output is only logical 1 when only one input is logical 1. When both inputs are equal, either are 1 or both are 0, the output will be logical 0.

This is the reason an XOR gate is also called an anti-coincidence gate or inequality detector. This gate is called XOR or exclusive OR gate because its output is only 1 when its input is exclusively 1.

In digital electronics, other logic gates include NOT gates, OR gates, NAND gates, and NOR Gates.false. A common way to remember the XOR is "must have one or the other, but not both".

Another way to look at an XOR gate: a modulos sum of two variables in a binary system looks like this:

The logic gate performs this modulo sum operation without including carry is known as XOR gate. An XOR gate is normally two inputs logic gate where the output is only logical 1 when only one input is logical 1. When both inputs are equal, either are 1 or both are 0, the output will be logical 0.

This is the reason an XOR gate is also called an anti-coincidence gate or inequality detector. This gate is called XOR or exclusive OR gate because its output is only 1 when its input is exclusively 1.

In digital electronics, other logic gates include NOT gates, OR gates, NAND gates, and NOR Gates.

XOR Gate Truth Table

Truth tables list the output of a particular digital logic circuit for all the possible combinations of its inputs. The truth table of an XOR gate is given below:

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

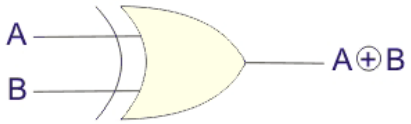


Figure 2.28:Truth Table and Symbol of XOR

XOR Gate Circuit Diagram

The above expression, A ⊕ B can be simplified as,

$$A \oplus B = A\overline{B} + \overline{A}B$$

Let us prove the above expression.

In first case consider, A = 0 and B = 0.

$$\therefore A \oplus B = 0 \oplus 0 = 0.\overline{0} + \overline{0}.0 = 0.1 + 1.0 = 0$$

In second case consider, A = 0 and B = 1.

$$\therefore A \oplus B = 1 \oplus 1 = 1.1 + 1.1 = 1.0 + 0.1 = 0$$

In third case consider, A = 1 and B = 0.

$$\therefore A \oplus B = 1 \oplus 0 = 1.\overline{0} + \overline{1}.0 = 1.1 + 0.0 = 1$$

In fourth case consider, A = 1 and B = 1.

$$\therefore A \oplus B = 1 \oplus 1 = 1.\overline{1} + \overline{1}.1 = 1.0 + 0.1 = 0$$

So it is proved that the Boolean expression for A ⊕ B is $A\overline{B}$ + $\bar{A}B$, as this Boolean expression satisfied all output states with respect to an **XOR gate's inputs conditions**.

From this Boolean expression, one can easily realize the logical circuit of an XOR gate, and this is shown below

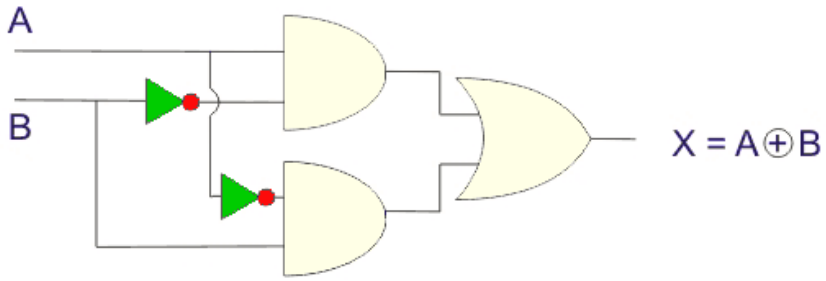


Figure 2.30:Logic diagram of XOR Gate

2.7 XNOR Gate

The XNOR gate (also known as a XORN'T, ENOR, EXNOR or NXOR) – and pronounced as Exclusive NOR – is a digital logic gate whose function is the logical complement of the exclusive OR gate (XOR gate). Logically, an XNOR **gate** is a NOT gate followed by an XOR gate.

The XOR operation of inputs A and B is A ⊕ B; therefore, XNOR operation those inputs will be $(A + B)\overline{}$.

That means the output of the XOR gate is inverted in the XNOR gate.In the XOR gate operation, the output is only 1 when only one input is 1. The output is logical 0 when both inputs are the same, meaning they are either 1 or 0. But in the XNOR gate, the inverse is true. Hence the output is 0 when only one input is 0, and the output is 1 when both inputs are the same (i.e. either two 0's or two 1's).

See the XNOR gate truth table below for a visual representation of this.

XNOR Gate Truth Table

The truth table of the **XNOR gate** is shown below:

The logical XNOR operation is represented by ⊙. That is a dot surrounded by a circle. The expression of XNOR operation between variables A and B is represented as A ⊙ B.

Now again, the truth table is satisfied by the equation $AB + \bar{A}\bar{B}\overline{}$

$$When, A = 0, \ and \ B = 0 \ , AB + \overline{A} \ \overline{B} = 0.0 + \overline{0}. \ \overline{0} = 0.0 + 1.1 = 1.$$
$$When, A = 0, \ and \ B = 1, AB + \overline{A} \ \overline{B} = 0.1 + \overline{0} \ . \overline{1} = 0.1 + 1.0 = 0.$$
$$When, A = 1, \ and \ B = 0, AB + \overline{A} \ \overline{B} = 1.0 + \overline{1} \ . \overline{0} = 1.0 + 0.1 = 0.$$
$$When, A = 1, \ and \ B = 1, AB + \overline{A} \ \overline{B} = 1.1 + \overline{1} \ . \overline{1} = 1.1 + 0.0 = 1.$$

Hence, it is proved that A ⊙ B = AB + $\bar{A}\bar{B}$. The same can be proved by using K-map also.

XNOR Gate Circuit Diagram

The expression of XNOR operation can be realized by using two NOT gates, two AND gates, and one OR gate as followers,

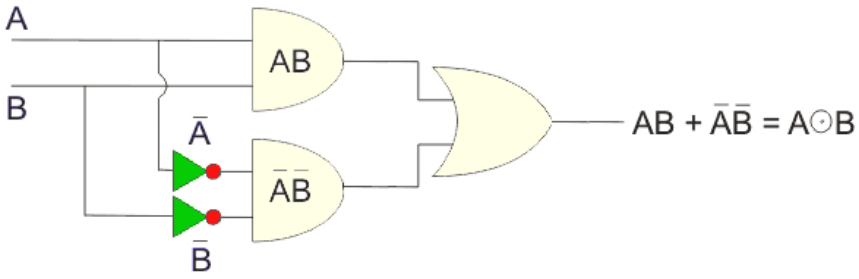
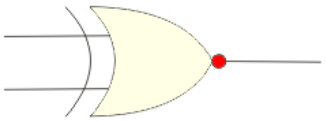


Figure 2.31:Logic diagram of XNOR Gate

The symbol of the XNOR gate is



3.Input XNOR Gate

Like the XOR gate, an XNOR gate only exists with two inputs, but for XNOR operation with more than two inputs, we have to use more than one XNOR gates.

XNOR operation with more than two inputs is like that. When there are odd numbers of inputs in high or logical 1 condition, the output will be 0; otherwise, the output will be 1.

Inputs			Output
A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

A	B	A ⊙ B	C	(A ⊙ B) ⊕ C
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
1	0	0	0	0
1	0	0	1	1
1	1	1	0	1
1	1	1	1	0

Figure 2.32:Truth table of 3 input XNOR Gate

From this elaborate truth table, the logical symbol of three inputs XNOR gate can be represented as

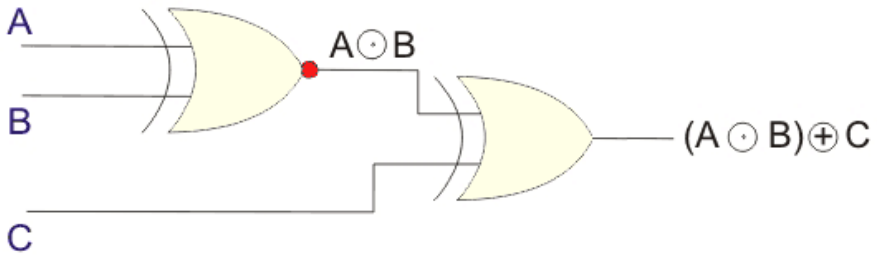


Figure 2.33:Logic diagram of XNOR Gate

Applications of XOR Gates

The main application of the Exclusive OR gate is in the operation of half and full adder. If we look at the truth table carefully, we will find that the first three results are totally satisfying the process of binary addition. Still, in the last input sequence i.e. when both the inputs are 1, the result according to the rule of addition should be 0 with a carry 1. In the truth table, we are getting the desired 0 but a missing 1.

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. The universal gate is:
(a) And gate (b) OR gate
(c) NAND (d) NOT

2. The inverter gate is:
(a) And gate (b)NOT (c)OR (d)X-OR

3. The NOR gate is OR gate followed by
(a) And (b) Nand (c) NOT (d) None

4. The NAND gate is AND gate followed by:
(a) And (b) Nand (c) NOT (d) None

5. Digital circuit can be made by the repeated use of:
(a) And (b) Nand (c) NOT (d) None

6. A single transistor can be used to build ….
(a) And (b) Nand (c) NOT (d) None

7. ____truth table entries are necessary for a four-input circuit.
(a) 4 (b)8 (c)12 (d)16

8. Which of gate will give a 0 when both of its inputs are 1?
(a) AND (b) OR (c) NAND (d) EXOR

9. The dependency notation ">=1" inside a block stands for which operation?:
(a) AND (b) OR (c) NAND (d) EXOR

10. How many NAND circuits are contained in a 7400 NAND IC?
(a) 1 (b) 2 (c) 4 (d) 8

ANSWER KEY

1.	c	2.	b	3.	C	4.	c	5.	b
6.	a	7.	d	8.	A	9.	b	10.	C

PART-II SAMPLE SHORT QUESTIONS

1. Define the term logic gates?
2. What is And gate?
3. What is OR gate?
4. What is NOT gate?
5. What is NAND gate?

6. Define the gate NOR?
7. What is X-OR gate?
8. What is X-NOR gate?
9. Draw the truth table of And gate?.
10. Draw the truth table of OR gate?
11. Draw the truth table of X-OR?
12. Draw the truth table of X-NOR?
13. Draw the truth table of NOR gate?
14. How to draw OR gate with diode?
15. How to make NOT gate with transistor?
16. How to make OR gate using NOR gate?
17. What is De-morgan law?
18. Draw 3 input X-NOR?
19. Draw 3 input X-OR?
20. What are the applications of X-OR gate?

PART-III SAMPLE LONG QUESTIONS

1. What are the logic gates also draw the symbols of all gates?.
2. Write down the applications of X-OR gate?
3. Explain the topic OR gate also make or gate using diode?
4. What is NOT gate also draw it with transistor?
5. Explain AND gate also draws it with both diode and transistor?

CHAPTER NO. 03

BOOLEAN ALGEBRA

3.1 Introduction

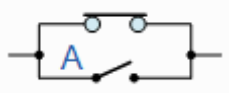
As well as the logic symbols “0” and “1” being used to represent a digital input or output, we can also use them as constants for a permanently “Open” or “Closed” circuit or contact respectively.

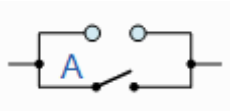
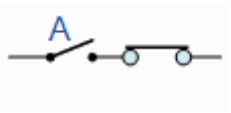
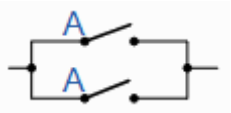
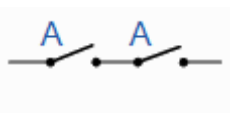
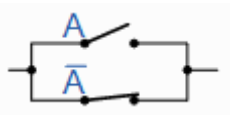
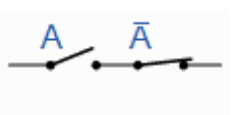
A set of rules or Laws of Boolean Algebra expressions have been invented to help reduce the number of logic gates needed to perform a particular logic operation resulting in a list of functions or theorems known commonly as the Laws of Boolean Algebra.

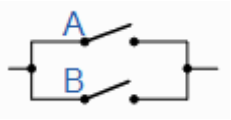
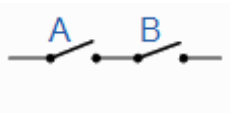
Boolean Algebra is the mathematics we use to analyse digital gates and circuits. We can use these “Laws of Boolean” to both reduce and simplify a complex Boolean expression in an attempt to reduce the number of logic gates required. Boolean algebra is therefore a system of mathematics based on logic that has its own set of rules or laws which are used to define and reduce Boolean expressions.

The variables used in Boolean Algebra only have one of two possible values, a logic “0” and a logic “1” but an expression can have an infinite number of variables all labeled individually to represent inputs to the expression, For example, variables A, B, C etc, giving us a logical expression of A + B = C, but each variable can ONLY be a 0 or a 1.Examples of these individual laws of Boolean, rules and theorems for Boolean Algebra are given in the following table.

Truth Tables for the Laws of Boolean

Boolean Expression	Description	Equivalent Switching Circuit	Boolean Algebra Law or Rule
A + 1 = 1	A in parallel with closed = “CLOSED”		Annulment

A + 0 = A	A in parallel with open = “A”		Identity
A . 1 = A	A in series with closed = “A”		Identity
A . 0 = 0	A in series with open = “OPEN”		Annulment
A + A = A	A in parallel with A = “A”		Idempotent
A . A = A	A in series with A = “A”		Idempotent
NOT A = A	NOT NOT A (double negative) = “A”		Double Negation
A + A = 1	A in parallel with NOT A = “CLOSED”		Complement
A . A = 0	A in series with NOT A = “OPEN”		Complement

A+B = B+A	A in parallel with B = B in parallel with A		Commutative
A.B = B.A	A in series with B = B in series with A		Commutative
A+B = A.B	invert and replace OR with AND		de Morgan's Theorem
A.B = A+B	invert and replace AND with OR		de Morgan's Theorem

The basic Laws of Boolean Algebra that relate to the Commutative Law allowing a change in position for addition and multiplication, the Associative Law allowing the removal of brackets for addition and multiplication, as well as the Distributive Law allowing the factoring of an expression, are the same as in ordinary algebra.

Each of the Boolean Laws above are given with just a single or two variables, but the number of variables defined by a single law is not limited to this as there can be an infinite number of variables as inputs too the expression. These Boolean laws detailed above can be used to prove any given Boolean expression as well as for simplifying complicated digital circuits.

A brief description of the various Laws of Boolean are given below with A representing a variable input

3.2 Laws of Boolean Algebra

1. Annulment Law – A term AND'ed with a "0" equals 0 or OR'ed with a "1" will equal 1

A . 0 = 0 A variable AND'ed with 0 is always equal to 0

A + 1 = 1 A variable OR'ed with 1 is always equal to 1

2. Identity Law – A term OR‘ed with a “0” or AND‘ed with a “1” will always equal that term
   A + 0 = A A variable OR’ed with 0 is always equal to the variable
   A . 1 = A A variable AND’ed with 1 is always equal to the variable
3. Idempotent Law – An input that is AND‘ed or OR´ed with itself is equal to that input
   A + A = A A variable OR’ed with itself is always equal to the variable
   A . A = A A variable AND’ed with itself is always equal to the variable
4. Complement Law – A term AND‘ed with its complement equals “0” and a term OR´ed with its complement equals “1”
   A . A = 0 A variable AND’ed with its complement is always equal to 0
   A + A = 1 A variable OR’ed with its complement is always equal to 1
5. Commutative Law – The order of application of two separate terms is not important
   A . B = B . A The order in which two variables are AND’ed makes no difference
   A + B = B + A The order in which two variables are OR’ ed makes no difference
6. Double Negation Law – A term that is inverted twice is equal to the original term
   A = A A double complement of a variable is always equal to the variable
7. de Morgan’s Theorem – There are two “de Morgan’s” rules or theorems,
   (1) Two separate terms NOR‘ed together is the same as the two terms inverted (Complement) and AND‘ed for example: A+B = A . B
   (2) Two separate terms NAND‘ed together is the same as the two terms inverted (Complement) and OR‘ed for example: A.B = A + B

3.3 Boolean Algebra Function

Using the information above, simple 2-input AND, OR and NOT Gates can be represented by 16 possible functions as shown in the following table.

Function	Description	Expression
1.	NULL	0
2.	IDENTITY	1
3.	Input A	A
4.	Input B	B
5.	NOT A	A
6.	NOT B	B
7.	A AND B (AND)	A . B
8.	A AND NOT B	A . B
9.	NOT A AND B	A . B
10.	NOT AND (NAND)	A . B
11.	A OR B (OR)	A + B
12.	A OR NOT B	A + B

13.	NOT A OR B	A + B
14.	NOT OR (NOR)	A + B
15.	Exclusive-OR	A . B + A . B
16.	Exclusive-NOR	A . B + A . B

3.3 De Morgan's Theory

De Morgan's Theorems are basically two sets of rules or laws developed from the Boolean expressions for AND, OR and NOT using two input variables, A and B. These two rules or theorems allow the input variables to be negated and converted from one form of a Boolean function into an opposite form.

De Morgan's first theorem states that two (or more) variables NOR´ed together is the same as the two variables inverted (Complement) and AND, while the second theorem states that two (or more) variables NAND´ed together is the same as the two terms inverted (Complement) and OR´ed. That is replace all the OR operators with AND operators, or all the AND operators with an OR operators.

3.3.1 De Morgan's First Theorem

De Morgan's First theorem proves that when two (or more) input variables are AND'ed and negated, they are equivalent to the OR of the complements of the individual variables. Thus the equivalent of the NAND function will be a negative-OR function, proving that A.B = A+B. We can show this operation using the following table.

De Morgan's First Theorem using Truth Table

Inputs		Truth Table Outputs For Each Term				
B	A	A.B	A.B	A	B	A + B

0	0	0	**1**	1	1	**1**
0	1	0	**1**	0	1	**1**
1	0	0	**1**	1	0	**1**
1	1	1	**0**	0	0	**0**

We can also show that A.B = A+B using logic gates as shown.De Morgan's First Law Implementation using Logic Gates.

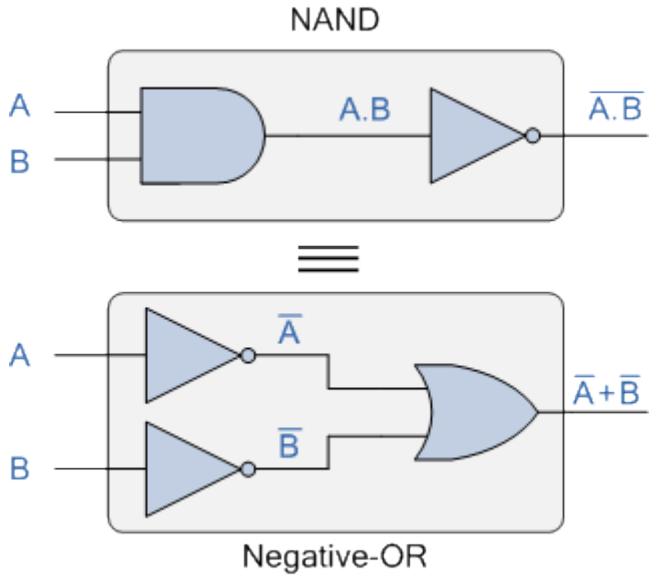


Figure 3.1:First Law implementation

The top logic gate arrangement of: A.B can be implemented using a standard NAND gate with inputs A and B. The lower logic gate arrangement first inverts the two inputs producing A and B. These then become the inputs to the OR gate. Therefore the output from the OR gate becomes: A+B

Then we can see here that a standard OR gate function with inverters (NOT gates) on each of its inputs is equivalent to a NAND gate function. So an individual NAND gate can be represented in this way as the equivalency of a NAND gate is a negative-OR.

3.3.2 De Morgan's Second Theorem

De Morgan's Second theorem proves that when two (or more) input variables are OR'ed and negated, they are equivalent to the AND of the complements of the individual variables. Thus the equivalent of the NOR function is a negative-AND function proving that A+B = A.B, and again we can show operation this using the following truth table.

De Morgan's Second Theorem using Truth Table

Inputs		Truth Table Outputs For Each Term				
B	A	A+B	A+B	A	B	A . B
0	0	0	**1**	1	1	**1**
0	1	1	**0**	0	1	**0**
1	0	1	**0**	1	0	**0**
1	1	1	**0**	0	0	**0**

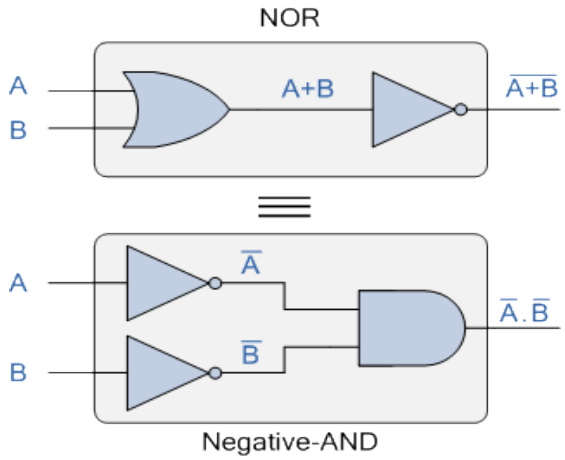


Figure 3.2:logic implementation of DE Morgan second law

The top logic gate arrangement of: A+B can be implemented using a standard NOR gate function using inputs A and B.

The lower logic gate arrangement first inverts the two inputs, thus producing A and B. Thus then become the inputs to the AND gate. Therefore the output from the AND gate becomes: A.B

Then we can see that a standard AND gate function with inverters (NOT gates) on each of its inputs produces an equivalent output condition to a standard NOR gate function, and an individual NOR gate can be represented in this way as the equivalency of a NOR gate is a negative-AND.

Although we have used De Morgan's theorems with only two input variables A and B, they are equally valid for use with three, four or more input variable expressions, for example:

For a 3-variable input

$$A.B.C = A+B+C$$

also

$$A+B+C = A.B.C$$

For a 4-variable input

$$A.B.C.D = A+B+C+D$$

and also

$$A+B+C+D = A.B.C.D$$

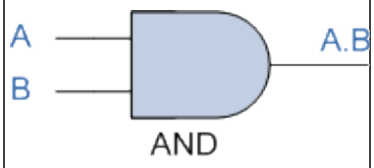
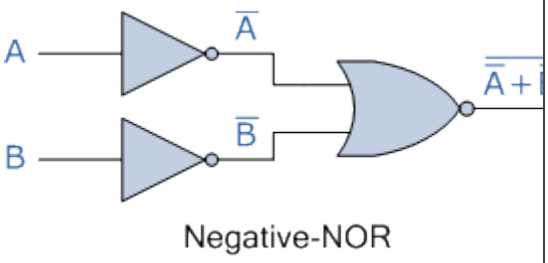
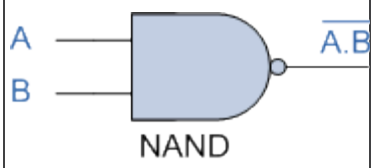
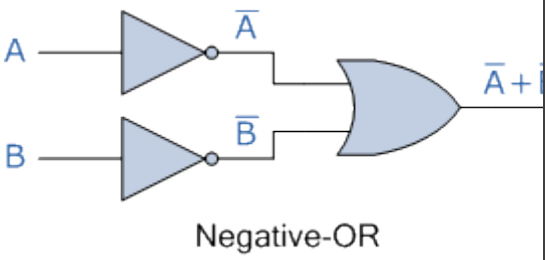
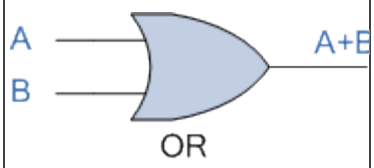
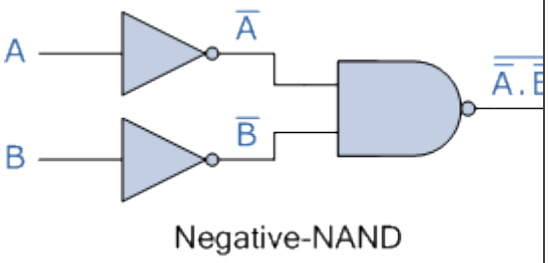
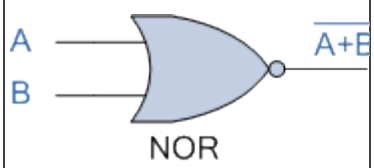
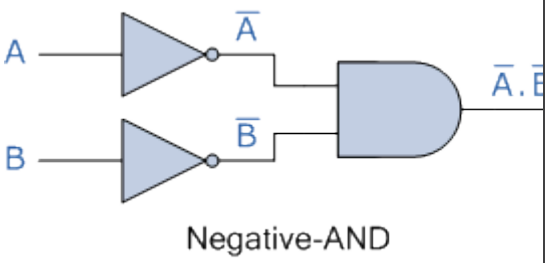
and so on.

3.3.3 De Morgan's Equivalent Gates

We have seen here that by using De Morgan's Theorems we can replace all of the AND (.) operators with an OR (+) and vice versa, and then complements each of the terms or variables in the expression by inverting it, that is 0's to 1's and 1's to 0's before inverting the entire function.

Thus to obtain the De Morgan equivalent for an AND, NAND, OR or NOR gate, we simply add inverters (NOT-gates) to all inputs and outputs and change an AND symbol to an OR symbol or change an OR symbol to an AND symbol as shown in the following table.

DE Morgan Equivalent Gate

Standard Logic Gate	De Morgan's Equivalent Gate
	
	
	
	

Then we have seen in this tutorial about De Morgan's Theorem that the complement of two (or more) AND'ed input variables is equivalent to the OR of the complements of these variables, and that the complement of two (or more) OR'ed variables is equivalent to the AND of the complements of the variables as defined by De Morgan.

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. The expression for Absorption law is given by:
(a) A + AB = A (b) A + AB = B (c) AB + AA' = A(d) A+B = B + A

2. According to boolean law: A + 1 = ?:
(a) 1 (b) A (c) 0 (d) A'

3. The involution of A is equal to _____:
(a) A (b) A' (c) 1 (d)0

4. A(A + B) = ?:
(a) AB (b) 1 (c) (1+AB) (d) A

5. DeMorgan's theorem states that _:
(a) (AB)' = A' + B'(b) (A + B)'=A'* B(c) A' + B'=A'B'(d) (AB)'=A'+B

6. (A + B)(A' * B') = ?:
(a) 1 (b)0 (c)AB (d) AB'

7. Complement of the expression A'B + CD' is:
(a) (A'+ B)(C'+ D)(b) (A+B')(C'+D) c) (A'+B)(C'+D)(d)(A+B')(C+D')

8. The logical sum of two or more logical product terms is called:
(a) SOP (b)POS (c)OR operation (d)Nand

9. In which of the following gates the output is 1 if and only if at least one input is 1?
(a) AND (b)NOR (c) NAND (d)OR

10. Odd parity of word can be conveniently tested by:
(a) OR gate (b) And gate (c)Nand gate (d) X-OR

ANSWER KEY

1.	a	2.	a	3.	a	4.	d	5.	A
6.	b	7.	b	8.	a	9.	d	10.	D

PART-II SAMPLE SHORT QUESTIONS

1. What is Boolean algebra?
2. What is annulment law?

3. What is identity law?
4. What is idempotent law?
5. What is complement law?
6. What is commutative law?.
7. What is double negation law
8. What are the functions of De-morgan?.
9. What is De-Morgan fist theorem?
10. What is De-Morgan second theorem?
11. Draw the diagram of De-Morgan equivalent gate?
12. Draw the equivalent of and into negative Nor gate?

PART-III SAMPLE LONG QUESTIONS

1. Explain De-Morgan Laws?
2. What are the laws of De-Morgan's?
3. Explain the functions of De-Morgan's?
4. Implement De-Morgan Law using logic gates?
5. Draw the truth table of Boolean?.

CHAPTER NO. 04

ARITHMETIC LOGIC CIRCUITS

4.1 Introduction

One important aspect of digital design with MSI circuits not dealt with in earlier chapters is the design and implementation of arithmetic circuits. Originally, the basic arithmetic circuits were designed using discrete components, but this method has long been superseded by the introduction of MSI circuits. Multi-bit adders, arithmetic logic units and other circuits are now readily available as medium scale integrated circuits.

In some cases, a required arithmetic function is not available in a standard MSI package and modifying logic may be required. A typical example of this is the implementation of a binary adder/sub tractor or a circuit used for the implementation of BCD arithmetic.

The modifying logic can be provided by discrete gates or by another MSI circuit, so that some arithmetic circuits may be implemented by a combination of MSI and SSI chips.

Programmable logic devices may also be used in arithmetic applications.

For example, ROMs programmed as look-up tables can implement the multiplication process, while a combination of multi-bit adders and ROMs, in some cases, can extend the range of multiplication that can be provided.

Definition (*An arithmetic circuit is a set of gates with a separate set of inputs for each number that has to be processed)*.

The gates are connected so as to carry out an arithmetic action and the outputs of the gate circuit are the digits of the result (addition, subtraction, multiplication, or division.

4.2 Half Adder

Binary arithmetic is carried out by combinational logic circuits, the simplest of which is the half adder, shown in figure. This circuit consists, in its most basic form of two gates, an XOR gate that produces a logic 1 output whenever A is 1 and B is 0, or when B is 1 and A is 0.

The AND gate produces a logic 1 at the carry output when both A and B are 1. The half adder truth table is shown in Table and describes the result of binary addition.

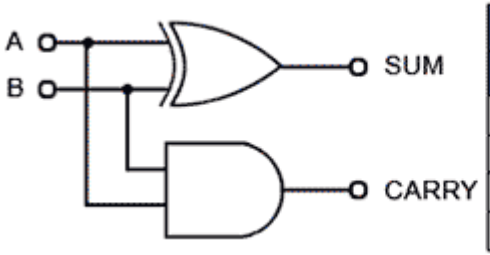


Table 4.1.1

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

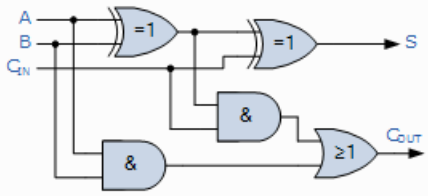
Figure 4.1:Half adder logic diagram and truth table

The half adder is fine for adding two 1-bit numbers together, but for binary numbers containing several bits, a carry may be produced at some time (as a result of adding 1 and 1) that must be added to the next column. As the half adder has only two inputs it cannot add in a carry bit from a previous column, so it is not practical for anything other than 1-bit additions.

4.3 Full Adder

When 2 or more bits are to be added, the circuit used is the Full Adder, shown in Fig 4.2, (blue background) together with its simplified block diagram symbol. This circuit simply comprises two half adders, the sum of A and B from the first half adder is used as input A on the second half adder, which now produces a sum of the first half adder sum (S_1) plus any 'carry in' from the C_{IN} terminal. Any carries produced by the two half adders are then 'OR' together to produce a single C_{OUT} output. The truth table for the circuit is given in Table.

Symbol



Truth Table

C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 4.2:Full adder logic diagram and truth table

4.4 Half Subtractor

A half subtractor is a logical circuit that performs a subtraction operation on two binary digits. The half subtractor produces a sum and a borrow bit for the next stage.

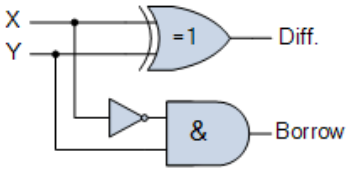
Symbol	Truth Table			
	Y	X	DIFFERENCE	BORROW
	0	0	0	0
	0	1	1	0
	1	0	1	1
	1	1	0	0

Figure 4.3: Half subtractor Logic diagram and truth table

From the truth table of the half subtractor we can see that the DIFFERENCE (D) output is the result of the Exclusive-OR gate and the Borrow-out (Bout) is the result of the NOT-AND combination. Then the Boolean expression for a half subtractor is as follows.

For the DIFFERENCE bit:

$$D = X \text{ XOR } Y = X \oplus Y$$

For the BORROW bit

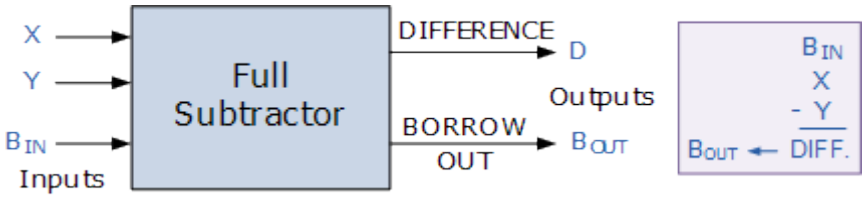
$$B = \text{not-}X \text{ AND } Y = X.Y$$

If we compare the Boolean expressions of the half subtractor with a half adder, we can see that the two expressions for the SUM (adder) and DIFFERENCE (subtractor) are exactly the same and so they should be because of the Exclusive-OR gate function. The two Boolean expressions for the binary subtractor BORROW is also very similar to that for the adders CARRY. Then all that is needed to convert a half adder to a half subtractor is the inversion of the minuend input X.

One major disadvantage of the Half Subtractor circuit when used as a binary subtractor, is that there is no provision for a “Borrow-in” from the previous circuit when subtracting multiple data bits from each other. Then we need to produce what is called a “full binary subtractor” circuit to take into account this borrow-in input from a previous circuit.

4.5 Full Subtractor

The main difference between the Full Subtractor and the previous Half Subtractor circuit is that a full subtractor has three inputs. The two single bit data inputs X (minuend) and Y (subtrahend) the same as before plus an additional Borrow-in (B-in) input to receive the borrow generated by the subtraction process from a previous stage as shown below.



Then the combinational circuit of a "full subtractor" performs the operation of subtraction on three binary bits producing outputs for the difference D and borrow B-out. Just like the binary adder circuit, the full subtractor can also be thought of as two half subtractors connected together, with the first half subtractor passing it's borrow to the second half subtractor as follows

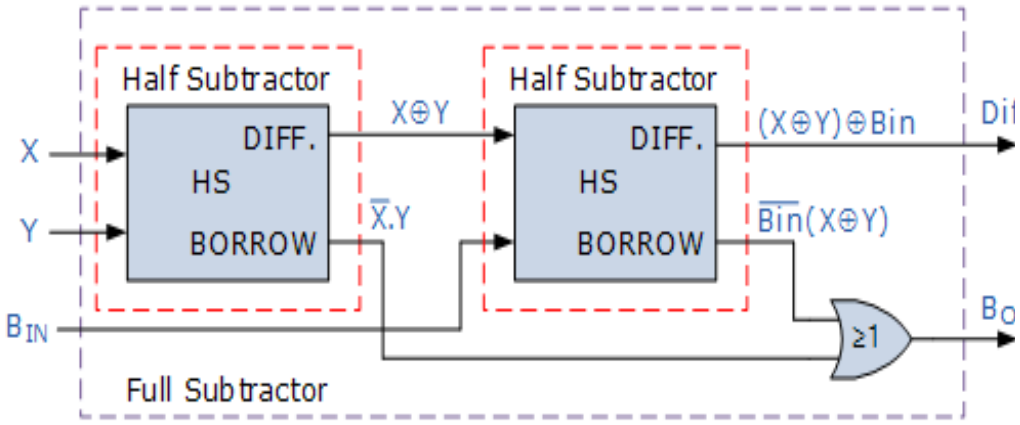


Figure 4.4:logic diagram of full subtractor

As the full subtractor circuit above represents two half subtractors cascaded together, the truth table for the full subtractor will have eight different input combinations as there are three input variables, the data bits and the Borrow-in, BIN input. Also includes the difference output, D and the Borrow-out, BOUT bit.

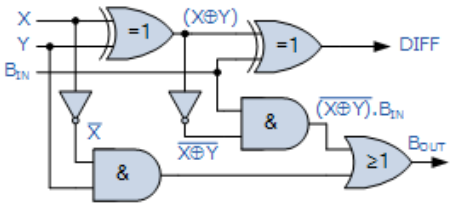
Symbol	Truth Table				
	B-in	Y	X	Diff.	B-out
	0	0	0	0	0
	0	0	1	1	0
	0	1	0	1	1
	0	1	1	0	0
	1	0	0	1	1
	1	0	1	0	0
	1	1	0	0	1
	1	1	1	1	1

Figure 4.5:Truth table of full subtractor

4.5.1 8 Bit Twos Complement Adder/Subtractor

To carry out arithmetic however, it is also necessary to be able to subtract. A further development of the parallel adder is shown in Fig.4.1.4. This is an 8-bit parallel adder/subtractor. This circuit adds in the same way as the adder in Fig. 4.1.3 but subtracts using the twos complement method described in Digital Electronics Module 1.5 (Ones and Twos Complement).

When subtraction is required, the control input is set to logic 1, which causes the bit at any particular B input to be complemented by an XOR gate before being fed to input B of the full adder circuit.

Twos complement subtraction in an 8-bit adder/subtractor requires that the 8-bit number at input B is complemented (inverted) and has 1 added to it, before being added to the 8-bit number at input A. The result of this will be an 8-bit number in twos complement format, i.e. with its value represented by the lower 7 bits (bit 0 to bit 6) and the sign represented by the most significant bit (bit 7). The logic 1 on the control input is therefore also fed to the first carry input of the adder to be included in the addition, which for subtraction is therefore:

Input A + Input B + 1

(Here + signifies addition rather than OR)

Alternatively, if addition of A and B is required, then the control input is at logic 0 and number B is fed to the adder without complementing.

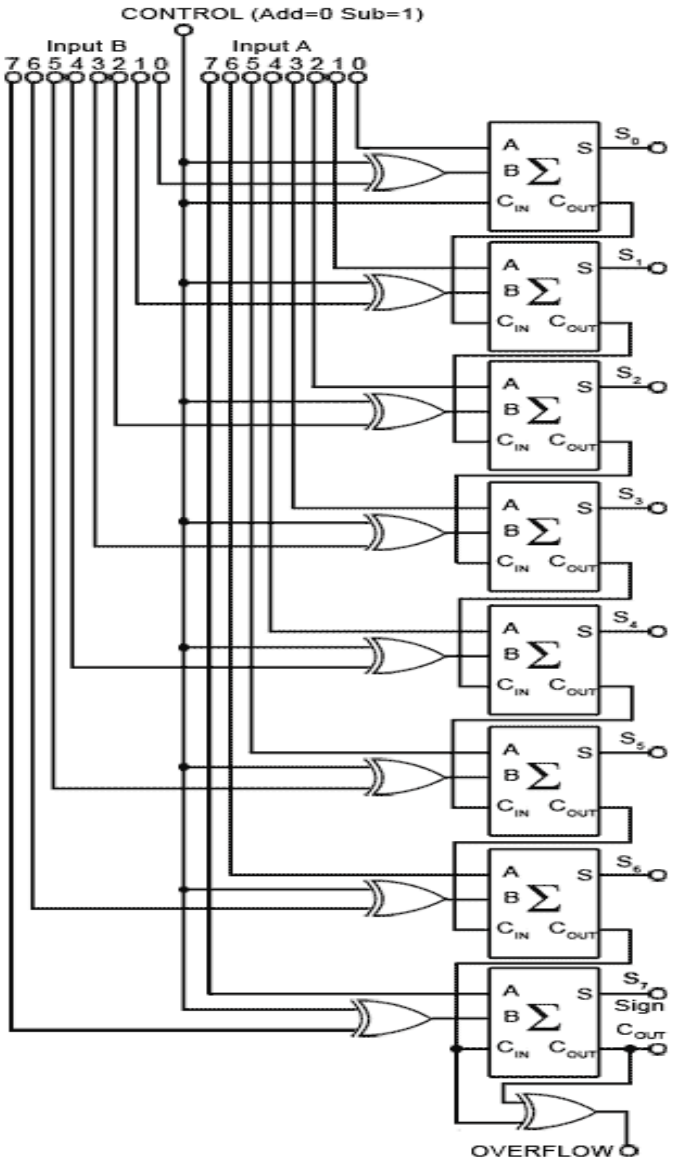


Figure 4.6:8 bit Full adder and subtractor

How an XOR gate is used here to change the adder into a subtractor by inverting the B inputs can be seen from the truth table for an XOR gate, shown in Table (in Fig. 4.6.)Notice that if input A, (used as the CONTROL input) of the XOR gate is at logic 0, then the XOR gate selects input B, but if input A is logic 1, then it selects the inverse of input B.

4.6 Comparator

Binary comparators, also called digital comparators or logic comparators, are combinational logic circuits that are used for testing whether the value represented by one binary word is greater than, less than, or equal to the value represented by another binary word. Two basic types of comparator can be used.

1. Equality comparators.
2. Magnitude comparators

4.6.1 Equality Comparator

An equality comparator, such as that illustrated in Fig 4.7 is the simplest multi-bit logic comparator, and can be used for such circuits as electronic locks and security devices where a binary password consisting of multiple bits is input to the comparator to be compared with another preset word.

In Fig.4.7 logic 1 will be present at the output if the two input words match, otherwise the output remains at 0. Therefore there is only one input combination that is correct, and the more bits the input words possess, the more possible wrong combinations there are. With extra circuitry for counting, additional security may be provided by limiting the number of tries before the input is inhibited.The circuit of the equality comparator consists of an exclusive NOR gate (XNOR) per pair of input bits. If the two inputs are identical (both 1s and both 0s) an output of logic 1 is obtained.

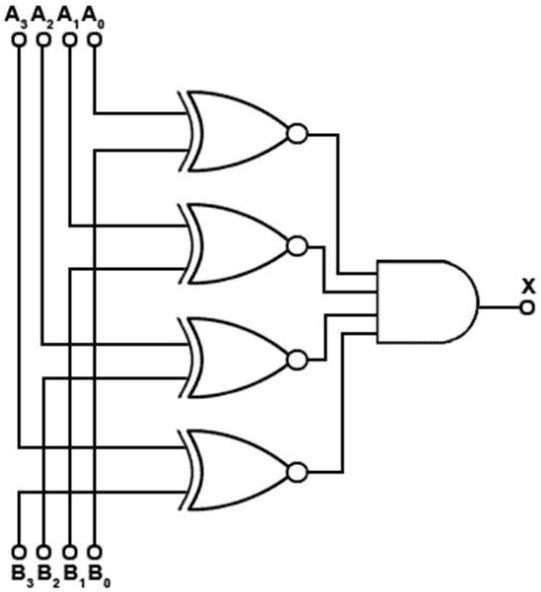


Figure 4.7:4 bit equality comparator

The outputs of the XNOR gates are then combined in an AND gate, the output of which will be 1, only when all the XNOR gates indicate matched inputs.

4.6.2 Magnitude Comparator

The magnitude comparator can also be used to indicate equality, but has a further two outputs, one that is logic 1 when word A is greater than word B, and another that is logic 1 when word A is less than word B. Magnitude comparators therefore form the basis of decision making in logic circuits. Any logical problem can be reduced to one or more (sometimes many) yes/no decisions based on a pair of compared values.

A simple 1-bit magnitude comparator is shown in Fig 4.8 ,2 Gate 1 produces the function A>B and gate 3 gives A<B while gate 2 is an XNOR gate giving an equality output.

This basic circuit for a magnitude comparator may be extended for any number of bits but the more bits the circuit has to compare, the more complex the circuit becomes. Integrated circuit magnitude comparators are available that can be used to provide comparisons between multi-bit words. One such IC is the 74HC85 CMOS 4-bit magnitude comparator from Philips Semiconductors (NXP) . This IC compares two 4-bit words and provides an output on pins 5, 6 and 7 that indicate whether the input words are equal, or if not, whether A or B has the higher numerical value.

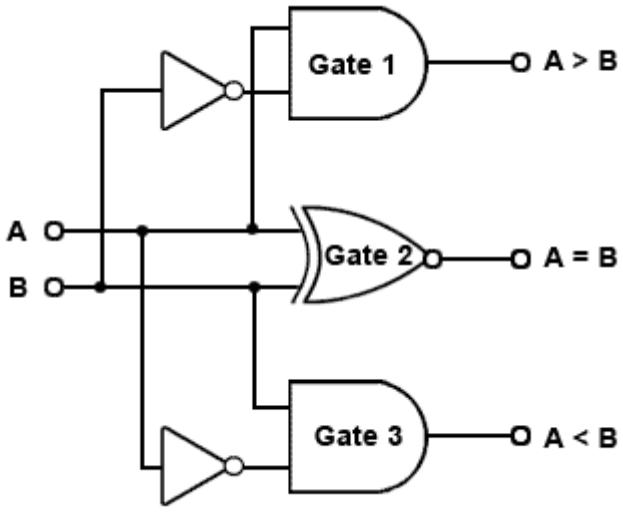


Figure 4.8:1 bit magnitude comparator

4.6.3 Comparators in Cascade

When two or more ICs are cascaded together, as shown in Fig. 4.9 the outputs of the first IC (representing the least significant 4 bits) are connected to the cascade inputs of the second IC and so on. The final result of the comparison appears on the three cascade outputs of the most significant 4-bit comparator.

To ensure a correct comparison, the cascade inputs of the first (least significant) comparator should be connected as follows:

A<B (pin 2) and A>B (pin 4) = logic 0.

A=B (pin 3) = logic 1.

This also applies to a single IC if only two 4-bit words are being compared.

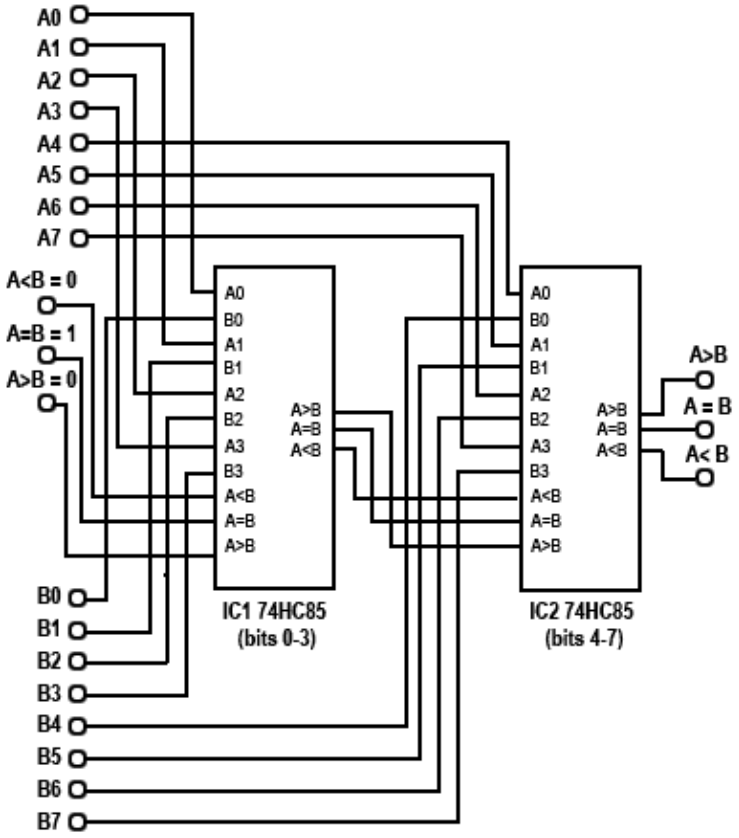


Figure 4.9: Cascaded Comparator

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. Half adder circuit requires two binary:
(a) inputs (b) outputs
(c) Digits (d)both a & b

2. The most significant bit of arithmetic addition is :
(a) overflow (b) carry (c) output (d) Zero bit

3. two bit addition is done by is:
(a) Half adder (b) Full adder (c) half subtractor(d) None

4. If A and B are the inputs of a half adder, the carry is given by __
(a) A AND B (b) A OR B (c) A XOR B d) A EX-NOR B

5. How many AND, OR and EXOR gates are required for configuration of full adder:
(a) 1, 2, 2 (b) 2, 1, 2 (c) 3, 1, 2 (d) 4, 0, 1

6. Half subtractor is used to perform subtraction of:
(a) 2 bits (b)3 bits (c) 4 bits (d)5 bits

7. Full subtractor is used to perform subtraction of:
(a) 2 bits (b)3 bits (c) 4 bits (d)5 bits

8. The output of a full subtractor is same as _____
(a) Half adder (b)Full adder (c)full subtractor(d)Decoder

9. Let A and B is the input of a subtractor then the borrow will be:
(a) A AND B' (b) A' AND B (c) A OR B (d) A AND B

10. if input of subtractor is A and B then what the output will be if A = B?
(a) 0 (b) 1 (c) A (d) B

ANSWER KEY

1.	D	2.	B	3.	A	4.	A	5.	B
6.	A	7.	B	8.	b	9.	B	10.	A

PART-II SAMPLE SHORT QUESTIONS

1. Define the term half adder?
2. Draw the symbol and truth table of half adder?
3. Define the term Full adder also draw its symbol?
4. Draw the truth table of Full adder?
5. Define the term half subtractor?
6. Define the term full subtractor?

7. Draw 8 bit subtractor?
8. Draw truth table of Adder/Subtractor?
9. Define the term comparator?
10. Enlist the types of comparator?
11. What is magnitude comparator?
12. What is equality comparator?
13. Draw 4 bit equality comparator?
14. Define the term cascaded?
15. How to make cascaded comparator?

PART-III SAMPLE LONG QUESTIONS

1. Explain the term half adder and full adder?
2. Explain the term half subtractor and full subtractor?
3. Explain comparator also its types?
4. What is cascaded comparator also draw 8 bit cascaded comparator?
5. Draw the block diagram of Full subtractor?

CHAPTER NO. 05

COMBINATIONAL LOGIC AND DATA PROCESSING CIRCUITS

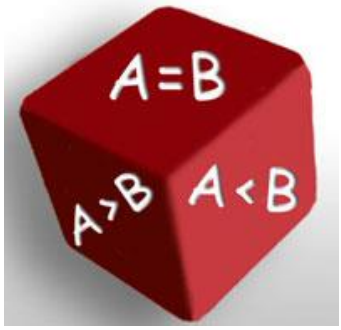
5.1 INTRODUCTION

Binary Arithmetic



Combinational logic has many uses in electronic systems. It is used to carry out the essential arithmetic, not only in computers and calculators, but also in navigation systems, robots and many other types of automatic machinery. However complex such calculations need to be, they all depend on some basic combinational logic circuits to carry out binary addition and subtraction.

Logical Decisions



Mathematics and logical decision making also uses combinational logic in the form of Comparators. These circuits decide whether one value is the same, larger or smaller than another value.

Data Routing

After making a logical decision, data may need to be routed to different parts of the electronic system; this routing is controlled by more combinational logic circuits such as the data selectors, multiplexers and de-multiplexers.



Encoding and Decoding

When connecting a logic circuit with the outside world, the incoming information from a keyboard or other input device will almost invariably need to be changed (encoded) into an appropriate binary form. Also before binary data produced by the digital system can be used by an output device, such as a

display, it must be decoded into a form that can be used by the display. Encoders and decoders used for such jobs are also combinational logic.

5.2 Data Selectors & Multiplexers

A simple way to connect multiple sources of information in analogue electronic systems is by using mechanical switches, such as those illustrated in Fig. 5.1. In example (a) a single pole double throw switch is used to select either input A or input B to be connected to output X. Example (b) shows a rotary selector switch that can multiplex any one of eight inputs to a single output.

In digital electronics, selecting multiple data sources can be performed by combinational logic circuits. Logic signals applied to one or more data select inputs initiate the selection of data, which may be steady logic levels or whole streams of digital information. Switching digital signals in this way is much faster and more reliable than using mechanical switch contacts. Digital data selectors and multiplexers are therefore a vital part of many digital systems. The names 'data selector' and 'multiplexer' are commonly interchanged, with multiplexers called data selectors and vice versa. If there is any difference, a circuit selecting between two inputs may be called a data selector, and more complex circuits combining multiple inputs into a single output, using various methods and existing in both digital and analogue forms, would be called multiplexers.

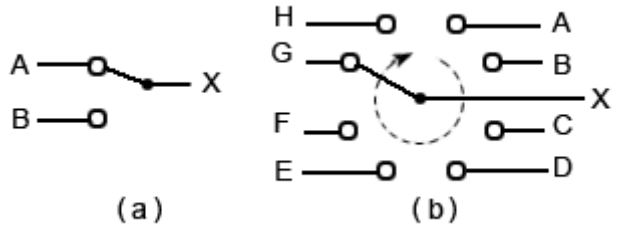


Figure 5.1:Mechanical selector switch

5.2.1 Basic Data Select (Multiplexer) Circuits

A simple data selector consisting of a single XOR gate was used in the 8 Bit Adder/Subtractor circuit to change the function of the circuit from addition to subtraction, but this was only required to select data B or its

inverse B. The circuit shown in Fig. 5.2 however can select either of two completely independent data inputs.

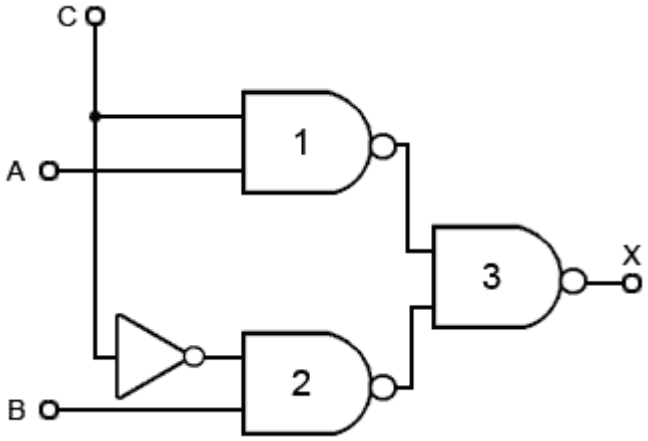


Figure 5.2:logic diagram of data selector

The operation of Fig. 5.2 is quite straightforward and relies on 'enabling' either of a pair of NAND gates (1 and 2), but not both.

5.2.2 Multiplexing

The control input (C) to the circuit in Fig.5.2 is fed directly to gate 1, but inverted to gate 2. This ensures that whatever the logic state of C, one gate is enabled, whilst the other is disabled.

Therefore when C = 1, NAND gate 1 will be enabled and its output will be the inverse of its data input (i.e. A), and because C (in this case logic 0) is applied to the control input of gate 2 its output will be logic 1.

Applying logic 0 to input C will cause gate 1 to be disabled, making its output logic 1, and gate 2 will be enabled making its output B.

Gate 3 will therefore always have one of its inputs held at logic 1, because either gate 1 or gate 2 is disabled, whilst the other input to gate 3 will be either A or B. Gate 3 output will be the inverse of this input, so the result at the output X will be either A or B depending on the state of the control line.

Note that this arrangement of three NAND gates (or four if an additional NAND gate is used in place of the inverter) works just the same as having an inverter (NOT gate) select either of two AND gates whose outputs are combined by an OR gate (De Morgan's theorem) but uses only one Quad 2-input gate IC instead of the three required by the NOT/AND/OR solution.

5.2.3 De−Multiplexing

Having combined, or multiplexed two data sources into one output line, it will usually be necessary at some point to separate or de-multiplex the combined data into separate outputs once more.

Two connections from the data select circuit are required to connect the data to this simple de−multiplexer, one to connect the data from output X of the data select circuit in Fig. 5.2 to the combined A/B input of another connection from C on Fig. 5.2.

Having to use two connecting lines to connect the multiplexer to the de-multiplexer to carry two signals does not apparently justify using these two extra circuits, however the principle of multiplexing demonstrated in Figs.5.2 can be extended to multiplex a greater number of data inputs, and the more lines that are multiplexed in this way, the more efficient the system becomes. Also there are additional ways to use these techniques other than transferring data from one place to another.

5.2.4 Multi−Bit Multiplexers

There are many uses for multiplexers. Wherever a number of signals, or logic states, need to be passed down a single communication channel such as a wire, a radio channel, or a telephone, some form of multiplexing is used. Sometimes the multiplexing and de-multiplexing can be very complex, much more so than the circuit in Figs 5.2. In some systems, data is transferred over very long distances, in others such as transferring data within computers; the distances may be very short. Fig. 5.3 shows a 4 to 1 line multiplexer, which enables a 4-bit binary number to be passed over 3 lines, one for data and two for control.

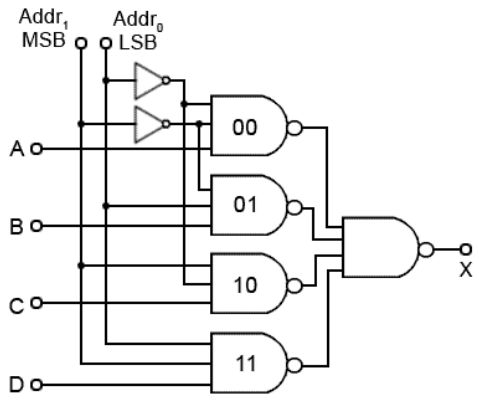


Figure 5.3:4-1 line Multiplexer

5.2.5 8-to-1 Multiplexer

An 8-to-1 multiplexer consists of eight data inputs D0 through D7, three input select lines S0 through S2 and a single output line Y. Depending on the select lines combinations, multiplexer selects the inputs.

The below figure shows the block diagram of an 8-to-1 multiplexer with enable input that can enable or disable the multiplexer. Since the number data bits given to the MUX are eight, then 3 bits (23 = 8) are needed to select one of the eight data bits.

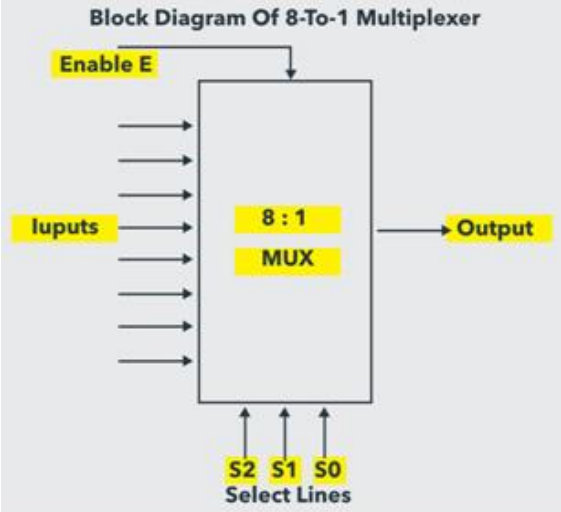


Figure 5.4:8-1 line MUX

Now the circuit diagram is shown below

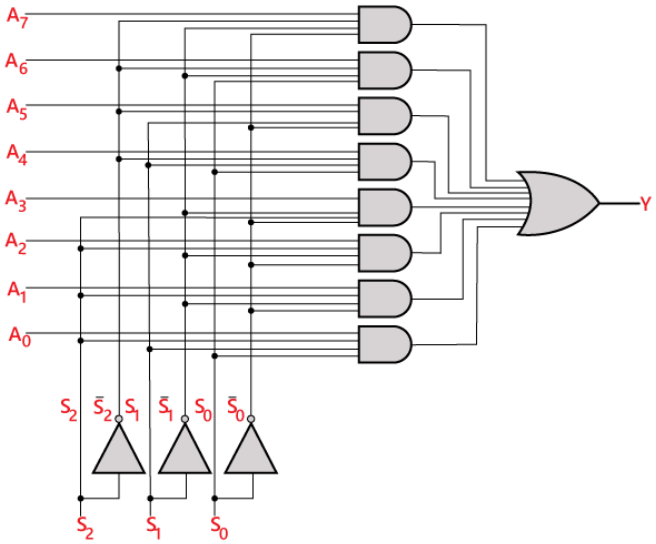


Figure 5.5:Circuit diagram of 8-1 line MUX

Truth Table of 8-1 line MUX

Input 1 (S1)	Input 2(S2)	Input 3(S3)	Output
0	**0**	**0**	**A**
1	**0**	**0**	**B**
0	**1**	**0**	**C**
1	**1**	**0**	**D**
0	**0**	**1**	**E**
1	**0**	**1**	**F**
0	**1**	**1**	**G**
1	**1**	**1**	**H**

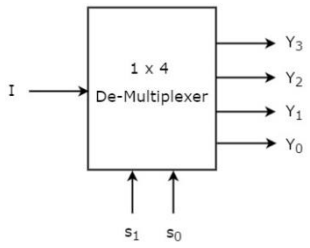
Larger multiplexers, such as 4, 8 or 16 bit types, which are readily available in IC form, use a method of 'addressing' a particular data gate using a binary code. Fig 5.3 shows a 4 to 1 multiplexer where, in order to output data from a particular input, one of the four 3−input NAND gates must be enabled by a logic 1 on two of its inputs, leaving the third input for data. To achieve this two address lines are used, giving four possible combinations of 1 and 0. Look carefully at the address lines. When both are at logic 0 the two inverters (NOT gates) produce logic 1s at two of the inputs to NAND gate 00. None of the other NAND gates addressed by these lines has both its address inputs at logic 1. If the least significant bit (LSB) of the address is 1 and the most significant bit (MSB) is 0 then NAND gate 01 is enabled. Because two address lines can give four possible binary combinations, you should find that, counting from the top, gate 00 is enabled by the address inputs 002, gate 01 by address 012, gate 10 by 102 and gate 11 by 112.

5.2.6 De-multiplexer

De-Multiplexer is a combinational circuit that performs the reverse operation of Multiplexer. It has single input, 'n' selection lines and maximum of 2n outputs. The input will be connected to one of these outputs based on the values of selection lines. Since there are 'n' selection lines, there will be 2n possible combinations of zeros and ones. So, each combination can select only one output. De-Multiplexer is also called as De-Mux.

5.2.6.1 1x4 De-Multiplexer

1x4 De-Multiplexer has one input I, two selection lines, s1 & s0 and four outputs Y3, Y2, Y1 &Y0. The block diagram of 1x4 De-Multiplexer is shown in the following figure.



The single input 'I' will be connected to one of the four outputs, Y3 to Y0 based on the values of selection lines s1 & s0. The Truth table of 1x4 De-Multiplexer is shown below.

Selection Inputs		**Outputs**			
S_1	**S_0**	**Y_3**	**Y_2**	**Y_1**	**Y_0**
0	0	0	0	0	**I**
0	1	0	0	**I**	0
1	0	0	**I**	0	0
1	1	**I**	0	0	0

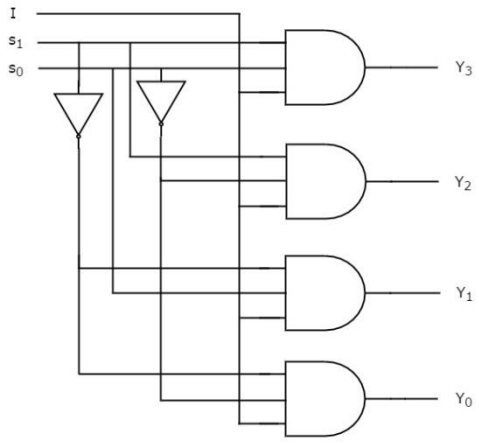


Figure 5.6:Circuit diagram of 1*4 MUX

Similarly, you can implement 1x8 De-Multiplexer and 1x16 De-Multiplexer by following the same procedure.

5.2.6.2 1*8 De-Multiplexer

Let us implement 1x8 De-Multiplexer. We know that 1x4 De-Multiplexer has single input, two selection lines and four outputs. Whereas, 1x8 De-Multiplexer has single input, three selection lines and eight outputs. So, we require two 1x4 De-Multiplexers in second stage in order to get the final eight outputs. Since, the number of inputs in second stage is two, we require 1x2 DE Multiplexer in first stage so that the outputs of first stage will be the inputs of second stage. Input of this 1x2 De-Multiplexer will be the overall input of 1x8 De-Multiplexer. Let the 1x8 De-Multiplexer has one input I, three selection lines s2, s1 & s0 and outputs Y7 to Y0. The Truth table of 1x8 De-Multiplexer is shown below.

Selection Inputs			**Outputs**							
s_2	**s_1**	**s_0**	**Y_7**	**Y_6**	**Y_5**	**Y_4**	**Y_3**	**Y_2**	**Y_1**	**Y_0**
0	0	0	0	0	0	0	0	0	0	**I**
0	0	1	0	0	0	0	0	0	**I**	0
0	1	0	0	0	0	0	0	**I**	0	0
0	1	1	0	0	0	0	**I**	0	0	0
1	0	0	0	0	0	**I**	0	0	0	0
1	0	1	0	0	**I**	0	0	0	0	0
1	1	0	0	**I**	0	0	0	0	0	0
1	1	1	**I**	0	0	0	0	0	0	0

We can implement 1x8 De-Multiplexer using lower order Multiplexers easily by considering the above Truth table. The **block diagram** of 1x8 De-Multiplexer is shown in the following figure.

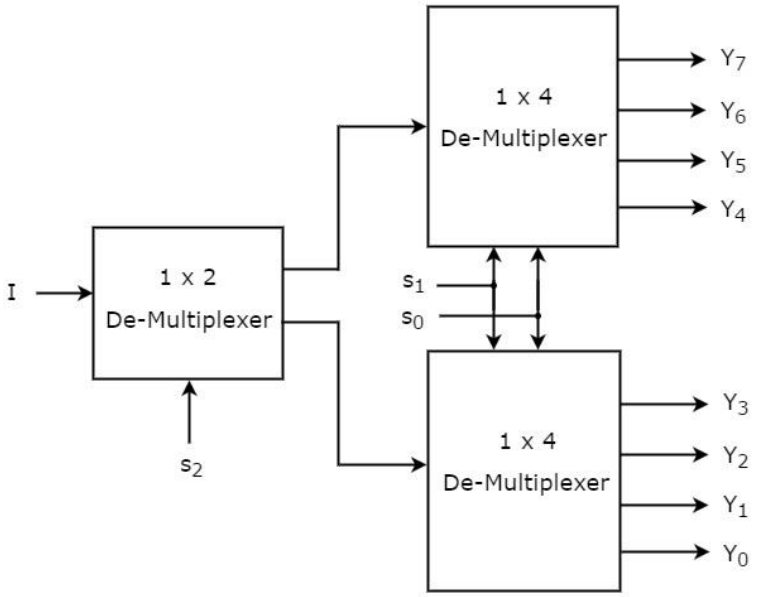


Figure 5.7:1-8 line De-Multiplexer

The common selection lines, s1 & s0 are applied to both 1x4 De-Multiplexers. The outputs of upper 1x4 De-Multiplexer are Y7 to Y4 and the outputs of lower 1x4 De-Multiplexer are Y3 to Y0.

The other selection line, s2 is applied to 1x2 De-Multiplexer. If s2 is zero, then one of the four outputs of lower 1x4 De-Multiplexer will be equal to input, I based on the values of selection lines s1 & s0. Similarly, if s2 is one, then one of the four outputs of upper 1x4 De-Multiplexer will be equal to input, I based on the values of selection lines s1 & s0.

5.2.6.3 1x16 De-Multiplexer

In this section, let us implement 1x16 De-Multiplexer using 1x8 De-Multiplexers and 1x2 De-Multiplexer. We know that 1x8 De-Multiplexer has single input, three selection lines and eight outputs. Whereas, 1x16 De-Multiplexer has single input, four selection lines and sixteen outputs.

So, we require two 1x8 De-Multiplexers in second stage in order to get the final sixteen outputs. Since, the number of inputs in second stage is two, we require 1x2 De-Multiplexer in first stage so that the outputs of first stage will be the inputs of second stage. Input of this 1x2 De-Multiplexer will be the

overall input of 1x16 De-Multiplexer. Let the 1x16 De-Multiplexer has one input I, four selection lines s3, s2, s1 & s0 and outputs Y15 to Y0. The block diagram of 1x16 De-Multiplexer using lower order Multiplexers is shown in the following figure.

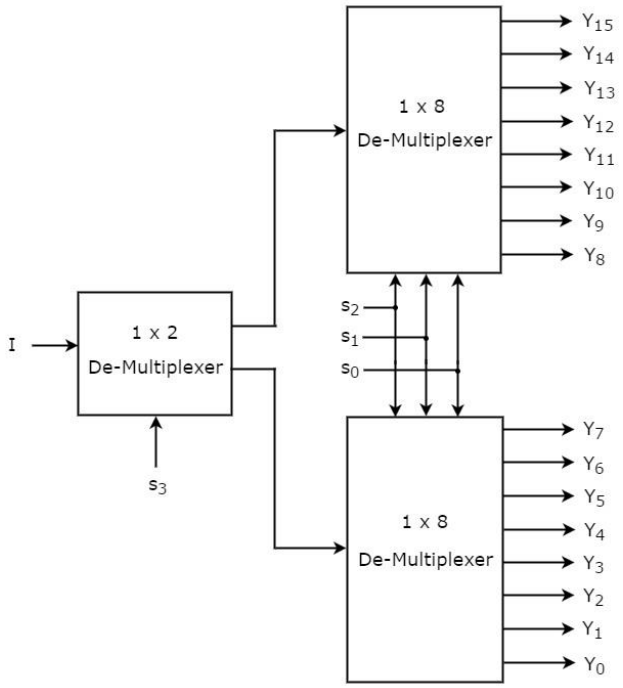


Figure 5.8:1-16 line De-Multiplexer

The common selection lines s2, s1 & s0 are applied to both 1x8 De-Multiplexers. The outputs of upper 1x8 De-Multiplexer are Y15 to Y8 and the outputs of lower 1x8 De-Multiplexer are Y7 to Y0.

The other selection line, s3 is applied to 1x2 De-Multiplexer. If s3 is zero, then one of the eight outputs of lower 1x8 De-Multiplexer will be equal to input, I based on the values of selection lines s2, s1 & s0. Similarly, if s3 is one, then one of the 8 outputs of upper 1x8 De-Multiplexer will be equal to input, I based on the values of selection lines s2, s1 & s0.

5.3 Binary Decoders

These circuits in IC form are often called Decoders/Demultiplexers and perform the opposite function to an encoder (or multiplexer).

Binary data is used in digital circuits in the form of one or another binary code, which is an arrangement of the binary bits in a particular order to represent 'real' quantities such as a set of decimal numbers (BCD code) or

text (ASCII). In a complete digital system therefore it is often necessary to convert one code to another, or to convert a binary code to drive some user interface such as a LED display.

A decoder is a combinational logic circuit that takes a binary input, usually in a coded form, and produces a one-bit output, on each of a number of output lines. The logic state (1 or 0) on any of the output lines depends on a particular code appearing on the input lines.

5.3.1 2-to-4-Line Decoder

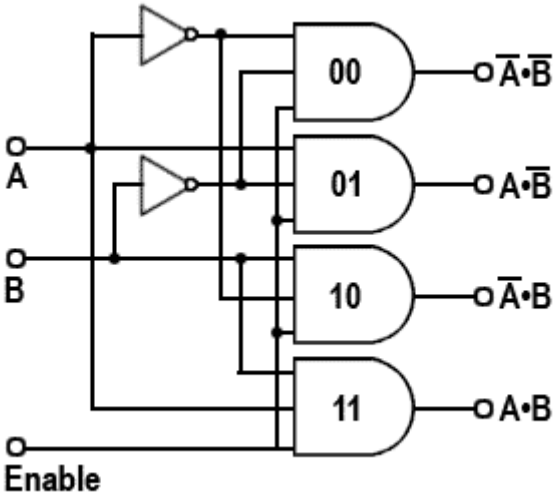


Figure 5.9:2-4 line decoder

A 2-to-4-line decoder is shown in Fig, in this circuit the two input lines can be set to any one of four binary values, 00, 01, 10 or 11. Resulting from this input, and provided that the (active high) Enable input is set to logic 1, the output line corresponding to the binary value at inputs A and B changes to logic 1. The other output lines remain at logic 0.

When the binary value at inputs A and B changes, the logic 1 on the output changes to a different line as appropriate. If the enable input is set to logic 0, all the outputs remain at logic 0 whatever values appear at inputs A and B.

To obtain logic 1 at any of the four outputs, the appropriate 3 input AND gate must have all of its inputs at logic 1. Provided that the Enable input is at logic 1, the output is controlled by using NOT gates to invert the logic applied from inputs A and B as required.

For example if inputs A and B are both at logic 0, the NOT gates at the inputs to the top (00) AND gate, invert both 0 inputs to logic 1, and therefore logic 1

appears at the 00 output. The 01 and 10 AND gates each have one input directly connected to the A or B input, whilst the other input is inverted.
The 11 gate has both A and B inputs directly connected to the AND gate so that 112 applied to A and B results in logic 1 at the 11 output.

Inputs			Outputs			
EN	B	A	00	01	10	11
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Figure 5.10: Truth table of 2-4 line decoder

5.4 BCD-to-Decimal Decoder

The 74HC42 BCD-to-Decimal decoder IC from Texas Instruments contains a more complex circuit, as illustrated in both block and logic schematic form in Fig.
The input is in 4-bit BCD8421 format, and each of the ten outputs, labeled Y0 to Y9 produce a logic 0 for an appropriate BCD8421 input of 00008421 to 01018421.
Any input value greater than 01018421 results in all of the output pins remaining at their high level, as shown in pale blue in Table.
On most data sheets for ICs the levels are shown as H (the higher voltage) and L (the lower voltage) to avoid confusion in cases where negative logic is used.
BCD to decimal decoders were originally used for driving cold cathode numerical displays (Nixie tubes), which are neon filled glass plug-in tubes with ten anodes in the shape of numbers 0 to 9 that glow when activated by a high voltage.
However, decimal decoders are also useful for a variety of other uses. Remember that decoders are often also called de-multiplexers, as they can be used for many de-multiplexing tasks and for driving devices such as lamps, motors and relays in control systems.

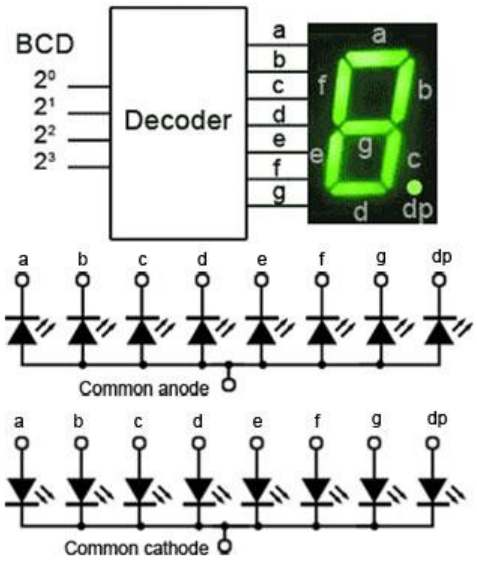


Figure 5.11:Driving seven segment display

5.4.1 BCD to 7 Segment Decoders

Because cold cathode displays require a high voltage drive, they have mostly been replaced by low voltage LED or LCD displays using 7 segment displays, therefore the BCD-to-7-segment decoder has become one of the most commonly available decoders.

As shown in block diagram format in Fig. 4.4.8, this type of decoder has 4 inputs for binary coded decimal and an output for each of the 7 LEDs that make up the 7-segment display. The eighth LED will normally be controlled by some extra logic outside the decoder. 7-segment displays may have a common cathode connection, needing to be driven by logic 1 outputs, or common anode connection requiring logic 0 outputs from the decoder.

5.4.2 Decoder/Drivers

Depending on the decoder IC and display type used, although it may sometimes be necessary to use a transistor amplifier to drive each segment of the display, there are Decoder/Driver ICs available, such as the 74LS46, 47, 48 and 49 that have sufficient output current and a choice of output designs such as open collector, internal pull-up resistors and active high or active low output levels that allow direct connection to both led and filament lamp displays.

When illuminated by the correct logic levels, the seven-segment display will show all the decimal numbers from 0 to 9. Depending on the logic design of the IC, some decoders will automatically blank the display for any value

greater than 9, while others display a unique (non-numeric) pattern for each value from 10 to 15 as shown in Fig, and may display 6 and 9 with or without a 'tail'. For displaying Hexadecimal numbers, the letters A B C D E and F are used to avoid confusion between capital B and 8, and capital D and 0.

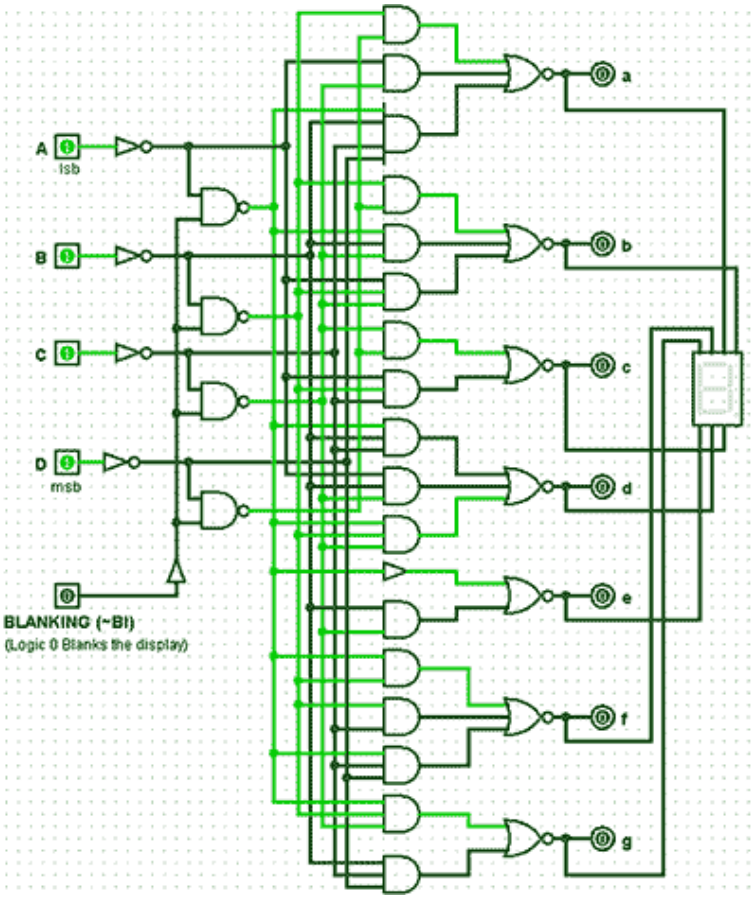


Figure 5.12: Logisim simulation of a basic BCD to 7 Segment Decoder

5.4.3 74LS49 BCD to 7-Segment Decoder

Above figure is a screen grab from Logisim, showing a working simulation of a basic BCD to 7-segment decoder (based on the 74LS49 in the TTL range of 7-segment decoders from Texas Instruments). This IC uses the font illustrated in fig and a single active low BI pin for use as a blanking input.

The blanking input pin BI can be used to turn off the display to reduce power consumption, or it can be driven with a variable width pulse waveform to rapidly switch the display on and off thereby varying the apparent brightness of the display.

Making the BI input logic 0 blanks the display whatever data is present at the decoder BCD inputs.

5.4.4 74LS48 BCD to 7-Segment Decoder Features

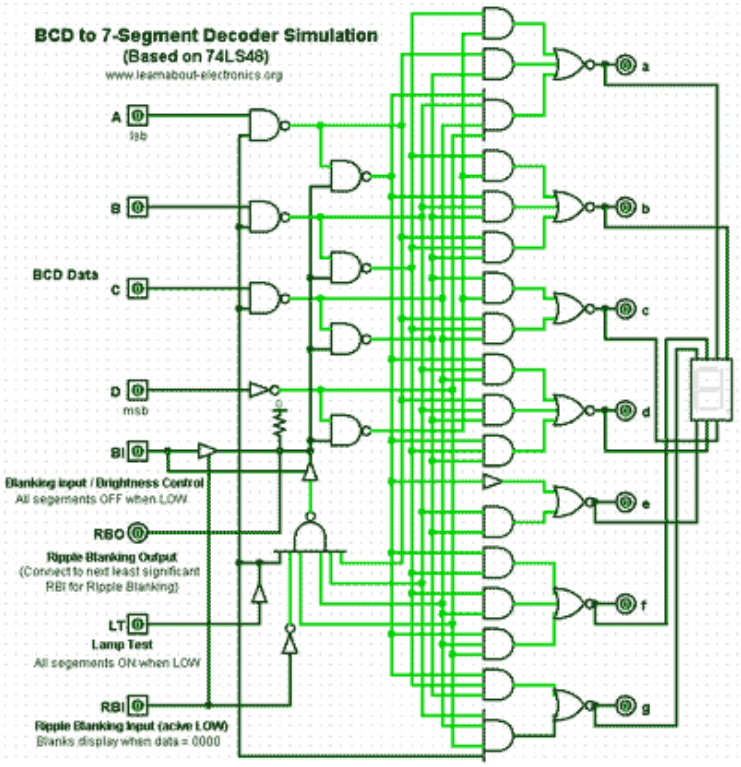


Figure 5.13 BCD to 7 Segment Decoder/Driver with Ripple Blanking and Lamp Test Facilities

5.4.5 Block Diagram of BCD

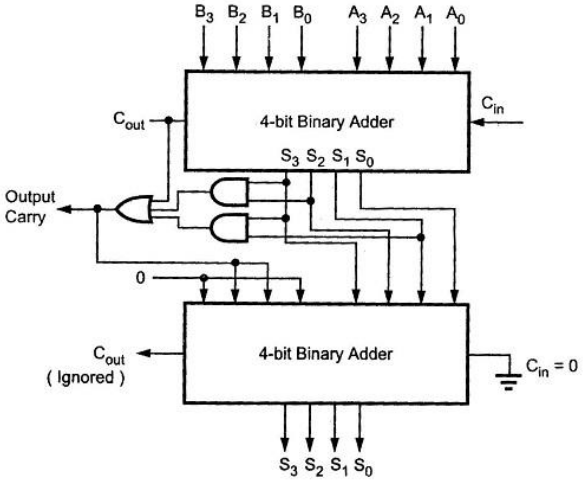


Figure 5.14 block diagram of BCD

5.4.6 Seven Segment display

A seven-segment LED is a digital display module specialized to display numerical information. Light-emitting diodes (LEDs) arranged in the shape of numbers offer an easily visible display. They are sometimes called "seven-segment displays" or "seven-segment indicators

Decimal	Binary DCBA	7 Segment Code a b c d e f g
0	0000	1 1 1 1 1 1 0
1	0001	0 1 1 0 0 0 0
2	0010	1 1 0 1 1 0 1
3	0011	1 1 1 1 0 0 1
4	0100	0 1 1 0 0 1 1
5	0101	1 0 1 1 0 1 1
6	0110	0 0 1 1 1 1 1
7	0111	1 1 1 0 0 0 0
8	1000	1 1 1 1 1 1 1
9	1001	1 1 1 0 0 1 1

Figure 5.15 Truth Table of seven segment display

Seven-segment LED circuit configuration

LED display devices have two kinds of circuit: common anode and common cathode.

1. Common anode: when the common pin is positive
2. Common cathode: when the common pin is negative

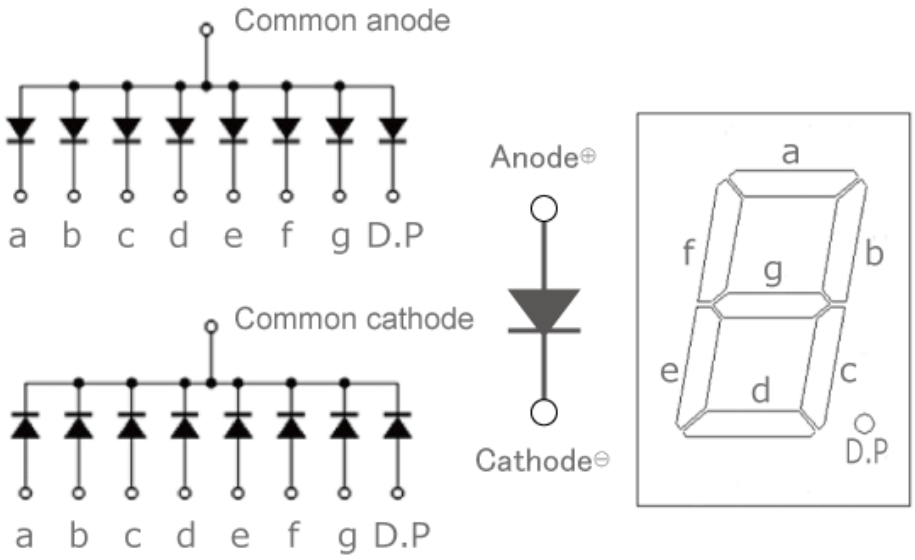


Figure 5.16:SSD PIN Configuration

5.4.7 LED display driver ICs

We can offload the job of driving an LED display to one of a number of ICs designed specifically for the purpose. Two examples are the CD4511 latching Binary Coded Decimal (BCD) to seven segment decoder driver and the MAX7219 Serially Interfaced 8-Digit LED Display Driver.

The MAX7219 has a number of sophisticated features including the ability to drive up to eight LED displays, bar-graph displays, or 64 individual LEDs. It communicates with the host microcontroller through a serial interface.

The circuit in figure 8 uses the much simpler CD4511 to drive a single LED display. The host microcontroller sends a 4 bit BCD digit between 0 and 9 to the CD4511. The CD4511 produces the corresponding seven segment bit pattern (shown in table) at its output pins.Only four microcontroller I/O pins are required to send the data.

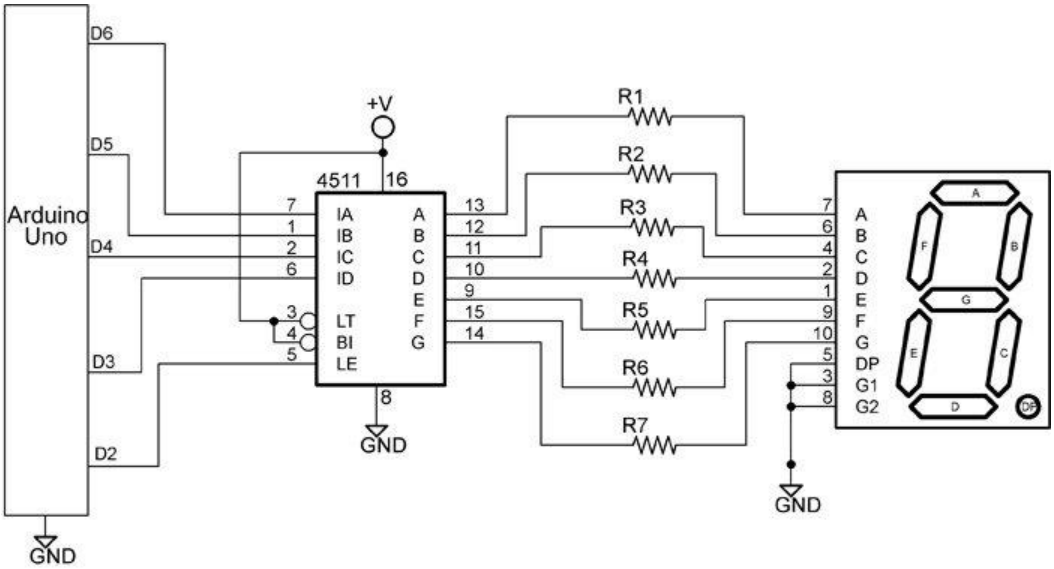


Figure 5.18:Seven Segment driver IC

5.5 Binary Encoders

Binary encoders described a number of different binary codes that are used to perform a range of functions in digital circuits. Mathematics, graphics, data manipulation and physical control systems are among many of the functions that are carried out using binary data, and each of these uses may

require binary data arranged in various forms of binary codes. For example text may be represented by an ASCII code (American standard Code for Information Interchange), in which each letter, number or symbol is represented by a 7-bit binary code. Decimal numbers in a calculator may be sent to a numeric display using BCD (Binary Coded Decimal). Notice that the word 'code' appears in each of these titles, and a binary code differs from normal binary because it is arranged in a particular way to suit a given purpose**.**

5.6.1 Priority Encoders

Binary Encoders generally have a number of inputs that must be mutually exclusive, i.e. only one of the inputs can be active at any one time. The encoder then produces a binary code on the output pins, which changes in response to the input that has been activated.

5.6.2 8-to-3-Line Encoder

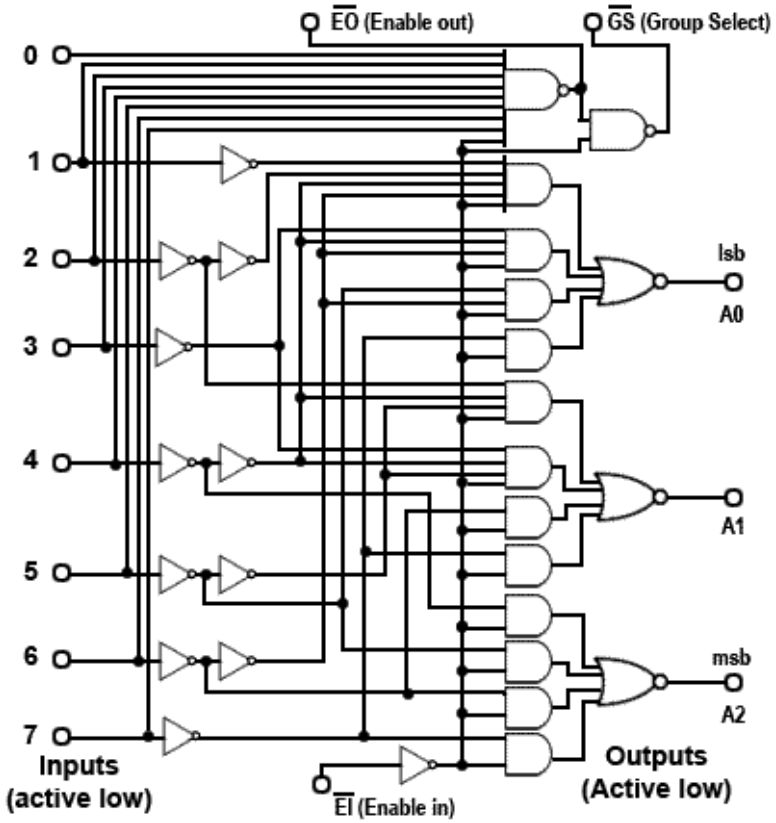


Figure 5.19:IC base 8 to 3 line Encoder

The 74HC148 also uses priority encoding and features eight active low inputs and a three-bit active low binary (Octal) output. The internal logic of the 74HC148 is shown in Figure.

The IC is enabled by an active low Enable Input (EI), and an active low Enable output (EO) is provided so that several ICs can be connected in cascade, allowing the encoding of more inputs, for example a 16-to-6-line encoder using two 8-to-3 encoders. The CMOS 74HC148 also uses active low inputs and outputs. The operation of the 74HC148 can be seen from its truth table.

Inputs									Outputs				
$\overline{EI}$	0	1	2	3	4	5	6	7	A2	A1	A0	$\overline{GS}$	$\overline{EO}$
H	X	X	X	X	X	X	X	X	H	H	H	H	H
L	H	H	H	H	H	H	H	H	H	H	H	H	L
L	X	X	X	X	X	X	X	L	L	L	L	L	H
L	X	X	X	X	X	X	L	H	L	L	H	L	H
L	X	X	X	X	X	L	H	H	L	H	L	L	H
L	X	X	X	X	L	H	H	H	L	H	H	L	H
L	X	X	X	L	H	H	H	H	H	L	L	L	H
L	X	X	L	H	H	H	H	H	H	L	H	L	H
L	X	L	H	H	H	H	H	H	H	H	L	L	H
L	L	H	H	H	H	H	H	H	H	H	H	L	H

5.5.3 Decimal to BCD Priority Encoder (74147 IC)

It is transistor-transistor logic (TTL) encoder that featured priority decoding of the inputs to ensure that only the highest-order data line is encoded. The implied decimal zero condition requires no input condition, as zero is encoded when all nine data lines are at a high logic level.

Specification

1. Supply voltage: 4.5V to 5.5V
2. High level output current: -0.4mA
3. Low level output current: 0mA
4. Operating temperature range: 0ºC to 70ºC

Product Details

1. Product: 10-to-4 line decimal to BCD priority encoder IC
2. IC number: 74147
3. Family: 74 series
4. Package type: PDIP-16
5. No of pins: 16
6. Mounting type: through hole

7. Package contains: 1 x 74147 10-to-4 Decimal to BCD Priority Encoder IC

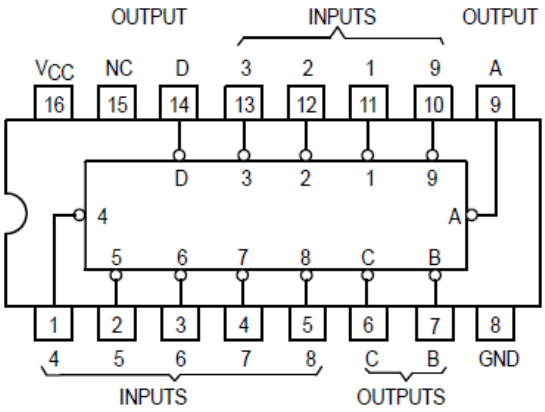


Figure 5.20 Pin configuration of IC

The truth table of encoder is given below

INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	X	L	H	H	H	H	L	L	H
X	X	X	X	L	H	H	H	H	H	L	H	L
X	X	X	L	H	H	H	H	H	H	L	H	H
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

H = high logic level, L = low logic level, X = irrelevant

Figure 5.21: Truth table of Priority Encoder

5.6 Read only Memories (ROM)

Read Only Memory (ROM) is a kind of memory which forms an integral part of many electronic components including those of computers. The major trait of this memory is the fact that, here, the number of reads are unrestricted while the number of writes are fixed. This means that in a ROM, the user can store the data only once (usually done during fabrication) or sometimes only a couple of times. However this stored data can be read from it how many every times needed. Next, the information so stored in

ROMs will not be erased even if the power shuts down. As a result, they are referred to as permanent, non-volatile memory devices.

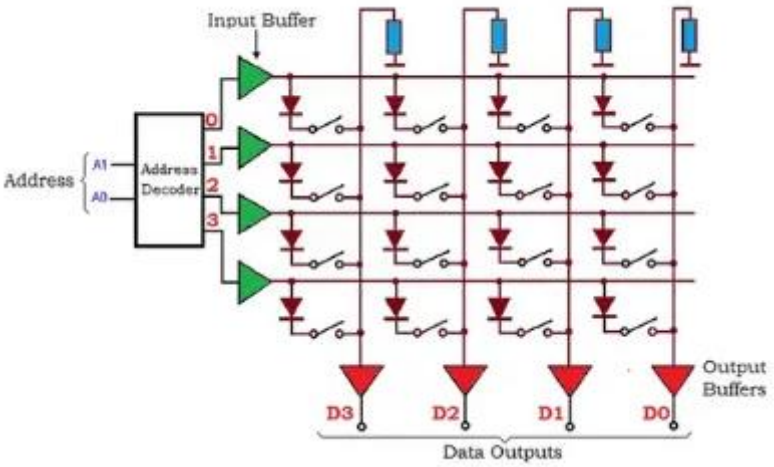


Figure 5.22 internal circuit diagram of ROM

Internally, ROM comprises of a row-column grid which will be connected by the diodes depending on the programming bit received. This indicates that ROM is a hard-wired memory device. As a consequence, one need not refresh its memory contents on a timely-basis in order to keep-up the data stored, unlike RAM (Random Access Memory) which demands for frequent refresh cycles.

The single-write, multiple-read nature of ROM makes it most suitable to store boot-up programs and the firmware applications which require no/very less updates. Calculators, laser printers, washing machines and gaming consoles are a few among the numerous appliances wherein ROM finds its place.

5.7 Diode Rom

Diode ROM Memory consists of only diodes and a decoder. As shown in the below figure address lines A0 and A1 are decoded by 2 : 4 decoder and used to select one of the four rows.

As decoder output is active low, it places logic 0 on the selected row. Each output data line goes to logic 0 if a diode connects the output data column to the selected row. Data is available on the output data lines only when output enable (OT) signal is low.

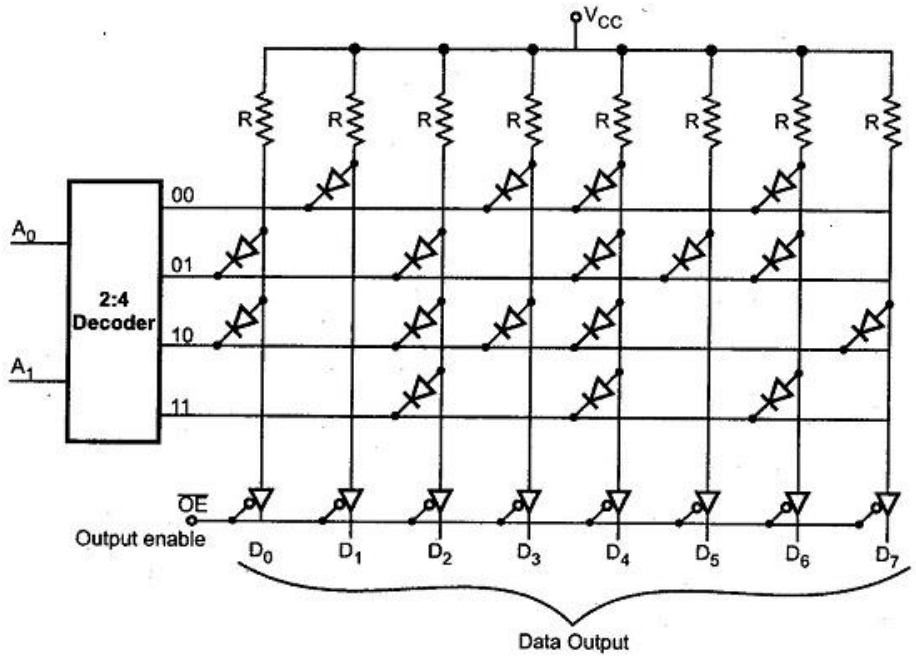


Figure 5.23:simple four byte ROM

The below figure show us the content of ROM

Address in binary	Binary Data								Data in Hex
	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	
00	1	0	1	0	0	1	0	1	A5
01	0	1	0	1	0	0	0	1	51
10	0	1	0	0	0	1	1	0	46
11	1	1	0	1	0	1	0	1	D5

Figure 5.24:Content of ROM

5.8 Commercially Available ROM

1. Masked Read Only Memory (MROM)

These were the first set of ROMs developed which are hard-wired and read-only in nature. MROM is programmed by the manufacturer during fabrication after which the contents of the memory can never be changed. This is because, MROMs are fabricated using masks (integrated circuits which have definite opaque and transparent areas) which either allow or block the passage of light (like that of UV) during photolithography. These devices are compact than any other kind of semiconductor memories for a given bit and are preferred when the need is for mass production.

2.Programmable Read Only Memory (PROM)

PROMs in the market are blank ROMs and contain no pre-recorded instructions like those of MROMs. These allow the user to program it only once, using PROM programmers, after which their contents can never be changed. This is because; a typical PROM with all 1's in it will be programmed to have 0's by blowing-off the fuses using high voltage pulses, wherever required. As this is an irreversible process, PROMs are one-time programmable.

2.Erasable and Programmable Read Only Memory (EPROM)

These devices comprise of an array of transistors characterized by floating-gates, which are programmed one-by-one with the help of high-voltage pulses. However, such a programmed EPROM can be erased by exposing them to strong ultra-violet (UV) light for certain duration, after which they can be programmed once again. Nevertheless, multiple-erase operations cause wearing out of the device due to which they lack the unlimited reprogramming ability. This kind of memory devices are generally used to store firmware programs which demand frequent upgrades.

3.Electrically Erasable and Programmable Read Only Memory (EEPROM)

EEPROMs can be programmed more than once, as they are capable of being erased electrically. This means that they need not be removed from the system in order to carry on the process of erasing and programming. However writing into an EEPROM is a slow process when compared to reading from it. Moreover they offer the flexibility of erasing byte-by-byte or selected areas only, instead of entire chip as in the case of EPROMs. Depending on this ability they can be classified as

1. **Electrically Alterable Read Only Memory (EAROM)**

   This kind of EEPROM allows bit-wise modification. However writing process is extremely slow and requires higher voltage than that for reading. Thus these are used to store set-up information of the system or for similar applications which require rare and/or partial rewriting.

2. **Flash Read Only Memory (FROM)**

These can be erased and reprogrammed much faster than EEPROMs and allow the user to access multiple locations of the memory at the same time.

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. 1. Which of the following is not a multiplexer?
(a) 4 into 1 (b) 8 into 1 (c) 1 into 4 (d) 16 into 1

2. 2. Which of the following have more inputs than outputs?
(a) Encoder (b) MUX (c) De-MUX (d) Both b & c

3. 3. It can be used in parallel-to-serial conversion.
(a) MUX (b) Decoder (c) BCD (d) None

4. 4. It can produce a binary code at its output.
(a) Decoder (b) Encoder (c) MUX (d) BCD

5. Only one of its outputs is activated at one time.
(a) DeMUX (b) Decoder (c) Both A &B d) None

6. Used to route input signals to one of several outputs
(a) DeMUX (b) Decoder (c) Both A &B d) None

7. 7. Following one is an example of 3 line-to-8 line or binary to octal.
(a) Decoder (b) Encoder (c) Multiplexer (d)All of these

8. 8. The _____ seven segment display use seven individual
(a) LED (b) LRT (c) LET (d) None

9. Binary coded decimal is a combination of
(a) 2 binary digit (b)3 binary digit (c)4 binary digit (d)5 binary digit

1**0.** Add the two BCD numbers: 1001 + 0100 = ?
(a)10101111 (b)01010000 (c)00010011 (d)00101011

ANSWER KEY

1.	c	2.	b	3.	a	4.	b	5.	d
6.	B	7.	a	8.	a	9.	c	10.	C

PART-II SAMPLE SHORT QUESTIONS

1. Define Encoder?

2. What is meant by 4 into 1 MUX?
3. Explain diode ROM?
4. Differentiate between multiplexer and de multiplexer?
5. Draw truth table for encoder?
6. What is meant by SSD?
7. Define working of decoder?
8. What is BCD stand for also define its working?
9. Define ROM classified its types?
10. Define encoding and decoding?
11. What is meant by sequential logic?
12. Draw a circuit diagram of 4 into 1 multiplexer?
13. Enlist commercially available ROM?
14. What is addressing?
15. What is multiplexing?
16. What is D-Multiplexing?
17. Draw 1*8 Mux?
18. Draw1*16MUX?
19. What is seven segment display?
20. Define the term BCD?

PART-III SAMPLE LONG QUESTIONS

1. Differentiate Multiplexer and De-Multiplexer?
2. Draw the block diagram of BCD?
3. Explain binary Encoder also explain its types?
4. Describe working of seven segment display also draw its circuit?
5. Write a note on Read only Memory?

CHAPTER NO. 06

FLIP FLOPS

6.1 INTRODUCTION

A flip flop is an electronic circuit with two stable states that can be used to store binary data. The stored data can be changed by applying varying inputs. Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems.

For example, let us talk about SR latch and SR flip-flops. In this circuit when you Set S as active the output Q would be high and Q' will be Low. This is irrespective of anything else. (This is an active-low circuit so active here means low, but for an active high circuit active would mean high).

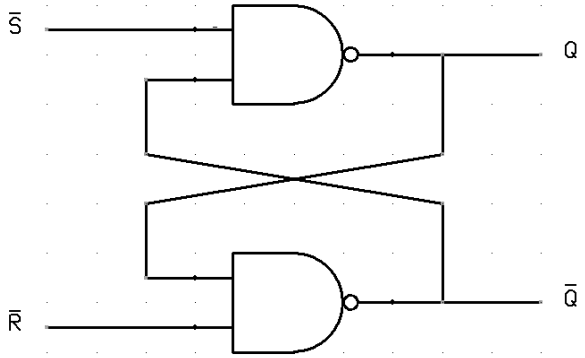


Figure 6.1:SR Latch diagram

A flip-flop, on the other hand, is synchronous and is also known as a gated or clocked SR latch.

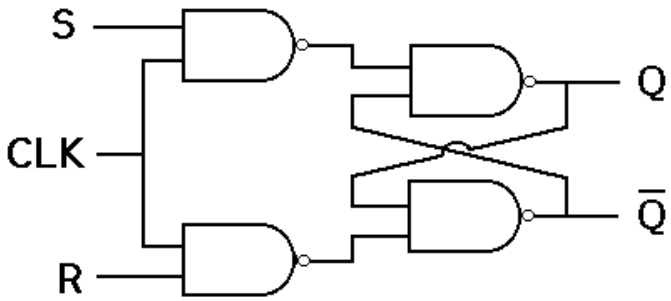


Figure 6.2:SR Flip flop

In this circuit diagram, the output is changed (i.e. the stored data is changed) only when you give an active clock signal. Otherwise, even if the S or R is active the data will not change. Let's look at the types of flip-flops to understand better.

Types of flip-flops:

1. RS Flip Flop
2. Clocked RS Flip Flop
3. D-Flip Flop
4. Edge triggered D-Flip Flop
5. JK Flip Flop

6.2 RS Flip Flop

The RS Flip Flop is considered as one of the most basic sequential logic circuits. The Flip Flop is a one-bit memory bi-stable device. It has two inputs, one is called "SET" which will set the device (output = 1) and is labeled S and another is known as "RESET" which will reset the device (output = 0) labeled as R. The RS stands for SET/RESET. The flip-flop is reset back to its original state with the help of RESET input and the output is Q that will be either at logic level "1" or logic"0". It depends upon the set/reset condition of the flip-flop. Flip flop word means that it can be "FLIPPED" into one logic state or "FLOPPED" back into another.

The basic NAND gate RS Flip Flop circuit is used to store the data and thus provides feedback from both of its outputs again back to its inputs. The RS Flip Flop actually has three inputs, SET, RESET and its current output Q relating to its current state. The symbol of the RS Flip-Flop is shown below

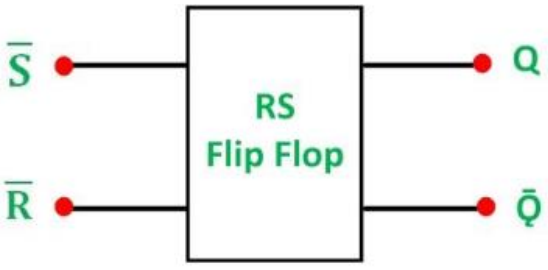


Figure 6.3:Symbolic representation of RS Flip Flop

6.2.1 The NAND Gate RS Flip Flop

A pair of cross-coupled 2 unit NAND gates is the simplest way to make any basic one-bit set/reset RS Flip Flop. It forms Set/Reset bi-stable or an active LOW RS NAND gate latch. The feedback is fed from each output to one of the other NAND gate input. The device consists of two inputs; one is known as SET, (S) and the other is called as RESET, (R).The two outputs are Q and Q bar as shown in the figure below

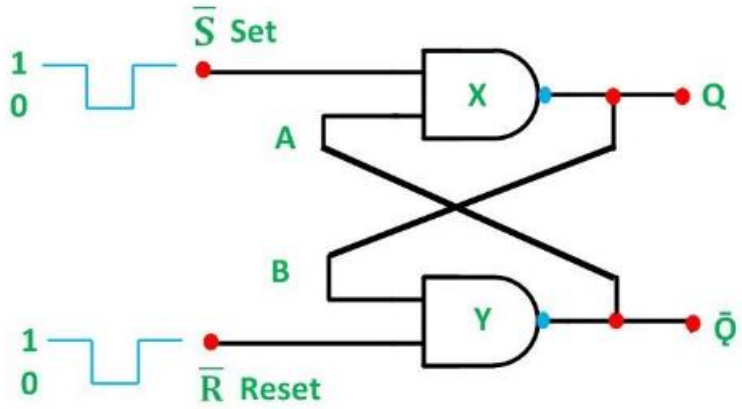


Figure 6.4:Nand gate RS Flip Flop

The Set State

Considering the above circuit, If the input R is at logic level "0" (R = 0) and input S is at the logic "1" (S = 1), the NAND gate Y has, at least, one of its inputs at a logic "0". Therefore, its output Q must be at a logic level "1" (NAND gate principles). The Output (Q) is fed back to the input "A". Both the inputs of the NAND gates X are at logic "1", and therefore, its output Q must be at the logic level"0".

The reset input R changes its state, and goes HIGH to logic "1" with S constant at logic "1". The NAND gate Y input are now (R = 1) and (B = 0). The output at Q remains at HIGH or at logic level "1" as one of its inputs is still at logic level "0".As a result, there is no change in state. Therefore, the flip-flop circuit is said to be "LATCHED" or "SET" with Q = 1 and $\bar{Q}$ = 0.

The Reset State

In this second stable state, Q is at logic level '0" and its inverse output Q is at logic level "1". And is given by (R = 1) and (S = 0). As gate X has one of its inputs at logic "0" its output Q must equal logic level "1". (According to the

NAND gate principle). The output Q is fed to input B, so both the inputs to NAND gate Y are at logic "1"., therefore, Q = 0.If the set input S now changes the state to logic "1" with the input R remaining at logic "1", the output Q still remains LOW at logic level "0". And there is no change in the state. Therefore, the flip-flop circuits "RESET" state has been latched. The truth table of the Set/Reset is given below

State	S	R	Q	$\bar{Q}$	Description
SET	1	0	1	0	Set Q >>1
	1	1	1	0	No Change
RESET	0	1	0	1	Reset Q >>0
	1	1	0	1	No Change
INVALID	0	0	0	1	Memory with Q = 0
	0	0	1	0	Memory with Q = 1

From the truth table, it is clear that when both the inputs S = 1 and R =1 the outputs Q, and $\bar{Q}$ can be at either logic level '1' or "0" depending upon the state of the inputs.

When the input state R = 0 and S = 0 is an invalid condition and must be avoided because this will give both outputs Q and $\bar{Q}$ at logic level "1" at the same time and the necessary condition is that Q to be the inverse of $\bar{Q}$.

The flip-flop goes to an unstable state as both the output goes LOW. This unstable condition arises when the LOW input is switched to HIGH. The flip-flop switches to one state or the other and any one output of the flip-flop switches faster than the other. This unstable condition is known as Meta-stable state.

The bistable RS flip flop is activated or set at logic “1” applied to its S input and deactivated or reset by a logic “1” applied to R. The RS flip-flop is said to be in an invalid condition if both the set and reset inputs are activated simultaneously.

6.2.2 Clocked RS Flip Flop

A clock is a device that generates a signal that periodically cycles between a high state, “1”, and a low state, “0”. Clocks ensure that the operations performed by a computer proceed in an orderly manner. They do so by enabling certain operations to occur only at specific points in time.

Clocks divide time into “cycles” that consist of two phases, a high phase and a low phase. Specifically, a clock cycle is defined as the interval of time beginning when the clock goes to a high state, lasting through the return to a low state, and ending with the start of the transition back to the high state again. Below figure illustrates four complete cycles of a clock.

Each clock cycle lasts for only a brief instant of time. The CPU of a modern PC, for example, runs at billions of clock cycles per second, or gigahertz. The clock speeds of other components, such as the system bus, are usually somewhat slower, but still in the range of hundreds of millions of clock cycles per second, or megahertz. The various operations that a computer can perform require one or more clock cycles to complete. The exact number of cycles depends upon the complexity of the particular operation.

Flip-flops can be used to implement the most basic unit of storage, the bit. Memory devices based on R-S flip-flops perform read operations by retrieving the contents of the Q outputs of a number of selected bits. The write operation, in turn, stores bit patterns into memory by placing 1’s on either the S (set) or R (reset) inputs of various bits.

The clock ensures that these operations happen in an orderly manner. During one phase of the clock cycle (e.g. low) the contents of memory can be examined but not modified. During the opposite phase of the cycle (e.g. high) the contents of memory can be updated. This sort of timing is critical for the reliable operation of a computer; since among other things, it allows time

after a “write” operation for the flip-flops to settle into their stable configurations before “read” operations can be attempted.

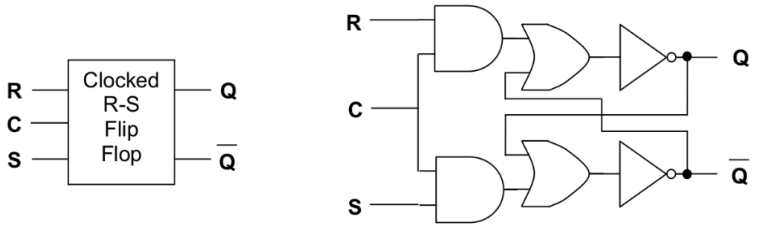


Figure 6.4:Clocked RS Flip Flop

In the clocked R-S flip flop the appropriate levels applied to their inputs are blocked till the receipt of a pulse from another source called clock. The flip flop changes state only when clock pulse is applied depending upon the inputs. This circuit is formed by adding two AND gates at inputs to the R-S flip flop.

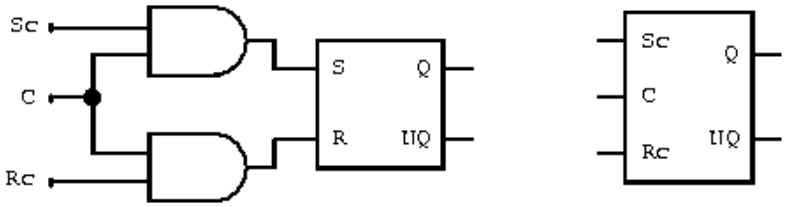


Figure 6.5:Schematic diagram of RSFF

The truth table of this Flip Flop is given below

S_C	R_C	C	Q	$\overline{Q}$
x	x	0	no change	
0	0	1	no change	
0	1	1	0	1
1	0	1	1	0
1	1	1	undefined	
0	0	p	no change	
0	1	p	0	1
1	0	p	1	0
1	1	p	undefined	

Figure 6.6:Truth Table of SR Flip Flop

6.3 D-FLIP FLOP

In **SR NAND Gate Bistable** circuit, the undefined input condition of SET = "0" and RESET = "0" is forbidden. It is the drawback of the SR flip flop. This state:

1. Override the feedback latching action.
2. Force both outputs to be 1.
3. Lose the control by the input, which first goes to 1, and the other input remains "0" by which the resulting state of the latch is controlled.

We need an **inverter** to prevent this from happening. We connect the inverter between the Set and Reset inputs for producing another type of flip flop circuit called **D flip flop**, Delay flip flop, D-type Bistable, D-type flip flop. The D flip flop is the most important flip flop from other clocked types. It ensures that at the same time, both the inputs, i.e., S and R, are never equal to 1. The Delay flip-flop is designed using a gated SR flip-flop with an inverter connected between the inputs allowing for a single input D(Data).

This single data input, which is labeled as "D" used in place of the "Set" input and for the complementary "Reset" input, the inverter is used. Thus, the level-sensitive D-type or D flip flop is constructed from a level-sensitive SR flip flop. So, here S=D and R= ~D (complement of D).

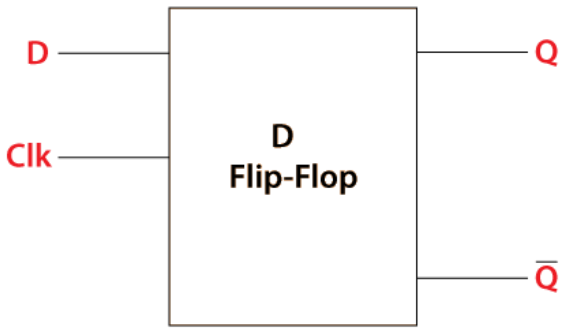


Figure 6.7:Symbolic representation of D-Flip Flop

We know that the SR flip-flop requires two inputs, i.e., one to "SET" the output and another to "RESET" the output. By using an inverter, we can set and reset the outputs with only one input as now the two input signals complement each other. In SR flip flop, when both the inputs are 0, that

state is no longer possible. It is an ambiguity that is removed by the complement in D-flip flop.

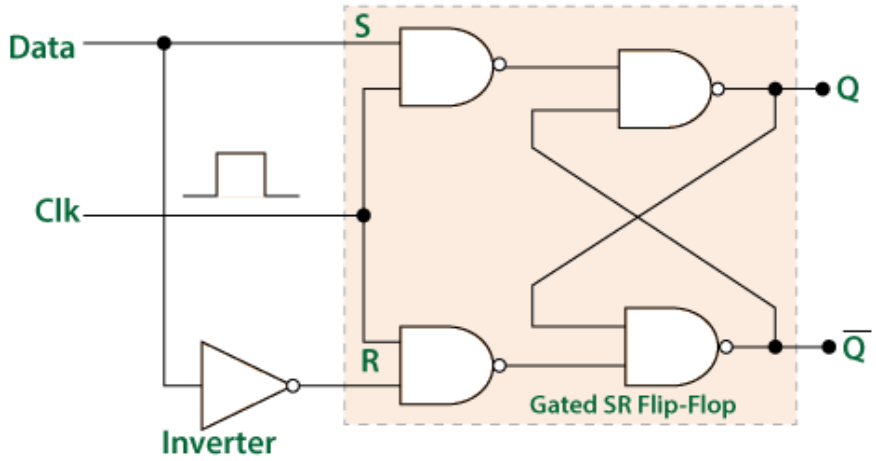


Figure 6.8:Logic circuit of D-Flip Flop

In D flip flop, the single input "D" is referred to as the "Data" input. When the data input is set to 1, the flip flop would be set, and when it is set to 0, the flip flop would change and become reset. However, this would be pointless since the output of the flip flop would always change on every pulse applied to this data input.

The "CLOCK" or "ENABLE" input is used to avoid this for isolating the data input from the flip flop's latching circuitry. When the clock input is set to true, the D input condition is only copied to the output Q. This forms the basis of another sequential device referred to as D Flip Flop.

When the clock input is set to 1, the "set" and "reset" inputs of the flip-flop are both set to 1. So it will not change the state and store the data present on its output before the clock transition occurred. In simple words, the output is "latched" at either 0 or 1.

Clock	D	Q	Q'	Description
↓ » 0	X	Q	Q'	Memory no change
↑ » 1	0	0	1	Reset Q » 0
↑ » 1	1	1	0	Set Q » 1

Figure 6.9:Truth Table of D-type Flip Flop

6.4 Edge-Triggered D Flip Flop

The latch responds to the data inputs (S-R or D) only when the enable input is activated. In many digital applications, however, it is desirable to limit the responsiveness of a latch circuit to a very short period of time instead of the entire duration that the enabling input is activated.

One method of enabling a multi-vibrator circuit is called edge triggering, where the circuit's data inputs have control only during the time that the enable input is transitioning from one state to another. Let's compare timing diagrams for a normal D latch versus one that is edge-triggered:

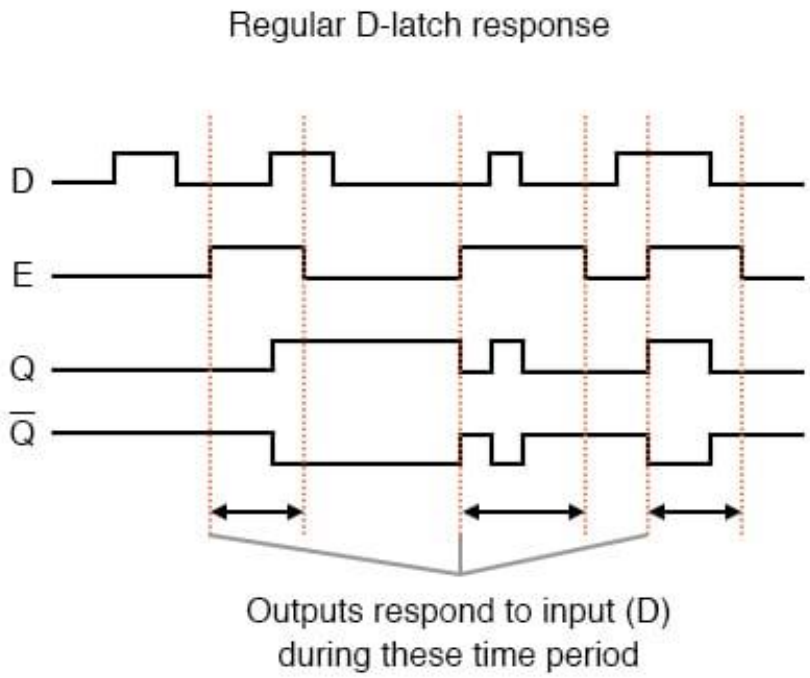


Figure 6.10:Regular response D-latch

In the figure 6.10 the outputs respond to input D whenever the enable (E) input is high, for however long it remains high. When the enable signal falls back to a low state, the circuit remains latched.

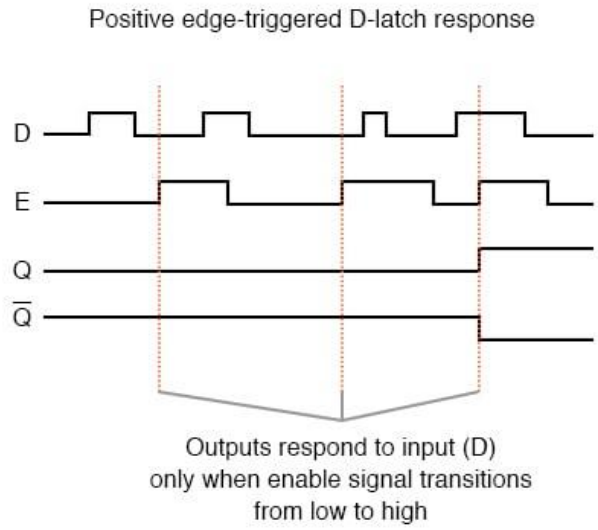


Figure 6.11:Positive edge trigger diagram of D-latch

In the figure 6.11diagram, we note a distinctly different response in the circuit output(s): it only responds to the D input during that brief moment of time when the enable signal *changes*, or *transitions, from low to high. This is known as positive edge-triggering.*

There is such a thing as *negative* edge triggering as well, and it produces the following response to the same input signals.

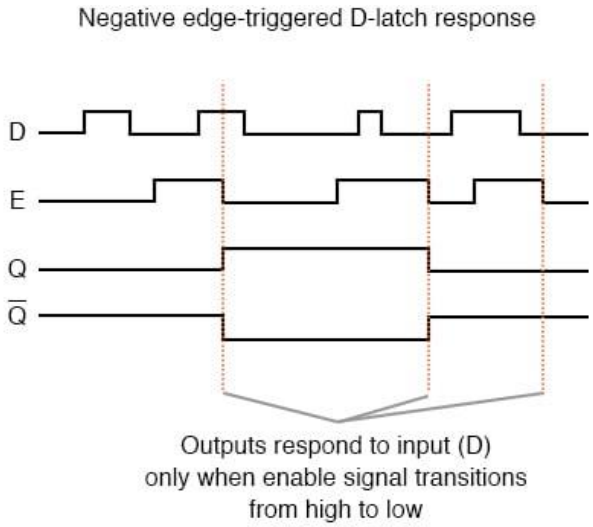


Figure 6.12:Negative edge trigger of D-latch

Whenever we enable a multi-vibrator circuit on the transitional edge of a square-wave enable signal, we call it a flip-flop instead of a latch.

Consequently, and edge-triggered S-R circuit is more properly known as an S-R flip-flop, and an edge-triggered D circuit as a D flip-flop. The enable signal is renamed to be the clock signal. Also, we refer to the data inputs (S, R, and D, respectively) of these flip-flops as synchronous inputs, because they have effect only at the time of the clock pulse edge (transition), thereby synchronizing any output changes with that clock pulse, rather than at the whim of the data inputs.But, how do we actually accomplish this edge-triggering? To create a "gated" S-R latch from a regular S-R latch is easy enough with a couple of AND gates, but how do we implement logic that only pays attention to the rising or falling edge of a changing digital signal?

What we need is a digital circuit that outputs a brief pulse whenever the input is activated for an arbitrary period of time, and we can use the output of this circuit to briefly enable the latch. We're getting a little ahead of ourselves here, but this is actually a kind of mono-stable multi-vibrator, which for now we'll call a pulse detector.

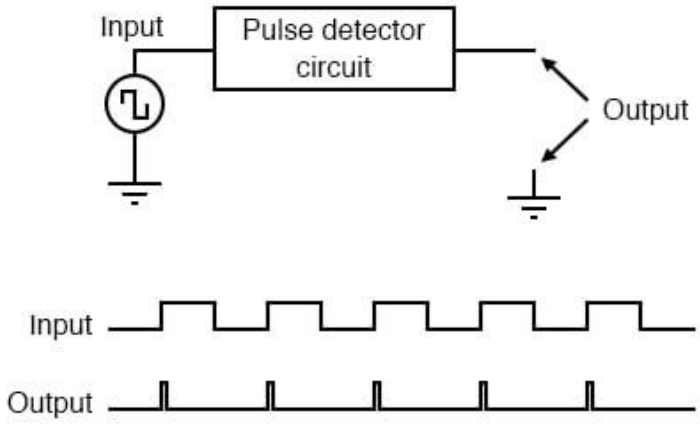


Figure 6.13:Pulse detector with timing diagram

Implementing this timing function with semiconductor components is actually quite easy, as it exploits the inherent time delay within every logic gate (known as propagation delay). What we do is take an input signal and split it up two ways, then place a gate or a series of gates in one of those signal paths just to delay it a bit, then have both the original signal and its delayed counterpart enter into a two-input gate that outputs a high signal for the brief moment of time that the delayed signal has not yet caught up to

the low-to-high change in the non-delayed signal. An example circuit for producing a clock pulse on a low-to-high input signal transition is shown here.

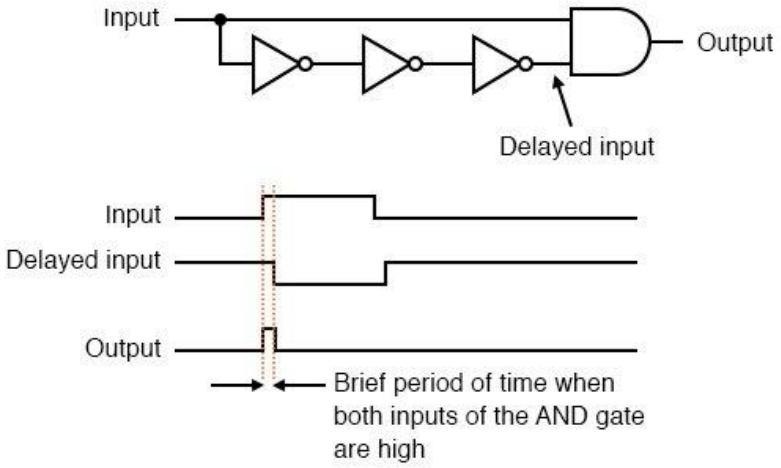
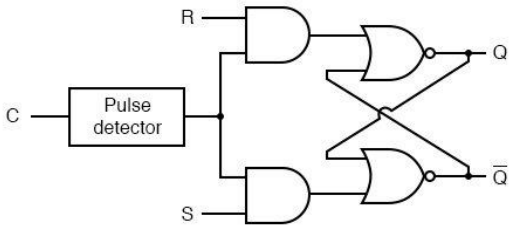


Figure 6.14:Pulse clock diagram with nand gate

Now that we know how a pulse detector can be made, we can show it attached to the enable input of a latch to turn it into a flip-flop. In this case, the circuit is S-R flip-flop.



C	S	R	Q	$\overline{Q}$
⌐	0	0	latch	latch
⌐	0	1	0	1
⌐	1	0	1	0
⌐	1	1	0	0
x	0	0	latch	latch
x	0	1	latch	latch
x	1	0	latch	latch
x	1	1	latch	latch

Figure 6.15:SR latch circuit

The triangle symbol next to the clock inputs tells us that these are edge-triggered devices, and consequently that these are flip-flops rather than latches.The symbols above are positive edge-triggered: that is, they “clock” on the rising edge (low-to-high transition) of the clock signal. Negative edge-triggered devices are symbolized with a bubble on the clock input line.

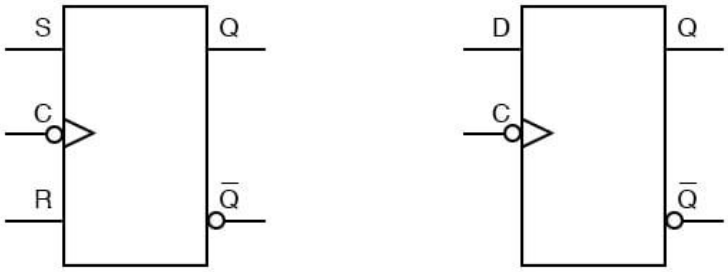


Figure 6.16:Symbolic representation of SR and D Flip Flop with clock

Both of the above flip-flops will “clock” on the falling edge (high-to-low transition) of the clock signal.

6.5 J-K FLIP FLOP

The basic S-R NAND flip-flop circuit has many advantages and uses in sequential logic circuits but it suffers from two basic switching problems.

1. The Set = 0 and Reset = 0 condition (S = R = 0) must always be avoided

2. If Set or Reset change state while the enable (EN) input is high the correct latching action may not occur

Then to overcome these two fundamental design problems with the SR flip-flop design, the JK flip Flop was developed. This simple JK flip Flop is the most widely used of all the flip-flop designs and is considered to be a universal flip-flop circuit. The two inputs labeled “J” and “K” are not shortened abbreviated letters of other words, such as “S” for Set and “R” for Reset, but are themselves autonomous letters chosen by its inventor Jack Kilby to distinguish the flip-flop design from other types.

The sequential operation of the JK flip flop is exactly the same as for the previous SR flip-flop with the same “Set” and “Reset” inputs. The difference this time is that the “JK flip flop” has no invalid or forbidden input states of the SR Latch even when S and R are both at logic “1”.

The JK flip flop is basically a gated SR flip-flop with the addition of a clock input circuitry that prevents the illegal or invalid output condition that can

occur when both inputs S and R are equal to logic level “1”. Due to this additional clocked input, a JK flip-flop has four possible input combinations, “logic 1”, “logic 0”, “no change” and “toggle”. The symbol for a JK flip flop is similar to that of an SR Bistable Latch as seen in the previous tutorial except for the addition of a clock input.

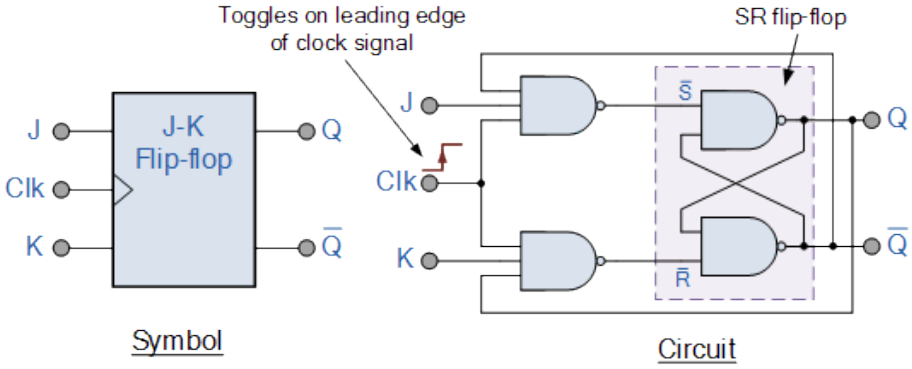


Figure 6.14 J-K FLIP FLOP with clock

Both the S and the R inputs of the previous SR bistable have now been replaced by two inputs called the J and K inputs, respectively after its inventor Jack Kilby. Then this equates to: J = S and K = R.

The two 2-input AND gates of the gated SR bistable have now been replaced by two 3-input NAND gates with the third input of each gate connected to the outputs at Q and Q. This cross coupling of the SR flip-flop allows the previously invalid condition of S = “1” and R = “1” state to be used to produce a “toggle action” as the two inputs are now interlocked.

If the circuit is now “SET” the J input is inhibited by the “0” status of Q through the lower NAND gate. If the circuit is “RESET” the K input is inhibited by the “0” status of Q through the upper NAND gate. As Q and Q are always different we can use them to control the input. When both inputs J and K are equal to logic “1”, the JK flip flop toggles as shown in the following truth table.

Clock	Input		Output		Description
ClK	J	K	Q	Q-	
X	0	0	1	0	Memory
X	0	0	0	1	No change
⁻↓_	0	1	1	0	Reset Q » 0
X	0	1	0	1	
⁻↓_	1	0	0	1	Set Q » 1
X	1	0	1	0	
⁻↓_	1	1	0	1	Toggle
⁻↓_	1	1	1	0	

Then the JK flip-flop is basically an SR flip flop with feedback which enables only one of its two input terminals, either SET or RESET to be active at any one time under normal switching thereby eliminating the invalid condition seen previously in the SR flip flop circuit.

However, if both the J and K inputs are HIGH at logic “1” (J = K = 1), when the clock input goes HIGH, the circuit will “toggle” as its outputs switch and change state complementing each other. This results in the JK flip-flop acting more like a T-type toggle flip-flop when both terminals are “HIGH”. However, as the outputs are fed back to the inputs, this can cause the output at Q to oscillate between SET and RESET continuously after being complemented once.While this JK flip-flop circuit is an improvement on the clocked SR flip-flop it also suffers from timing problems called “race” if the output Q changes state before the timing pulse of the clock input has time to go “OFF”. To avoid this the timing pulse period (T) must be kept as short as possible (high frequency). As this is sometimes not possible with basic JK flip-flops built using basic NAND or NOR gates, far more advanced master-slave (edge-triggered) flip-flops were developed which are more stable.

6.5.1 Master-Slave JK Flip-flop

The master-slave flip-flop eliminates all the timing problems by using two SR flip-flops connected together in a series configuration. One flip-flop acts as the “Master” circuit, which triggers on the leading edge of the clock pulse while the other acts as the “Slave” circuit, which triggers on the falling edge of the clock pulse. This results in the two sections, the master section and the

slave section being enabled during opposite half-cycles of the clock signal. The TTL 74LS73 is a Dual JK flip-flop IC, which contains two individual JK type bistable's within a single chip enabling single or master-slave toggle flip-flops to be made. Other JK flip flop IC's include the 74LS107 Dual JK flip-flop with clear, the 74LS109 Dual positive-edge triggered JK flip flop and the 74LS112 Dual negative-edge triggered flip-flop with both preset and clear inputs.

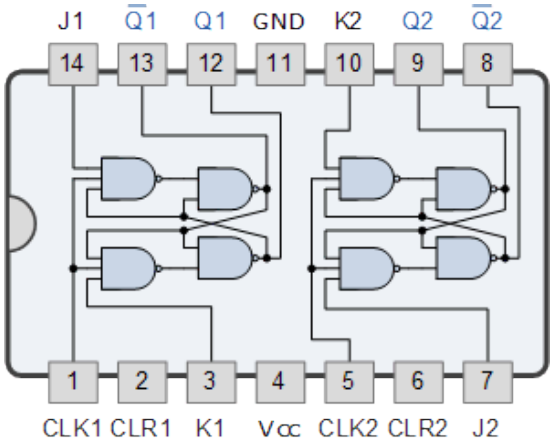


Figure 6.15:IC 74LS73

PART-I SAMPLE MULTIPLE CHOISE QUESTIONS

1. The flip flops are categorized into _____
(a) one (b) two (c) three (d) four

2. What is the standard form of S-R flip flop?
(a) set-reset (b)simple reset (c) single reset (d) Both b & c

3. When the set is enabled in S-R flip flop then the output will be
(a) Set (b) reset (c) no change (d) intermediate

4. When both set and reset are disabled in S-R flip flop then the output
(a) Set (b) reset (c) no change (d) intermediate

5. In which flip flop the present input will be the next output?
(a) S-R (b)J-K (c) D (d) T

6. The J-K flip flops has _____ memory
(a) Temporary (b) Random (c) Non random (d)True

7. There are total _____ steps for flip flop conversions
(a) 1 (b)2 (c) 3 (d)5

8. In SR to JK flip flop conversion which one is an available flip flop?
(a) SR (b) JK (c) T (d)Both SR &JK

9. The operation of the flip flop is slow?
(a) True (b)False (c) None of these(d)Both

10. How many types of latches are there?
(a) 1 (b) 2 (c) 6 (d) 4

ANSWER KEY

1.	D	2.	A	3.	A	4.	C	5.	C
6.	D	7.	D	8.	A	9.	A	10.	C

PART-II SAMPLE SHORT QUESTIONS

1. Define the term Flip Flop?
2. What is SR latch?
3. What is RS Flip Flop?
4. Draw the circuit diagram of RS Flip Flop?
5. Draw RS Flip Flop using NAND gate?
6. What is clocked RS Flip Flop?
7. Draw the circuit diagram of clocked RS Flip Flop?
8. Draw the four cycles of clock?
9. Draw the truth table of clocks RSFF?.
10. Define the term D-Flip Flop?
11. Draw the logic circuit of D-Flip Flop?
12. Draw the truth table of D-Flip Flop?
13. What is Edge triggered D Flip Flop?
14. What is pulse detector?
15. Draw the circuit diagram of Pulse detector?
16. What is JK Flip Flop?
17. Draw the truth table of J-K Flip Flop?
18. What is master slave JK Flip Flop?
19. Draw the logic diagram of JK Flip Flop?
20. Which IC is used for master slave JK Flip Flop?

PART-III SAMPLE LONG QUESTIONS

1. What is Flip Flop also explain its types?
2. Explain JK Flip Flop?
3. Explain D-Flip Flop?.
4. Explain Edge trigger D-Flip Flop?
5. Explain clocked RS Flip Flop?

CHAPTER NO. 07

CLOCKS & TIMERS

7.1 Introduction

This chapter describes clocks and timers and also its applications. Computers usage the inner clock to synchronize their entire calculations. The clock confirms that the many circuits inside a computer work together at the same time. The direct time any computer is accomplished of acting is one clock. It may be called one vibration of the clock chip.

Clock speed is dignified by how many ticks per second the clock makes. The unit of a dimension called a hertz (Hz).The speed of a computer processor is measured in clock speed.

The basic function of a hardware timer is to keep exact track of time.

A timer is up till now another peripheral that's available to the microcontroller; thus it may be controlled using registers.

Hardware timers are mostly used for timing and counting operations. They allow the processor to carry on with some other process while the timer process runs.

7.2 Chapter Objectives

1. Understand the basics of clocks and their application
2. Understand the basic of Timers and their function with application.

7.3 Clocks

Clock is a circuit which provides periodic signals to digital circuit or digital system. These signals are used to trigger or synchronize the different digital circuits.

7.3.1 TTL CLOCK

Transistor-transistor logic (TTL) is a digital logic design in which bipolar transistor s act on direct-current pulses. Many TTL logic gates are typically fabricated onto a single integrated circuit (IC). TTL ICs usually have four-digit numbers beginning with 74 or 54.

A TTL device employs transistor s with multiple emitters in gates

having more than one input. TTL is characterized by high switching speed (in some cases upwards of 125 MHz), and relative immunity to noise. Its principle drawback is the fact that circuits using TTL draw more current than equivalent circuits using metal oxide semiconductor (MOS) logic. Low-current TTL devices are available, but the reduced current demand comes at the expense of some operating speed.

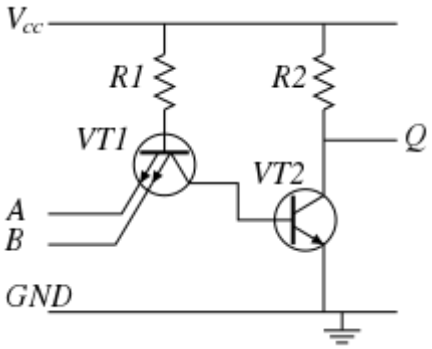


Figure:7.1 TTL clock

7.3.1.1 Explanation

TTL inputs are the emitters of bipolar transistors. In the case of NAND inputs, the inputs are the emitters of multiple-emitter transistors, functionally equivalent to multiple transistors where the bases and collectors are tied together. The output is buffered by a common emitter amplifier.

When all the inputs are held at high voltage, the base–emitter junctions of the multiple-emitter transistor are reverse-biased. Unlike DTL, a small "collector" current (approximately 10μA) is drawn by each of the inputs. This is because the transistor is in reverse-active mode. An approximately constant current flows from the positive rail, through the resistor and into the base of the multiple emitter transistor.[13] This current passes through the base–emitter junction of the output transistor, allowing it to conduct and pulling the output voltage low (logical zero).

An input logical zero. Note that the base–collector junction of the

multiple-emitter transistor and the base–emitter junction of the output transistor are in series between the bottom of the resistor and ground. If one input voltage becomes zero, the corresponding base–emitter junction of the multiple-emitter transistor is in parallel with these two junctions. A phenomenon called current steering means that when two voltage-stable elements with different threshold voltages are connected in parallel, the current flows through the path with the smaller threshold voltage. That is, current flows out of this input and into the zero (low) voltage sources. As a result, no current flows through the base of the output transistor, causing it to stop conducting and the output voltage becomes high (logical one). During the transition the input transistor is briefly in its active region; so it draws a large current away from the base of the output transistor and thus quickly discharges its base. This is a critical advantage of TTL over DTL that speeds up the transition over a diode input structure.

The main disadvantage of TTL with a simple output stage is the relatively high output resistance at output logical "1" that is completely determined by the output collector resistor. It limits the number of inputs that can be connected (the fan-out). Some advantage of the simple output stage is the high voltage level (up to VCC) of the output logical "1" when the output is not loaded.

A common variation omits the collector resistor of the output transistor, making an open-collector output. This allows the designer to fabricate logic by connecting the open-collector outputs of several logic gates together and providing a single external pull-up resistor. If any of the logic gates becomes logic low (transistor conducting), the combined output will be low. Examples of this type of gate are the 7401 and 7403 series. Open-collector outputs of some gates have a higher maximum voltage.

7.3.1.2 Advantages

There are following advantages of TTL clock which are given below

1. No external triggering required.
2. Circuit design is simple.

3. Inexpensive.
4. Can function continuously.

7.3.1.3 Disadvantages

The draw backs of using an TTL clock are as follow

1. Energy absorption is more within the circuit.
2. Output signal is of low energy.
3. Duty cycle is less than or equal to 50% cannot be achieved.

7.3.1.4 Applications

It is used in much application such as amateur radio equipment, Morse code generators, timer circuits, analog circuits and TV systems.

7.3.2 Astable Multivibrator

The Astable Multivibrator is another type of cross-coupled transistor switching circuit that has NO stable output states as it changes from one state to the other all the time.

The astable circuit consists of two switching transistors, a cross-coupled feedback network, and two time delay capacitors which allow oscillation between the two states with no external triggering to produce the change in state.

The basic transistor circuit for an Astable Multivibrator produces a square wave output from a pair of grounded emitter cross-coupled transistors. Both transistors either NPN or PNP, in the multivibrator are biased for linear operation and are operated as Common Emitter Amplifiers with 100% positive feedback.

7.3.2.1 Construction

Two transistors named Q1 and Q2 are connected in feedback to one another. The collector of transistor Q1 is connected to the base of transistor Q2 through the capacitor C1 and vice versa. The emitters of both the transistors are connected to the ground. The collector load resistors R1 and R4 and the biasing resistors R2 and R3 are of equal values. The capacitors C1 and C2 are of equal values. The following

figure shows the circuit diagram for Astable Multivibrator.

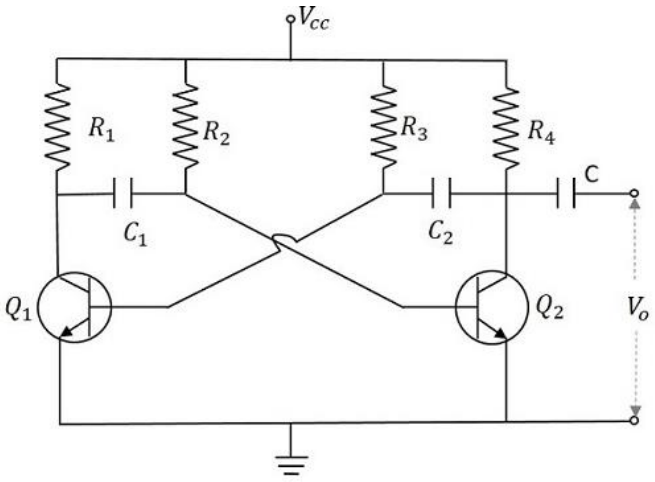


Figure: 7.2 Circuit diagram of Astable Multivibrator

The output waveforms at the collectors of Q_1 and Q_2 are shown in the following figures.

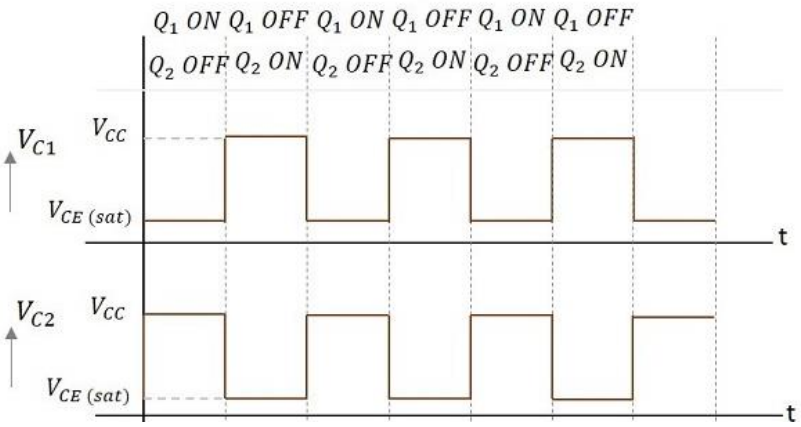


Figure: 7.3 Wave form of of Astable Multivibrator

7.3.3 Monostable Multivibrator

A monostable multivibrator, as the name implies, has only one stable state. When the transistor conducts, the other remains in non-conducting state. A stable state is such a state where the transistor remains without being altered, unless disturbed by some external trigger pulse. As Mon stable works on the same principle, it has another name called as One-shot Multivibrator.

7.3.3.1 Construction

Two transistors Q_1 and Q_2 are connected in feedback to one another. The collector of transistor Q_1 is connected to the base of transistor Q_2 through the capacitor C_1. The base Q_1 is connected to the collector of Q_2 through the resistor R_2 and capacitor C. Another dc supply voltage $-V_{BB}$ is given to the base of transistor Q_1 through the resistor R_3. The trigger pulse is given to the base of Q_1 through the capacitor C_2 to change its state. R_{L1} and R_{L2} are the load resistors of Q_1 and Q_2.

One of the transistors, when gets into a stable state, an external trigger pulse is given to change its state. After changing its state, the transistor remains in this quasi-stable state or Meta-stable state for a specific time period, which is determined by the values of RC time constants and gets back to the previous stable state. The following figure shows the circuit diagram of a Mono stable Multivibrator.

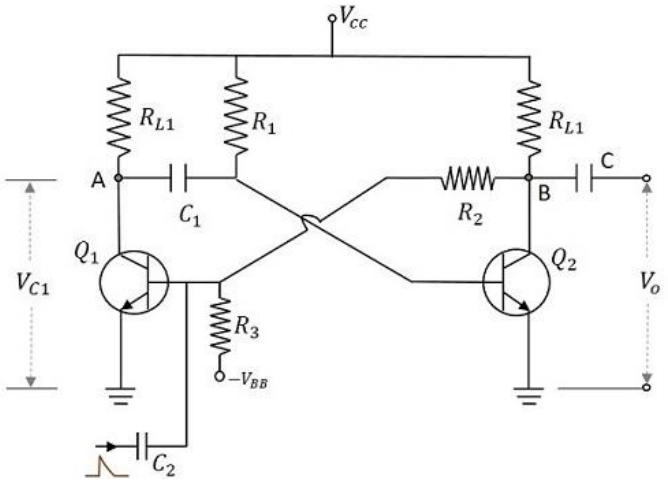


Figure: 7.4 Circuit diagram of Mono stable Multivibrator

The output waveforms at the collectors of Q1 and Q2 along with the trigger input given at the base of Q1 are shown in the following figures

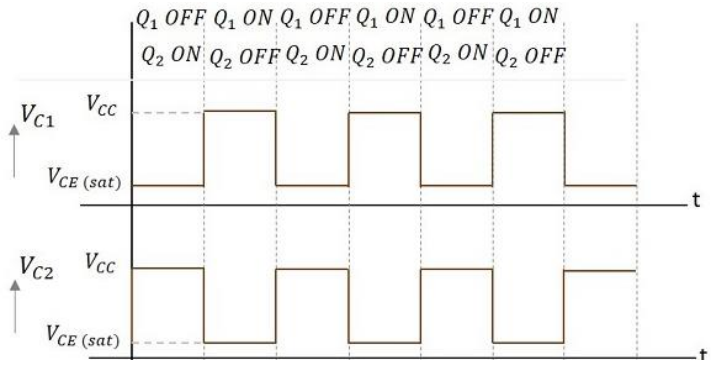


Figure: 7.5 Wave form of of Mono stable Multivibrator

7.3.4 Bi-stable Multivibrator

A Bistable Multivibrator has two stable states. The circuit stays in any one of the two stable states. It continues in that state, unless an external trigger pulse is given. This Multivibrator is also known as Flip-flop. This circuit is simply called as Binary.

7.3.4.1 Construction

Two similar transistors Q1 and Q2 with load resistors RL1 and RL2 are connected in feedback to one another. The base resistors R3 and R4 are joined to a common source –VBB. The feedback resistors R1 and R2 are shunted by capacitors C1 and C2 known as Commutating Capacitors. The transistor Q1 is given a trigger input at the base through the capacitor C3 and the transistor Q2 is given a trigger input at its base through the capacitor C4.The capacitors C1 and C2 are also known as Speed-up Capacitors. The following figure shows the circuit diagram of a self-biased Bistable Multivibrator.

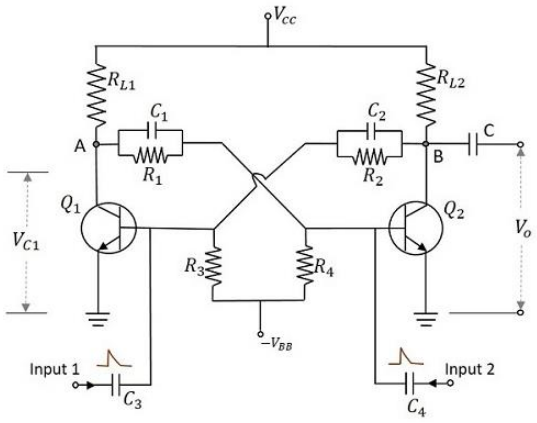


Figure: 7.6 Circuit diagram of Bi- stable Multivibrator

The output waveforms at the collectors of Q1 and Q2 along with the trigger inputs given at the bases of QW and Q2 are shown in the following figures.3

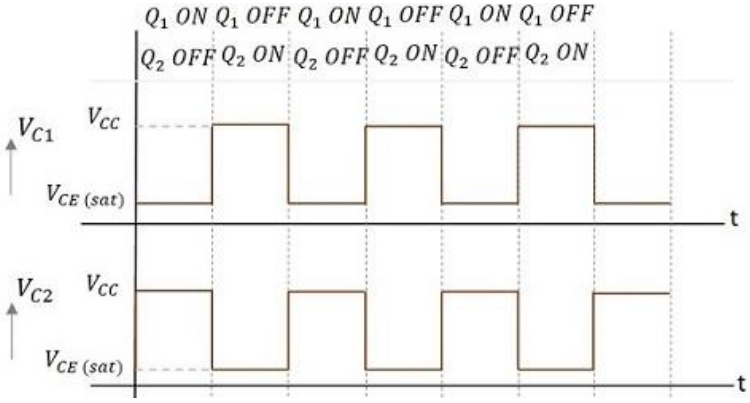


Figure: 7.7 wave form of Bi- stable Multivibrator

7.3.5 555 Timer:

The 555 timer is versatile and widely used IC which is used as clock generator.it can be configured in two different modes as either a monostable multivibrator or as an a stable multivibrator

555 timer provide the time delay from few microseconds to many hours.it was released in two parts: NE555 and SE 555.The NE 555 parts were of commercial usage with a temperature range of 0-70 degree and the SE 555 parts are designed to meet the military standards with the temperature range of -55-125 degree.it is monolithic IC and was the first commercially available timer IC.

7.3.5.1 Pin configuration of 555 Timer:

The 555 timer is available in 8 pin configuration and in metal package,8-pins Mini Dual in- line package and 14-pin DIP.The 14 pin DIP is IC-556 which consists of two 555 timers.

The 8 pin are most commonly used.The pin out diagrams of 555 timers is shown below.

1. **Pin1-GND**

Ground reference voltage, low level (0 V)

2. **Trigger Terminal**

The OUT pin goes high and a timing interval starts when this input falls below 1/2 of CTRL voltage (which is typically 1/3 Vcc, CTRL being 2/3 Vcc by default if CTRL is left open). In other words, OUT is high as long as the trigger low. Output of the timer totally depends upon the amplitude of the external trigger voltage applied to this pin.

3. Output

This output is driven to approximately 1.7 V below +Vcc, or to GND.

4. Reset

A timing interval may be reset by driving this input to GND, but the timing does not begin again until RESET rises above approximately 0.7 volts. Overrides TRIG which overrides threshold.

5. Control Voltage

Provides "control" access to the internal voltage divider (by default, 2/3 Vcc).

6. Threshold

The timing (OUT high) interval ends when the voltage at threshold is greater than that at CTRL (2/3 Vcc if CTRL is open).

7. Discharge

Open collector output which may discharge a capacitor between intervals. In phase with output.

8. Supply Voltage

Positive supply voltage, which is usually between 3 and 15 V depending on the variation.

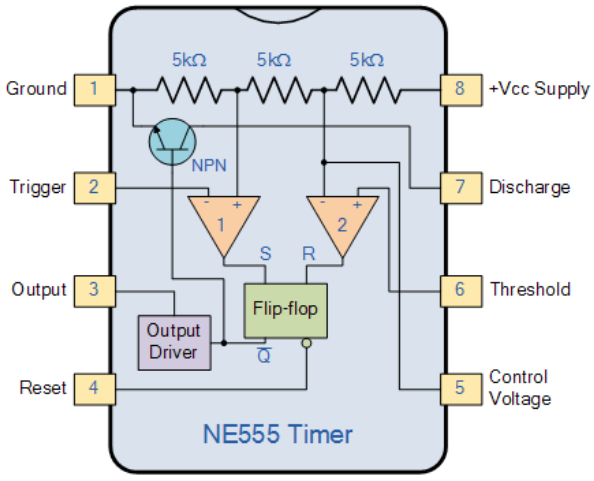


Figure 7.8 Pin Diagram of 555 Timer

7.3.5.2 Working

The 555 Timers name comes from the fact that there are three 5kΩ

resistors connected together internally producing a voltage divider network between the supply voltage at pin 8 and ground at pin 1. The voltage across this series resistive network holds the negative inverting input of comparator two at 2/3Vcc and the positive non-inverting input to comparator one at 1/3Vcc.

The two comparators produce an output voltage dependent upon the voltage difference at their inputs which is determined by the charging and discharging action of the externally connected RC network. The outputs from both comparators are connected to the two inputs of the flip-flop which in turn produces either a “HIGH” or “LOW” level output at Q based on the states of its inputs. The output from the flip-flop is used to control a high current output switching stage to drive the connected load producing either a “HIGH” or “LOW” voltage level at the output pin.

The most common use of the 555 timer oscillator is as a simple astable oscillator by connecting two resistors and a capacitor across its terminals to generate a fixed pulse train with a time period determined by the time constant of the RC network. But the 555 timer oscillator chip can also be connected in a variety of different ways to produce Monostable or Bistable multivibrators as well as the more common Astable Multivibrator.

7.3.6 555 Timer as Mono stable Multivibrator

The operation and output of the 555 timer monostable is exactly the same as that for the transistorized one we look at previously in the Monostable Multivibrators tutorial. The difference this time is that the two transistors have been replaced by the 555 timer device. Consider the 555 timer monostable circuit below.

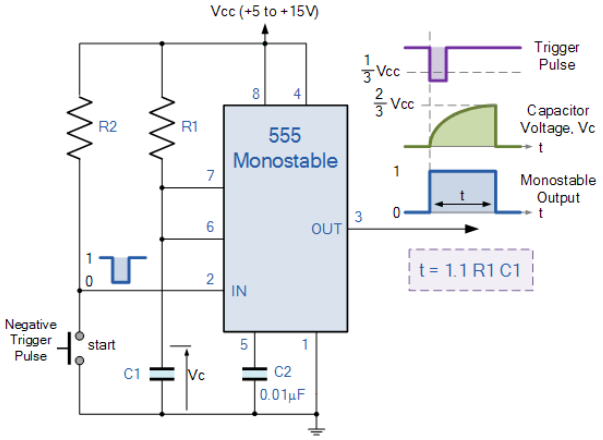


Figure 7.9 Circuit Diagram of 555 time Monostable Multivibrator

7.3.6.1 Working

When a negative (0V) pulse is applied to the trigger input (pin 2) of the Monostable configured 555 Timer oscillator, the internal comparator, (comparator No1) detects this input and “sets” the state of the flip-flop, changing the output from a “LOW” state to a “HIGH” state. This action in turn turns “OFF” the discharge transistor connected to pin 7, thereby removing the short circuit across the external timing capacitor, C1.

This action allows the timing capacitor to start to charge up through resistor, R1 until the voltage across the capacitor reaches the threshold (pin 6) voltage of 2/3Vcc set up by the internal voltage divider network. At this point the comparators output goes “HIGH” and “resets” the flip-flop back to its original state which in turn turns “ON” the transistor and discharges the capacitor to ground through pin 7. This causes the output to change its state back to the original stable “LOW” value awaiting another trigger pulse to start the timing process over again. Then as before, the Monostable Multivibrator has only “ONE” stable state.

The Monostable 555 Timer circuit triggers on a negative-going pulse applied to pin 2 and this trigger pulse must be much shorter than the output pulse width allowing time for the timing capacitor to charge and then discharge fully. Once triggered, the 555 Monostable will remain in this “HIGH” unstable output state until the time period set up by the R1 x C1 network has elapsed. The amount of time that the output voltage remains “HIGH” or at a logic “1” level

7.3.7 555 Timer as A stable Multivibrator

Astable Multivibrator mode of 555 timers IC is also called free running or self-triggering mode. Unlike Monostable Multivibrator mode it doesn’t have any stable state; it has two quasi stable states (HIGH and LOW). No external triggering is required in Astable mode, it automatically interchange its two states on a particular interval, hence generates a rectangular waveform. This time duration of HIGH and LOW output has been determined by the external resistors (R1 and R2) and a capacitor (C1). Astable mode works as a oscillator circuit, in which output oscillate at a particular frequency and generate pulses in rectangular wave form.

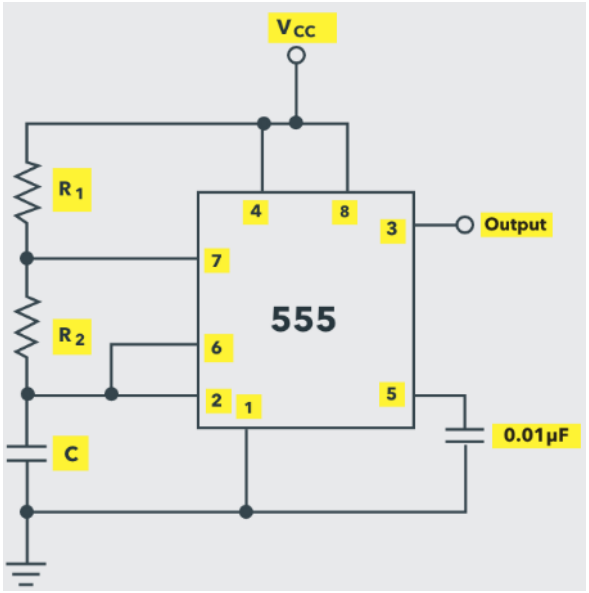


Figure 7.10 555 Timer as Astable Multivibrator

7.3.7.1 Working

Initially, on power-up, the flip-flop is RESET (and hence the output of the timer is low). As a result, the discharge transistor is driven to saturation (as it is connected to Q'). The capacitor C of the timing circuit is connected at Pin 7 of the IC 555 and will discharge through the transistor. The output of the timer at this point is low. The voltage across the capacitor is nothing but the trigger voltage. So, while discharging, if the capacitor voltage becomes less than 1/3 VCC, which is the reference voltage to trigger comparator (comparator 2), the output of the comparator 2 will become high. This will SET the flip-flop and hence the output of the timer at pin 3 goes to HIGH.

This high output will turn OFF the transistor. As a result, the capacitor C starts charging through the resistors R1 and R2. Now, the capacitor voltage is same as the threshold voltage (as pin 6 is connected to the capacitor resistor junction). While charging, the capacitor voltage increases exponentially towards VCC and the moment it crosses 2/3 VCC, which is the reference voltage to threshold comparator (comparator 1), its output becomes high.

As a result, the flip-flop is RESET. The output of the timer falls to LOW. This low output will once again turn on the transistor which provides a discharge path to the capacitor. Hence the capacitor C will discharge through the resistor R2. And hence the cycle continues. Thus, when the

capacitor is charging, the voltage across the capacitor rises exponentially and the output voltage at pin 3 is high. Similarly, when the capacitor is discharging, the voltage across the capacitor falls exponentially and the output voltage at pin 3 is low. The shape of the output waveform is a train of rectangular pulses. The waveforms of capacitor voltage and the output in a stable mode are shown below.

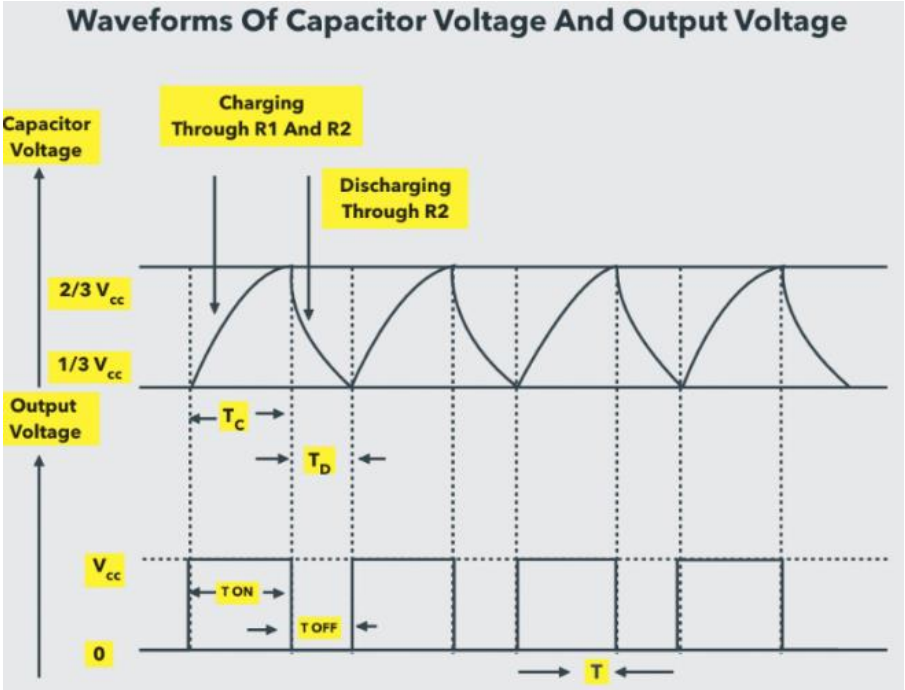


Figure 7.11 Wave form of A stable Multivibrator of 555 timer

7.3.7.2 Applications of Clock and timers:

There are following applications of clock and timers which are given below

1. Clock pulse is necessary for triggering the sequential circuits.
2. There are used in design of frequency counters
3. Provide time delay.

Part-I Multiple Choice Questions (MCQs)

1. Clock is a circuit which provides
   (a)Periodic signal (b) current (c) voltage (d) All of these
2. A clock pulse has a specific:
   (a) Amplitude (b) Frequency (c) duty cycle (d) All of these
3. A multi vibrator has no stable state is called
   (a) Astable (b) Monostable (c) Bistable (d) None of these
4. A multi vibrator has one stable state is called
   (a) Astable (b) Monostable (c) Bistable (d) None of these
5. A multi vibrator has two stable state is called
   (a) Astable (b) Monostable (c) Bistable (d) None of these
6. Mon stable multivibrator is also called
   (a) 1 shot (b) 2 shot (c) 3 shot (d) None of these
7. 555 Timer is used as Multi vibrator
   (a) Astable (b) Monostable (c) Bistable (d) None of these
8. Pin no of 555 timers for Reset is
   (a) 1 (b) 2 (c) 3 (d)4
9. Pin no of 555 timers for Ground is
   (a) 1 (b) 2 (c) 3 (d)4
10. Pin no of 555 timers for output is
    (a) 1 (b) 2 (c) 3 (d)4

ANSWER KEY

1.	a	2.	d	3.	a	4.	b	5.	c
6.	a	7.	c	8.	d	9.	a	10.	c

PART-II SAMPLE SHORT QUESTIONS

1. Define clock?
2. Enlist the types of multivibrator?
3. Define a stable multivibrator?
4. Define Mon stable multivibrator?
5. Define mono stable multivibrator?
6. Define bi-stable multivibrator?
7. Define Timer?

8. Define 555 timers?
9. Draw pin diagram of 555 timer/
10. Define trigger pin of 555 timers?
11. Define control voltage pin of 555 timer?
12. Define reset pin of 55 timers?
13. Define discharge pin of 555 timers?
14. Write applications of clock and timer?
15. Draw the circuit diagram of TTL clock?
16. Draw the circuit diagram of mono stable multi vibrator?
17. Draw the circuit diagram of Astable multi vibrator?
18. What is the importance of clock and timer?
19. Write the advantages of A stable multi vibrator?
20. Write the advantages of Mono stable multi vibrator?

Part-III Long Questions

1. Explain TTL clock with diagram?
2. Define 555 timers; explain its basic operation with internal diagram?
3. Explain 555 timer as A stable multi vibrator with diagram?
4. Explain 555 timer as Monostable multi vibrator with diagram?
5. Define A stable, Monostable and Bi-stable multivibrator?

CHAPTER NO. 08

SHIFT REGISTER

8.1 Introduction

Shift Registers are sequential logic circuits, capable of storage and transfer of data. They are made up of Flip Flops which are connected in such a way that the output of one flip flop could serve as the input of the other flip-flop, depending on the type of shift registers being created.

8.2 Chapter Objectives

3. Understand the basics of shift registers.
4. Understand the basic of PIPO,SISO,PISO,LIFO
5. Understand the shift right and shift left register.

8.3 Shift Register

A group of flip flops which is used to store multiple bits of data and the data is moved from one flip flop to another is known as Shift Register. The bits stored in registers shifted when the clock pulse is applied within and inside or outside the registers. To form an n-bit shift register, we have to connect n number of flip flops. So, the number of bits of the binary number is directly proportional to the number of flip flops. The flip flops are connected in such a way that the first flip flop's output becomes the input of the other flip flop.

A Shift Register can shift the bits either to the left or to the right. A Shift Register, which shifts the bit to the left, is known as "Shift left register", and it shifts the bit to the right, known as "Right left register".The shift register is classified into the following types

1. Shift Right Register
2. Shift Left Register
3. Serial In Serial Out
4. Serial In Parallel Out
5. Parallel In Serial Out
6. Parallel in parallel out

8.3.1 Shift Left Register

Shift register in which the data is transferred or shifted from right to left is

called shift left register as shown in figure.The figure shown that shift left register consists of four flip flop,in which the output of previous flip flop is connected with the input of next flip flop.

When the rising or positive edge of clock pulse is applied then the data is shift one bit per clock pulse from right to left.

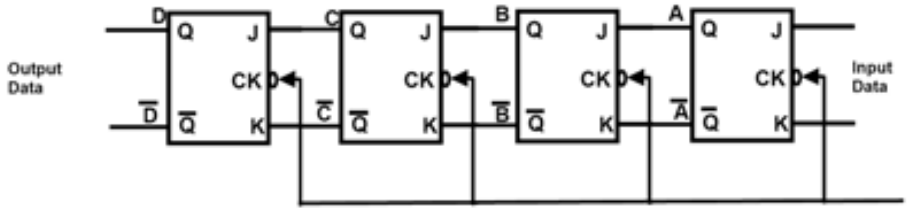


Figure 8.1:Block Diagram of Shift Left Register

8.3.2 Shift Right Register

Shift register in which the data is transferred or shifted from left to right is called shift right register as shown in below figure 8.2.The figure shown that shift right register consists of four flip flop in which output of previous is the input of next flip flop.

When the rising or positive pulse edge of clock pulse is applied then the data is transferred one bit per clock pulse from left to right.

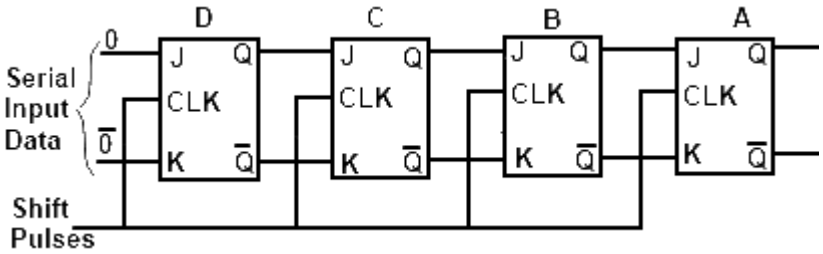


Figure 8.2: Block diagram of Shift Right Register

8.3.3 Serial in Serial out(SISO)

In "Serial Input Serial Output", the data is shifted "IN" or "OUT" serially. In SISO, a single bit is shifted at a time in either right or left direction under clock control.

Initially, all the flip-flops are set in "reset" condition i.e. Y3 = Y2 = Y1 = Y0 = 0. If we pass the binary number 1111, the LSB bit of the number is applied first to the Din bit. The D3 input of the third flip flop, i.e., FF-3, is directly

connected to the serial data input D3. The output Y3 is passed to the data input d2 of the next flip flop. This process remains the same for the remaining flip flops. The block diagram of the "Serial IN Serial OUT" is given below.

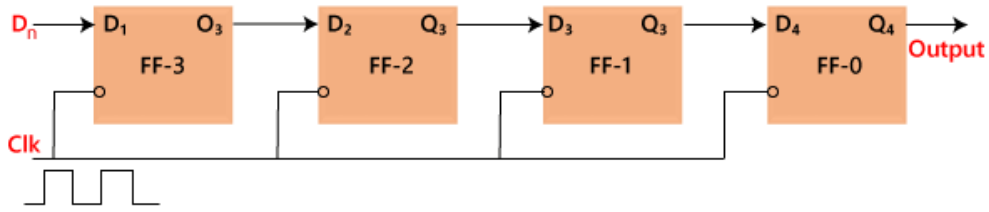


Figure 8.3 Block diagram of SISO

8.3.3.1 Working

When the clock signal application is disabled, the outputs Y3 Y2 Y1 Y0 = 0000. The LSB bit of the number is passed to the data input Din, i.e., D3. We will apply the clock, and this time the value of D3 is 1. The first flip flop, i.e., FF-3, is set, and the word is stored in the register at the first falling edge of the clock. Now, the stored word is 1000.

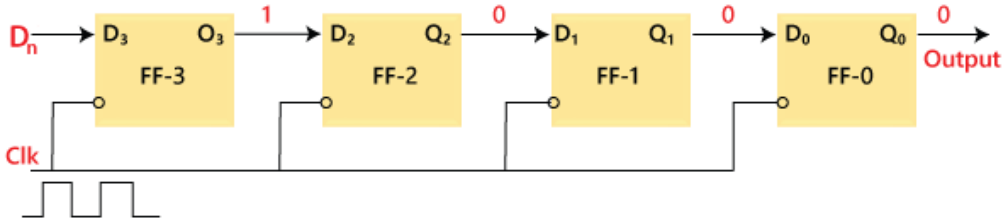


Figure 8.4: Block diagram of clock pulse 1 of SISO

The next bit of the binary number, i.e., 1, is passed to the data input D2. The second flip flop, i.e., FF-2, is set, and the word is stored when the next negative edge of the clock hits. The stored word is changed to 1100.

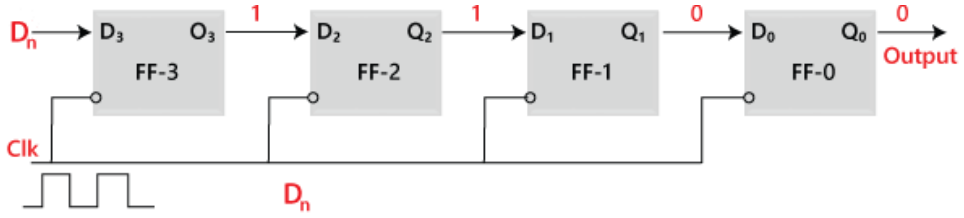


Figure 8.5: Block diagram of clock pulse 2 of SISO

The next bit of the binary number, i.e., 1, is passed to the data input D1, and

the clock is applied. The third flip flop, i.e., FF-1, is set, and the word is stored when the negative edge of the clock hits again. The stored word is changed to 1110.

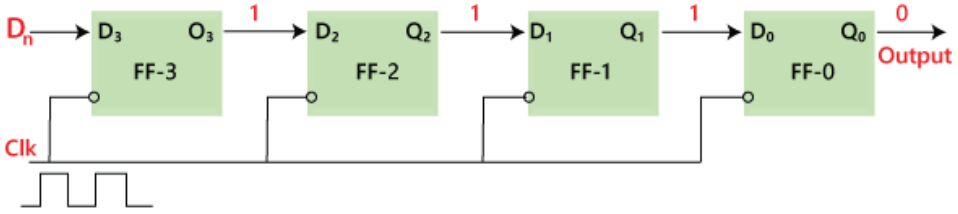


Figure 8.6: Block diagram of clock pulse 3 of SISO

Similarly, the last bit of the binary number, i.e., 1, is passed to the data input D0, and the clock is applied. The last flip flop, i.e., FF-0, is set, and the word is stored when the clock's negative edge arrives. The stored word is changed to 1111.

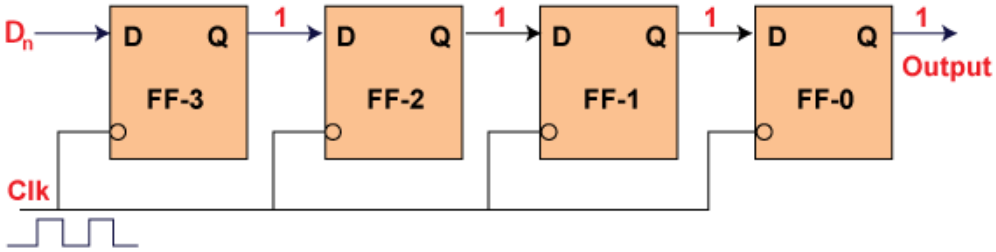


Figure 8.7: Block diagram of clock pulse 4 of SISO

The direction of travel of bits are shown in below figure

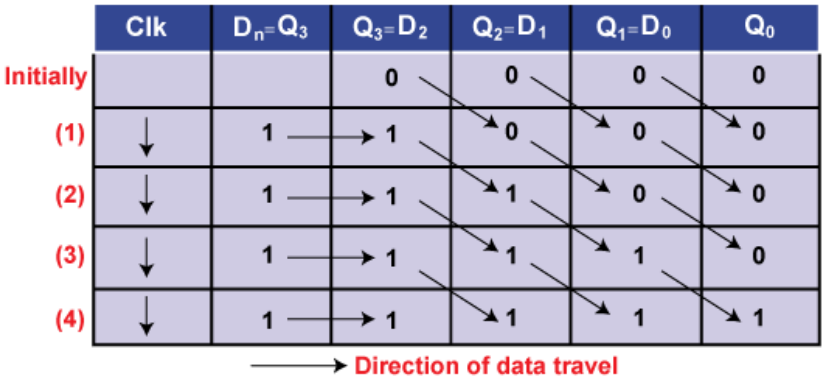


Figure 8.8:Direction of travel of bits in SISO

8.3.4 Serial in Parallel out(SIPO)

In the "Serial IN Parallel OUT" shift register, the data is passed serially to the

flip flop, and outputs are fetched in a parallel way. The data is passed bit by bit in the register, and the output remains disabled until the data is not passed to the data input. When the data is passed to the register, the outputs are enabled, and the flip flops contain their return value.

Below is the block diagram of the 4-bit serial in the parallel-out shift register. The circuit having four D flip-flops contains a clear and clock signal to reset these four flip flops. In SIPO, the input of the second flip flop is the output of the first flip flop, and so on. The same clock signal is applied to each flip flop since the flip flops synchronize each other.

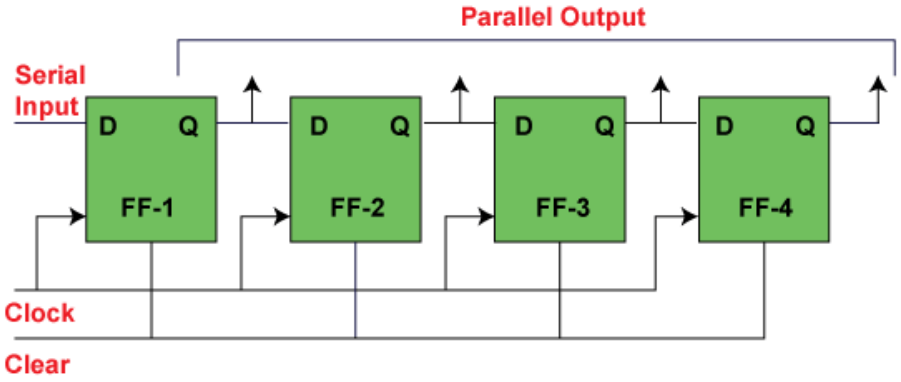


Figure 8.9:Block diagram of SIPO Register

8.3.5 Parallel in Serial out (PISO)

In the "Parallel IN Serial OUT" register, the data is entered in a parallel way, and the outcome comes serially. A four-bit "Parallel IN Serial OUT" register is designed below. The input of the flip flop is the output of the previous Flip Flop. The input and outputs are connected through the combinational circuit. Through this combinational circuit, the binary input B0, B1, B2, B3 are passed. The shift mode and the load mode are the two modes in which the "PISO" circuit works.(**Load mode)** The bits B0, B1, B2, and B3 are passed to the corresponding flip flops when the second, fourth, and sixth "AND" gates are active. These gates are active when the shift or load bar line set to 0. The binary inputs B0, B1, B2, and B3 will be loaded into the respective flip-flops when the edge of the clock is low. Thus, parallel loading occurs.(**Shift mode)** The second, fourth, and sixth gates are inactive when the load and shift line set to 0. So, we are not able to load data in a parallel way. At this time, the

first, third, and fifth gates will be activated, and the shifting of the data will be left to the right bit. In this way, the "Parallel IN Serial OUT" operation occurs.

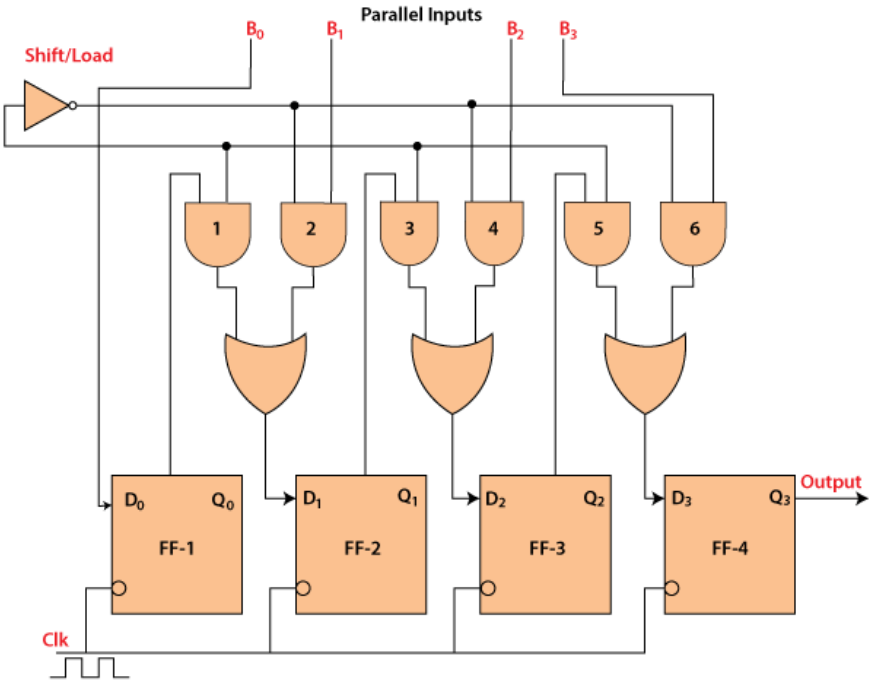


Figure 8.10: Block diagram of PISO Register

8.3.6 Parallel in Parallel Out(PIPO)

In "Parallel IN Parallel OUT", the inputs and the outputs come in a parallel way in the register. The inputs A0, A1, A2, and A3, are directly passed to the data inputs D0, D1, D2, and D3 of the respective flip flop. The bits of the binary input is loaded to the flip flops when the negative clock edge is applied. The clock pulse is required for loading all the bits. At the output side, the loaded bits appear.

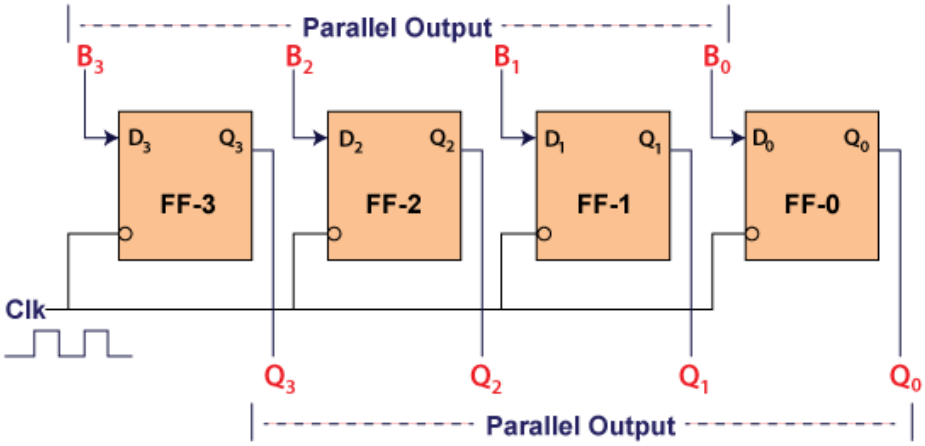


Figure 2.11:block diagram of PIPO Register

8.3.7 Applications of Shift Register

1. The shift registers are used for temporary data storage.
2. The shift registers are also used for data transfer and data manipulation.
3. The serial-in serial-out and parallel-in parallel-out shift registers are used to produce time delay to digital circuits.
4. The serial-in parallel-out shift register is used to convert serial data into parallel data thus they are used in communication lines where demultiplexing of a data line into several parallel line is required.
5. A Parallel in Serial out shift register us used to convert parallel data to serial data.

Part I-Multiple Choice Questions (MCQ)

1. Which of the following is used to store binary data
(a) counter (b)Register (c) Comparator (d) Adder

2. How many bits are stored in Flip flop in each register
(a) 8 (b) 4 (c) 2 (d) only 1

3. How many flip flop are required to store n bits in a register
(a) n (b) 2n (c) n^2 (d) 2n^2

4. How many flip flops are required to store 8 bits in a Register
(a) 4 (b) 8 (c) 2 (d) 1

5. Shift register which received data in serial form and out in serial form
(a)SISO (b) SIPO (c) PISO (d) PIPO

6. Shift register which received data in serial and out in parallel form
(a)SISO (b) SIPO (c) PISO (d) PIPO

7. Shift register which received data in parallel and out in serial form
(a)SISO (b) SIPO (c) PISO (d) PIPO

8. Shift register which received data in parallel and out in parallel form
(a)SISO (b) SIPO (c) PISO (d) PIPO

9. In which shift register all bits gain output at a time
(a)SISO (b) SIPO (c) PISO (d) PIPO

10. To enter data in a register is called
(a)Data storing (b) Data loading (c) Data transferring (d) Data shifting

ANSWER KEY

1.	B	2.	d	3.	a	4.	b	5.	a
6.	B	7.	c	8.	d	9.	d	10.	b

PART-II SAMPLE SHORT QUESTIONS

1. Define the term Register?
2. What is storage capacity of a register?
3. Define shift Register?
4. Write the name and working of shift left and shift right Register?
5. Define shift left and shift right register?

6. Enlist the types of Shift register?
7. Define SISO Register?
8. Define SIPO Register?
9. Define PIPO Register?
10. Define PISO Register?
11. Draw the diagram of SISO Register?
12. Draw the diagram of PISO Register?
13. Draw the diagram of PIPO Register?
14. Draw the diagram of SIPO Register?
15. State two applications of shift register?

Part-III Long Questions

1. Define Register and explain shift left and shift right Register?
2. Explain serial in and serial out register with diagram?
3. Explain serial in and parallel out register with diagram?
4. Explain parallel in and parallel out register with diagram?
5. Explain parallel in and serial out register with diagram?

CHAPTER NO. 09

COUNTERS

9.1 Introduction

Counter is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relationship to a clock signal. Counters are used in digital electronics for counting purpose, they can count specific event happening in the circuit. For example, in UP counter a counter increases count for every rising edge of clock. Not only counting, a counter can follow the certain sequence based on our design like any random sequence 0, 1, 3,2… .They can also be designed with the help of flip flops.

9.2 Chapter Objectives

6. Understand the basics of counters and their types
7. Understand the basic of Ripple counter and synchronous counter
8. Understand the working of digital clock.

9.3 Counter Types

Counters can be implemented by using register type circuits like flip-flops and these are classified into different types and a few of them are discussed below.

1. Asynchronous counter
2. Synchronous counter
3. Ripple counter
4. Presentable counter
5. Digital clock

9.3.1 Asynchronous counter

Asynchronous counters are those counters which do not operate on simultaneous clocking. In asynchronous counter, only the first flip-flop is externally clocked using clock pulse while the clock input for the successive flip-flops will be the output from a previous flip-flop.

This means that only a single clock pulse is not driving all the flip-flops in the

arrangement of the counter.

Asynchronous counters are also known as ripple counters and are formed by the successive combination of trailing edge-triggered flip-flops. It is called so because the data ripples between the output of one flip-flop to the input of the next.

9.3.1.1 Circuit Diagram of Asynchronous Counter

The figure given below shows the circuit diagram of a 3-bit asynchronous counter**.**

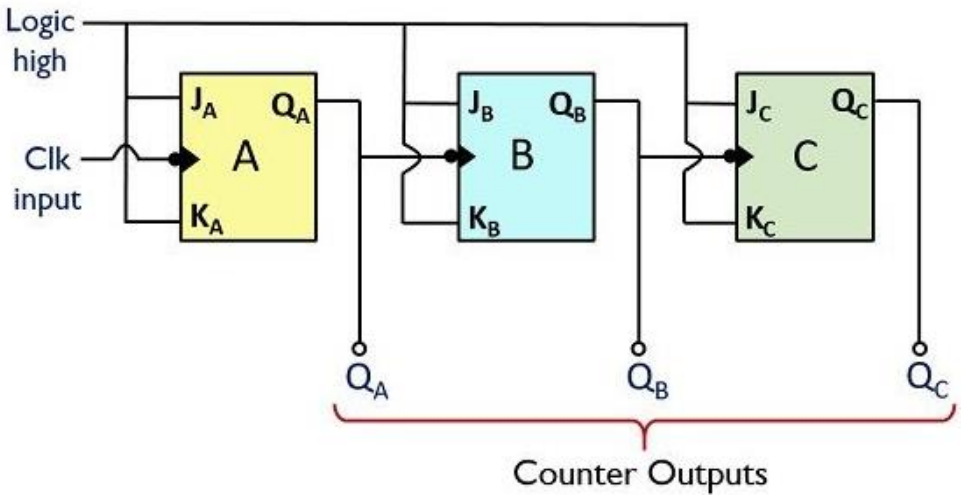


Figure 9.1:Circuit diagram of 3 bit asynchronous counter

Here as we can clearly see that 3 negative edge-triggered flip-flops are sequentially connected where the output of one flip-flop is provided as the input to the next. The input clock pulse is applied at the least significant or the first most flip-flop in the arrangement.

Also, logic high signal i.e., 1 is provided at the J and K input terminals of the flip-flops. Therefore, the toggling will be achieved at the negative transition of the applied clock input.

Let us now have a look at the timing diagram of the 3-bit asynchronous counter.

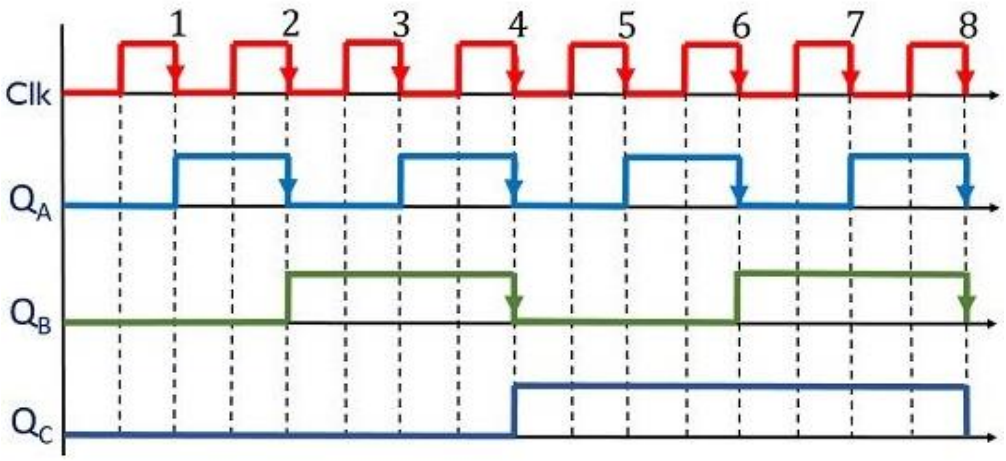


Figure 9.2: Timing diagram of 3 bit asynchronous counter

Initially when the clock input is applied at the LSB flip-flop i.e., A then the output QA will change from 0 to 1 at the falling edge of the clock pulse. As we can see that at the first count of a clock pulse at the falling edge, QA toggles from 0 to 1.

Further QA holds its state 1 and toggles from 1 to 0 only when another falling edge of the clock input is received. Again QA toggles from 0 to 1 at the next falling edge of the input clock pulse.

As we have already discussed that only the first flip-flop is triggered with an external clock signal. So, now the output of flip-flop A will act as the clock input for flip-flop B and the external clock signal will not be going to affect QB.

Therefore, the further toggling of the QB will depend on the falling edge of QA signal.

So, as we can see clearly in the timing diagram that QB undergoes toggling only at the falling edge of the QA signal. And the clock input signal is not affecting the output of flip-flop B.

Further for flip-flop C, the clock input will now be the output of flip-flop B i.e., QB. So, the output QC will be according to the transition of QB.

As we can see in the diagram that first time QC toggles from 0 to 1 only at the first falling edge of QB signal. And maintains the state till it reaches the next falling edge of QB.

So, in this way, we can say that we are not simultaneously providing a clock

input to all the flip-flops in asynchronous counters.

Now the question arises by this how the counter counts the number of states?

Basically, the number of states depends on the number of flip-flops employed in the circuit.

A 3 flip-flop arrangement counter can count the states up to 23 – 1 i.e., 8-1 = 7. Let's understand this by the help of the truth table given below:

Clock	Q_C	Q_B	Q_A	Decimal Equivalent
Initially	0	0	0	0
1st falling edge	0	0	1	1
2nd falling edge	0	1	0	2
3rd falling edge	0	1	1	3
4th falling edge	1	0	0	4
5th falling edge	1	0	1	5
6th falling edge	1	1	0	6
7th falling edge	1	1	1	7
8th falling edge	0	0	0	0

Figure 9.3: Truth table of asynchronous counter

As we can see that initially, the outputs of all the 3 flip-flop is 0. But as we move further then we see that at the first falling edge of the clock input, QA is 1 while QB and QC are 0, thereby providing decimal equivalent as 0. Again for the second falling edge of the clock input QB is 1 whereas QA and QC are 0, giving a decimal count 1.

Similarly, for the 3rd falling edge, QA and QB are 1 and QC is still 0. In the case of 4th falling edge, only QC is 1 while both QA and QB are 0 and so on.

In this way, we can draw the truth table by observing the timing diagram of the counters. And the truth table provides the count of the applied input clock pulse.

Thus, we can say an asynchronous counter counts the binary value according to the clock input applied at the least signal bit flip-flop of the arrangement.

A 3-bit counter is also known as mod 8 counters due to the presence of 8 states

9.3.1.2 Application of Asynchronous Counter

These are used in applications where low power consumption is required. And are also used in frequency divider circuits, ring and Johnson counters.

9.3.2 Synchronous Counter

Synchronous Counters are so called because the clock input of all the individual flip-flops within the counter are all clocked together at the same time by the same clock signal.

9.3.2.1 Construction & Working

In Synchronous Counter, the external clock signal is connected to the clock input of EVERY individual flip-flop within the counter so that all of the flip-flops are clocked together simultaneously (in parallel) at the same time giving a fixed time relationship. In other words, changes in the output occur in "synchronization" with the clock signal.

The result of this synchronization is that all the individual output bits changing state at exactly the same time in response to the common clock signal with no ripple effect and therefore, no propagation delay.

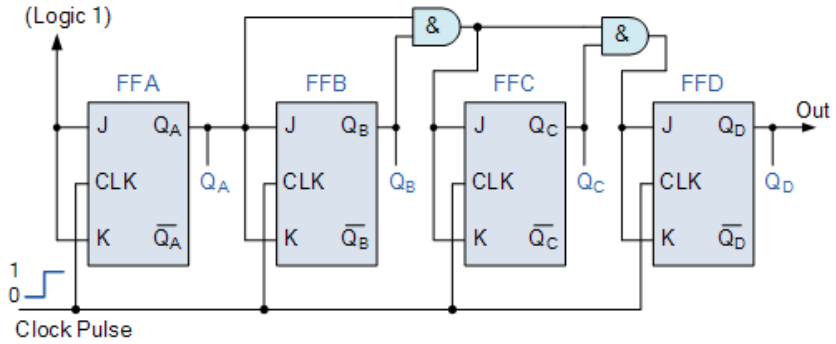


Figure 9.4: Binary 4-bit Synchronous Up Counter

It can be seen above, that the external clock pulses (pulses to be counted) are fed directly to each of the J-K flip-flops in the counter chain and that both the J and K inputs are all tied together in toggle mode, but only in the first flip-flop, flip-flop FFA (LSB) are they connected HIGH, logic "1" allowing the

flip-flop to toggle on every clock pulse. Then the synchronous counter follows a predetermined sequence of states in response to the common clock signal, advancing one state for each pulse.

The J and K inputs of flip-flop FFB are connected directly to the output QA of flip-flop FFA, but the J and K inputs of flip-flops FFC and FFD are driven from separate AND gates which are also supplied with signals from the input and output of the previous stage. These additional AND gates generate the required logic for the JK inputs of the next stage.

If we enable each JK flip-flop to toggle based on whether or not all preceding flip-flop outputs (Q) are "HIGH" we can obtain the same counting sequence as with the asynchronous circuit but without the ripple effect, since each flip-flop in this circuit will be clocked at exactly the same time.

Then as there is no inherent propagation delay in synchronous counters, because all the counter stages are triggered in parallel at the same time, the maximum operating frequency of this type of frequency counter is much higher than that for a similar asynchronous counter circuit.

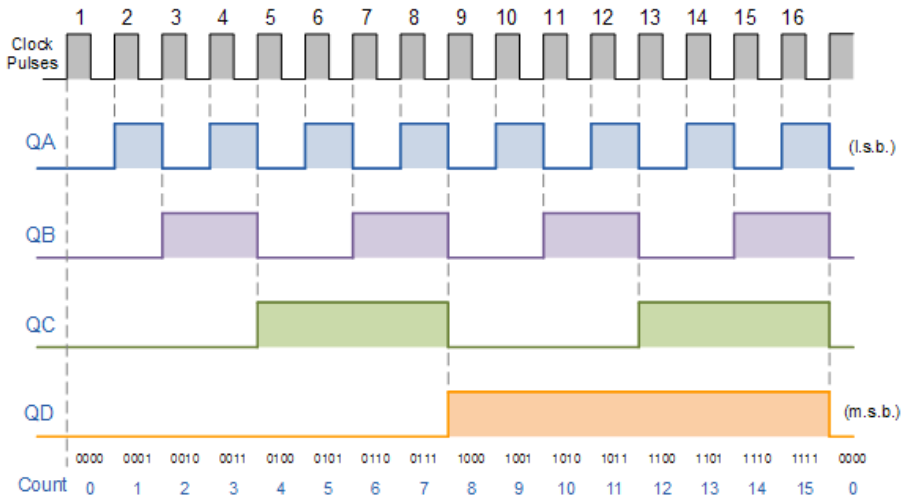


Figure 9.5: Timing diagram of 4-bit Synchronous Counter

Because this 4-bit synchronous counter counts sequentially on every clock pulse the resulting outputs count upwards from 0 (0000) to 15 (1111). Therefore, this type of counter is also known as a 4-bit Synchronous Up

Counter.

9.3.3 Ripple counter

Ripple counter is a special type of Asynchronous counter in which the clock pulse ripples through the circuit. The n-MOD ripple counter forms by combining n number of flip-flops. The n-MOD ripple counter can count 2n states, and then the counter resets to its initial value.

Based on their circuitry design, the counters are classified into the following types

1. Up Counter
2. Down Counter
3. Up-Down Counter

The up-counter counts the states in ascending order. The down counter counts the states in descending order. The up and down counter is a special type of bi-directional counter which counts the states either in the forward direction or reverse direction. It also refers to a reversible counter.

9.3.4 Presentable Counter

The counter which starts counting when we preset any number and this start counting from this preset number is called presentable counter.

9.3.4.1 Explanation

The presentable synchronous 4-bit binary counters with asynchronous reset are shown in below figure. These devices are high speed silicon-gate CMOS devices that are pin compatible with low power Schottky TTL (LSTTL). They feature an internal look-ahead carry and can be utilized for high-speed counting. Synchronous operation is provided by having all flip-flops clocked simultaneously on the positive-going edge of the clock (CP).

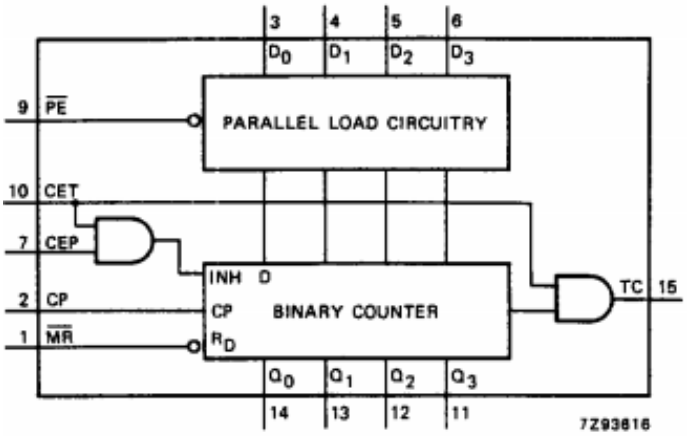


Figure 9.6:4 bit presentable counter

The outputs (Q0 to Q3) of the counters may be preset to a HIGH or LOW level. A LOW level at the parallel enable input (PE) disables the counting action and causes the data at the data inputs (D0 to D3) to be loaded into the counter on the positive-going edge of the clock (providing that the set-up and hold time requirements for PE are met). Preset takes place regardless of the levels at count enable inputs (CEP and CET).

A LOW level at the master reset input (MR) sets all four outputs of the flip-flops (Q0 to Q3) to LOW level regardless of the levels at CP, PE, CET and CEP inputs (thus providing an asynchronous clear function).

The look-ahead carry simplifies serial cascading of the counters. Both count enable inputs (CEP and CET) must be HIGH to count. The CET input is fed forward to enable the terminal count output (TC). The TC output thus enabled will produce a HIGH output pulse of duration approximately equal to a HIGH level output of Q0. This pulse can be used to enable the next cascaded stage.

9.3.4.2 Features

1. Synchronous counting and loading
2. Two count enable inputs for n-bit cascading
3. Positive-edge triggered clock
4. Asynchronous reset
5. Output capability: standard

9.3.4.3 Application

1. Television sets
2. Home-sound sets
3. Multimedia systems
4. All mains fed audio systems
5. Car audio (boosters)

9.3.5 Digital Clock

A digital clock is a type of clock that displays the time digitally (i.e. in numerals or other symbols), as opposed to an analogue clock**.** a clock that displays the time in numerical digits rather than by hands on a dial.

9.3.5.1 Types of digital Clock

The digital clocks are basically categorized into three different varieties which include the traditional quartz clock, digital atomic clock and the auto adjusted clocks. Apart from these, the digital LED clocks, the digital calendar clock and the jumbo digital clock are some of the other popular digital clocks used all over the world.

9.3.5.2 Construction and working of Digital clock

The digital clock consists of following main parts LCD, display, switches, microcontroller, power supply, reset and clock circuit.

The power supply is implemented using a transformer followed by a rectifier and finally a regulator. The clock circuit is implemented using the standard circuit prescribed by INTEL for 8051. The crystal of 12MHz is used so that the 1µsec delay can be implemented easily for the timer application of 8051.
The delay can be accordingly adjusted to give 1µsec; after which the seconds' counts can be incremented. Once the seconds counter reaches 60, it should be reset to 0 and the minutes counter should be incremented. Also once the minutes counter reaches 60, it should be reset to zero and the hours counter should be incremented. Finally if the hours counter reaches 24, it should also

be reset to zero. Thus the crystal is a very important component for timer or digital clock.The standard power-on-reset circuit for the microcontroller 8051 is implemented as shown in figure.

The interfacing of the 16 x 2 LCD is done. The various signals connected to the LCD display are power supply pins (i.e. Vcc and GND), the VEE pin (for intensity adjustment of LCD backlight), data pins (D0-D7). RS(To indicate a command or data to LCD), RW (To indicate read or write operation to LCD) and E (To enable pin to indicate latching of command or data).The switches are connected as shown in the Fig. 12.1.a. whenever the switch is pressed a zero is obtained on the microcontroller port pin else a '1' is obtained. This can be used to identify the switch pressed. The switch may be required to do various operations on the timer like set the time, set the alarm, etc.

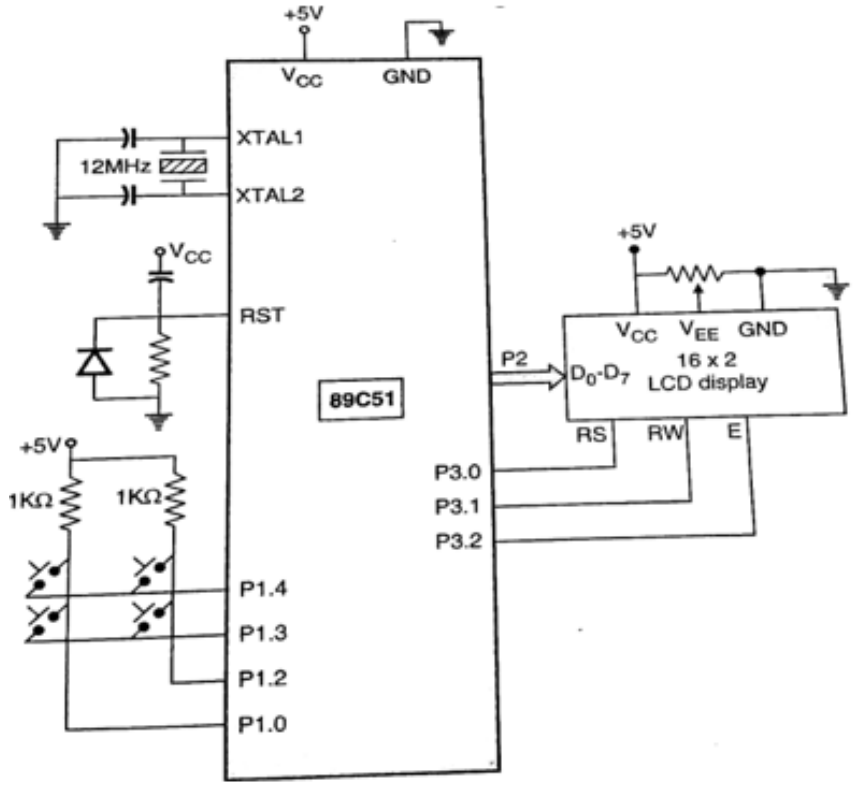


Figure 9.7: Construction diagram of digital clock

9.3.5.3 Application

There are many applications of digital clock. Because digital clocks can be very small and inexpensive devices that enhance the popularity of product designs, they are often incorporated into all kinds of devices such as cars, radios, televisions, microwave ovens, standard ovens, computers and cell phones

Part I-Multiple Choice Questions (MCQ)

1. Counters are made by connecting
(a) Flip flop (b) Register (c) Comparator (d) Adder

2. Heart of digital system is.
(a)Counter (b) Flip Flop (c) Adder (d) Both a &b

3. Ripple counter is
(a) simple (b)low speed (c)Frequency divider circuit(d)ALL

4. Modulus of counter is
(a)No of flip flop (b)No of it state (c) Its work in 1 sec (d) ALL

5. The no of flip flop required for 4 bit counter
(a)2 (b) 3 (c) 04 (d) 16

6. How many state of counter consists of n Flip flop
(a) n (b)n^2 (c)2^n (d) 2n^2

7. Synchronous counter is also called
(a)Serial counter (b) Parallel counter(c) Program counter (d)both a&c

8. Mostly counters available in these IC
(a)TTL (b)CMOS (c)Both a & b (d)DTL

9. Modulus of 3 bit binary is
(a)3 (b) 6 (c)8 (d)9

10. The counter has low speed
(a)Synchronous counter(b) ripple counter(c)serial counter (d)b &c

ANSWER KEY

1.	a	2.	a	3.	d	4.	b	5.	c
6.	c	7.	b	8.	C	9.	c	10.	b

PART-II SAMPLE SHORT QUESTIONS

1. Define the term counter?
2. What is basic function of counter?
3. Define synchronous counter?
4. What are the advantages of synchronous counter?
5. What is ripple counter?
6. What is asynchronous counter?

7. Draw the logic diagram of 4 bit synchronous counter?
8. Define presentable counter?
9. Draw the block diagram of digital clock?
10. Define UP counter?
11. What is down counter?
12. What are the disadvantages of asynchronous counter?
13. State the applications of counter?
14. Write down the types of counter?
15. What is the difference between ripple and asynchronous counter?

Part-III Long Questions

1. Define synchronous counter also explain 4 bit synchronous counter?
2. Explain asynchronous counter?
3. What is digital clock and what are its block diagram also explain it?
4. Explain 3 bit asynchronous counter?
5. Explain up and down counter with diagram?

CHAPTER NO. 10

MEMORIES

10.1 Introduction

Memory is the electronic holding place for the instructions and data a computer needs to reach quickly .or ***Memory is the ability to take in information, encode it, store it, and retrieve it at a later time***. It's where information is stored for immediate use. Memory is one of the basic functions of a computer, because without it, a computer would not be able to function properly. Memory is also used by a computer's operating system, hardware and software.

There are technically two types of computer memory: primary and secondary. The term memory is used as a synonym for primary memory or as an abbreviation for a specific type of primary memory called random access memory (RAM). This type of memory is located on microchips that are physically close to a computer's microprocessor.

If a computer's central processer (CPU) had to only use a secondary storage device, computers would become much slower. In general, the more memory (primary memory) a computing device has, the less frequently the computer must access instructions and data from slower (secondary) forms of storage.

10.1.1 Types of memory

There are two main types of memory are existed which are

1. Primary memory
2. Secondary memory

Primary Memory

It is that type of memory which is immediately access by the device like RAM and ROM. The more the storage capacity of primary memory the more the device speed to process the data

Secondary Memory

it is those type of memory which are attached to the device or computer to store the data which are on secondary options like Hard disk and memory card.The speed of the device doesn’t depends on the storage capacity.

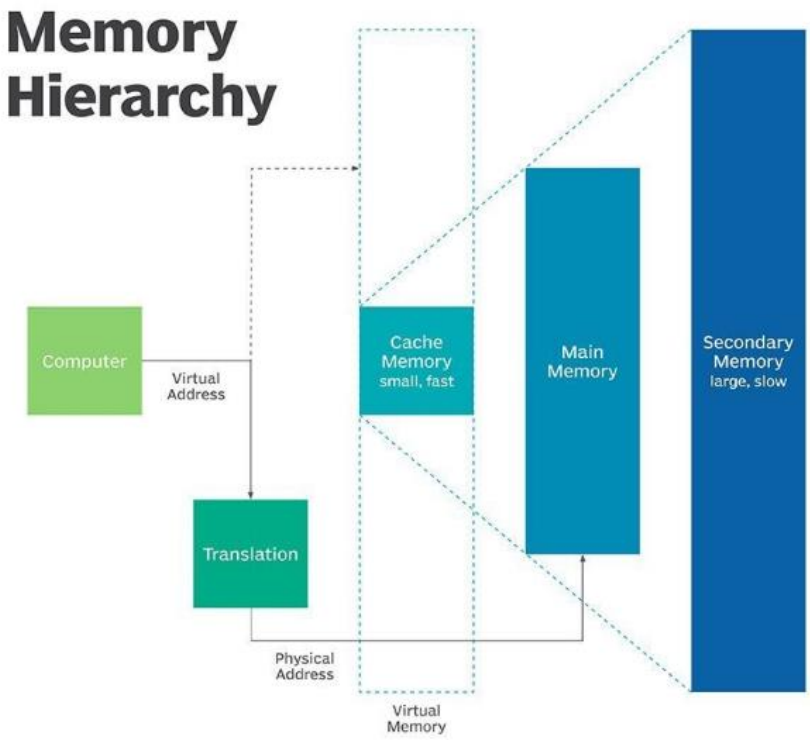


Figure 10.1 Memory hierarchy of computer

10.1.2 How does computer memory work?

When a program is open, it is loaded from secondary memory to primary memory. Because there are different types of memory and storage, an example of this could be a program being moved from a solid-state drive (SSD) to RAM. Because primary storage is accessed faster, the opened program will be able to communicate with the computer's processor at quicker speeds. The primary memory can be accessed immediately from temporary memory slots or other storage locations.

Memory is volatile, which means that data in memory is stored temporarily. Once a computing device is turned off, data stored in volatile memory will automatically be deleted. When a file is saved, it will be sent to secondary memory for storage.

There are multiple types of memory available to a computer. It will operate differently depending on the type of primary memory used, but in general, semiconductor-based memory is most associated with memory. Semiconductor memory will be made of integrated circuits with silicon-based metal-oxide-semiconductor (MOS) transistors.

10.1.3 Volatile Memory

It is the memory hardware that fetches/stores data at a high-speed. It is also referred as temporary memory. The data within the volatile memory is stored till the system is capable of, but once the system is turned off the data within the volatile memory is deleted automatically. RAM (Random Access Memory) and Cache Memory are some common examples of volatile memory. Here, data fetch/store is fast and economical.

10.1.4 Non-Volatile Memory

It is the type of memory in which data or information is not lost within the memory even power is shut-down. ROM (Read Only Memory) is the most common example of non-volatile memory. It's not economical and slow in fetch/store as compared to volatile memory however stores higher volume of data. All such information that needs to be stored for an extended amount of time is stored in non-volatile memory. Non-volatile memory has a huge impact on a system's storage capacity.

Below are the differences between volatile and non-volatile memory

Volatile Memory	Non-Memory
1. Volatile memory is the type of memory in which data is lost as it is powered-off.	1. Non-volatile memory is the type of memory in which data remains stored even if it is powered-off.
2. Contents of Volatile memory are stored temporarily.	2. Contents of Non-volatile memory are stored permanently.
3. It is faster than non-volatile memory	3. It is slower than volatile memory.
4. RAM (Random Access Memory) is an example of volatile memory.	4. ROM (Read Only Memory) is an example of non-volatile memory.
5. In volatile memory, data can be easily transferred in comparison to non-volatile memory.	5. In non-volatile memory, data cannot be easily transferred in comparison to volatile memory.
6. In Volatile memory, process can read and write.	6. In Non-volatile memory, process can only read.
7. Volatile memory generally	7. Non-volatile memory generally

has less storage capacity.	has more storage capacity than volatile memory.
8. In volatile memory, the program's data are stored which are currently in process by the CPU	8. In non-volatile memory, any kind of data which has to be saved permanently are stored.
9. Volatile memory is more costly per unit size.	9. Non-volatile memory is less costly per unit size.
10. Volatile memory has a huge impact on the system's performance.	10. Non-volatile memory has a huge impact on a system's storage capacity.
11. In volatile memory, processor has direct access to data.	11. In non-volatile memory, processor has no direct access to data.
12. Volatile memory chips are generally kept on the memory slot.	12. Non-volatile memory chips are embedded on the motherboard.

10.2 Memory Addressing

A memory address is a unique identifier used by a device or CPU for data tracking. ***Memory addresses are fixed-length sequences of digits conventionally displayed and manipulated as unsigned integers***. Memory usually hold instruction and data. This binary address is defined by an ordered and finite sequence allowing the CPU to track the location of each memory byte.

Modern computers are addressed by bytes which are assigned to memory addresses – binary numbers assigned to a random access memory (RAM) cell that holds up to one byte. Data greater than one byte is consecutively segmented into multiple bytes with a series of corresponding addresses.

Hardware devices and CPUs track stored data by accessing memory addresses via data buses. Before CPU processing, data and programs must be stored in unique memory address locations.

10.2.1 Memory Addressing Mode

There are many ways to locate data and instructions in primary memory and these methods are called "memory address modes".

Memory address modes determine the method used within the program to access data either from the Cache or the RAM. In this challenge we will focus on four different memory address modes:

1. Immediate Addressing
2. Direct Addressing
3. Indirect Addressing
4. Indexed Addressing

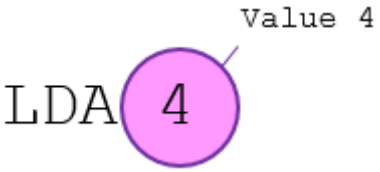
10.2.1.1 Immediate Addressing

Immediate addressing means that the data to be used is hard-coded into the instruction itself. For instance:

ADD 10 will increment the value stored in the accumulator by 10.

MOV R #20 initializes register R to a constant value 20.

Nothing needs to be fetched from memory. This instruction means that the value 7 will be added to the value currently stored in the accumulator. This is the fastest method of addressing as it does not involve fetching anything from the main memory at all.



Some of the more formats for this addressing modes are given below

ADD #7
ADD &7
ADD ~7
ADD 7[2]

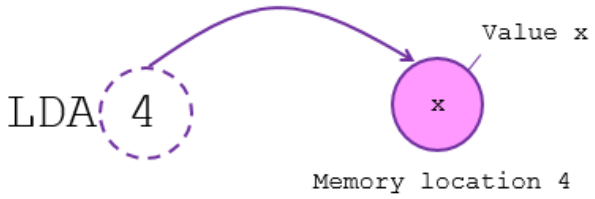
10.2.1.2 Direct Addressing

Direct Addressing is a very simple way of addressing memory – it means that the operand of an instruction refers directly to a location in memory.

For example:

ADD X will increment the value stored in the accumulator by the value stored at memory location X. AC ← AC + [X]

This instruction would not add the value 7 to the accumulator. Instead it would add the value currently stored at memory location 7.This memory address mode is fairly fast (not as fast as immediate addressing though).



10.2.1.3 Indirect Addressing

Indirect addressing means that the address of the data is held in an intermediate location so that the address is first 'looked up' and then used to locate the data itself. Fetching the value is a two-step process: First the indirect address is used to locate an entry into a lookup table (called Vector Table) where the actual address of where the value can be fetched from is stored. So, indirect addressing mode is a two steps process: the operand is an address towards a memory location that contains an address where the value can be fetched from.

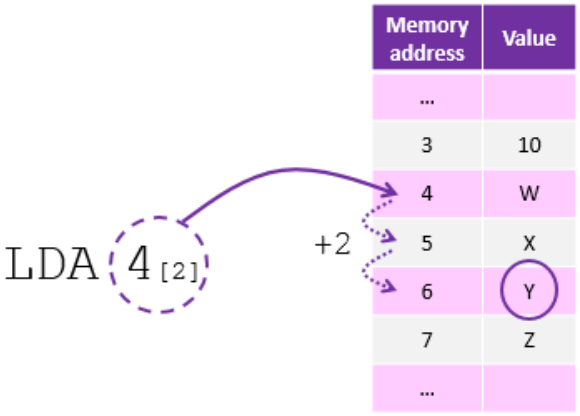
ADD X will increment the value stored in the accumulator by the value stored at memory location specified by X. AC ← AC + [[X]]



10.2.1.4 Indexed Addressing

Indexed addressing means that the final address for the data is determined by adding an offset to a base address. Effective address of the operand is obtained by adding the content of index register with the address part of the instruction.

This memory address mode is ideal to store and access values stored in arrays. Arrays are often stored as a complete block in memory (A block of consecutive memory locations). The array has a base address which is the location of the first element, then an index is used that adds an offset to the base address in order to fetch the specified element within the array.



10.3 Non Volatile Memory

It is the type of memory in which data or information is not lost within the memory even power is shut-down. Some of the examples are ROM,PROM and EPROM

10.3.1 ROM

Read-only memory (ROM) is a type of storage medium that permanently stores data on personal computers (PCs) and other electronic devices.

It contains the programming needed to start a PC, which is essential for boot-up; it performs major input/output tasks and holds programs or software instructions. This type of memory is often referred to as "firmware"—how it is altered has been a source of design consideration throughout the evolution of the modern computer.

Let us now discuss the various types of ROMs and their characteristics.

10.3.1.1 MROM (Masked ROM)

The very first ROMs were hard-wired devices that contained a pre-programmed set of data or instructions. These kind of ROMs are known as masked ROMs, which are inexpensive.

10.3.1.2 PROM (Programmable Read Only Memory)

PROM is read-only memory that can be modified only once by a user. The user buys a blank PROM and enters the desired contents using a PROM program. Inside the PROM chip, there are small fuses which are burnt open during programming. It can be programmed only once and is not erasable.

10.3.1.3 EPROM(Erasable Programmable Read Only Memory)

EPROM can be erased by exposing it to ultra-violet light for a duration of up to 40 minutes. Usually, an EPROM eraser achieves this function. During

programming, an electrical charge is trapped in an insulated gate region. The charge is retained for more than 10 years because the charge has no leakage path. For erasing this charge, ultra-violet light is passed through a quartz crystal window (lid). This exposure to ultra-violet light dissipates the charge. During normal use, the quartz lid is sealed with a sticker.

10.3.1.4 EEPROM (Electrically Erasable and Programmable Read Only Memory)

EEPROM is programmed and erased electrically. It can be erased and reprogrammed about ten thousand times. Both erasing and programming take about 4 to 10 ms (millisecond). In EEPROM, any location can be selectively erased and programmed. EEPROMs can be erased one byte at a time, rather than erasing the entire chip. Hence, the process of reprogramming is flexible but slow.

10.3.1.5 Advantages of ROM

The advantages of ROM are as follows −

1. Non-volatile in nature
2. Cannot be accidentally changed
3. Cheaper than RAMs
4. Easy to test
5. More reliable than RAMs
6. Static and do not require refreshing
7. Contents are always known and can be verified

10.4 RAM

RAM stands for random-access memory, but what does that mean? Your computer RAM is essentially short term memory where data is stored as the processor needs it. ... This is so that your processor can get to it quickly.

RAM is essentially a device's short-term memory. It temporarily stores (remembers) everything currently running on a device, like all OS-specific services and any web browser, image editor, or game you're playing. RAM prevents the CPU from digging through the device's slower storage — like a hard drive or even a solid-state drive (SSD) — every time you request a new browser tab or load a new enemy to shoot. As fast as storage is compared to drives of years gone by, they're still far slower than RAM.

Data that resides in RAM is readable from any capable component at almost

the same speed. Because it has a hard-wired connection to the device, there's no real latency in cabling or connection.

RAM doesn't remember everything forever, however. It's a "volatile" technology, meaning that once it loses power, it forgets everything. That makes it perfect for handling the multitude of high-speed tasks that your device throws at it each day. But it's also why storage systems like hard drives and SSDs are required. Unlike RAM, they hold information when the device powers off.

10.4.1 Types of RAM

There are two main types of RAM: dynamic random access memory (DRAM), or Dynamic RAM, and static random access memory (SRAM). The RAM in most personal computers (PC's) is Dynamic RAM. All dynamic RAM chips on DIMMs, SIMMs or RIMMs have to refresh every few milliseconds by rewriting the data to the module.

10.4.1.1 Static RAM

Static RAM (SRAM) is volatile memory and is often used in cache memory and registers because it is a lot faster and does not require refreshing like Dynamic RAM. As long as power is supplied, SRAM retains its content and there's no need to refresh it on a regular basis.SRAM uses a matrix of 6-transistors that do not depend on power to avoid leakages since they act as switches serving as 1s and 0s with no capacitor holding the charge. However, since more transistors are needed than a DRAM, the matrix requires extra space, making the manufacturing costs higher.

An SRAM is able to operate at higher speed and uses less power than a DRAM, but is also more expensive and is thus used only as the processor's cache memory. SRAM is used in memory cache on the central processing unit (CPU); it is called L1, L2 and L3 cache. Original SRAM was stored on the motherboard; later SRAM was inside of the CPU housing or stored on both the motherboard and inside of the CPU.

Advantages of SRAM

1. SRAM performance is better than DRAM in terms of speed. It means it is faster in operation.
2. SRAM is used to create a speed-sensitive cache.
3. It has medium power consumption

10.4.1.2 Dynamic RAM (DRAM)

Dynamic RAM is memory that needs refreshing. The refreshing is done by the memory controller which is part of the chipset on the motherboard. Each cell is held in an electrical capacitor with transistors serving as gates that determine whether each value could be read or written. To compensate for the capacitor's leaks, the DRAM must be refreshed with an electronic charge that rewrites the data every few milliseconds. DRAMs are usually smaller and less expensive than SRAMs and are used in practically every PC.

Advantages of DRAM

1. Cheaper compared to SRAM.
2. It has a higher storage capacity. Hence it is used to create a larger RAM space system.
3. Offers simple structure.
4. It doesn't require refreshing the memory contents.
5. You don't need to refresh the memory contents and its access time is faster
6. Logic or circuitry is needed, so the memory module itself is simpler.

10.4.1.3 Difference between DRAM & SRAM

SRAM	DRAM
SRAM has lower access time, which is faster compared to DRAM.	DRAM has a higher access time. It is slower than SRAM.
SRAM is costlier than DRAM.	DRAM cost is lesser compared to SRAM.

SRAM needs a constant power supply, but it consumes less power.	DRAM requires more power consumption as the information is stored in the capacitor.
SRAM offers low packaging density.	DRAM offers a high packaging density.
Uses transistors and latches.	Uses capacitors and very few transistors.
L2 and L3 CPU cache units are some general application of an SRAM.	The DRAM is mostly found as the main memory in computers.
The storage capacity of SRAM is 1MB to 16MB.	The storage capacity of DRAM is 1 GB to 16GB.
SRAM is in the form of on-chip memory.	DRAM has the characteristics of off-chip memory.
The SRAM is widely used on the processor or lodged between the main memory and processor of your computer.	The DRAM is placed on the motherboard.
SRAM is of a smaller size.	DRAM is available in larger storage capacity.
This type of RAM works on the principle of changing the direction of current through switches.	This type of RAM works on holding the charges.

Part I-Multiple Choice Questions (MCQ)

1. A 24 bit word consist of
(a) 2 bytes (b) 3 bytes (c) 4 bytes (d) 4 nibbles

2. A ROM is a
(a)volatile (b)Read/write (c)Nonvolatile (d)byte organized

3. The storage element of DRAM is
(a) Resistor (b) Transistor (c) Capacitor (d) Latch

4. The storage element of SRAM is
(a) Resistor (b) Transistor (c) Capacitor (d) Latch

5. In a computer the bios program are stored in the
(a)ROM (b) RAM (c) Flash Memory (d) DRAM

6. Arrangement of memory cell is called
(a)Read (b) Write (c) Array (d) Word

7. PROM is programmed by user only
(a) Once (b) Twice (c) thrice (d) any of these

8. This memory is Read/Write memory
(a)RAM (b) ROM (c) PROM (d) EPROM

9. The instruction of read and write is given through
(a)Data bus (b) Control bus (c) Address Bus (d) none of these

10. Floppy disk is type of memory
(a)Optical (b) Mechanical (c) Magnetic (d) None of these

ANSWER KEY

1.	B	2.	c	3.	C	4.	D	5.	a
6.	C	7.	a	8.	A	9.	D	10.	c

PART-II SAMPLE SHORT QUESTIONS

1. Define Memory?
2. Define memory element?
3. Define memory array?
4. Enlist the types of memory?

5. What is static RAM?
6. What is Dynamic RAM?
7. What are the addressing modes?
8. What is immediate addressing mode?
9. What is direct addressing mode?
10. What is indirect addressing mode?
11. Define volatile memory?
12. Define non-volatile memory?
13. Define RAM?
14. Define ROM?
15. Define PROM?
16. Define EPROM?
17. What are the difference between SRAM and DRAM?
18. Define Mask ROM?
19. What is indexed addressing mode?
20. What is EEPROM?

Part-III Long Questions

1. Define memory also differentiate between volatile and non-volatile memory?
2. Define RAM also explain its types?
3. Define ROM also explain its?
4. What are the addressing modes also explain any of two addressing modes with examples?
5. Write down at least ten differences between SRAM and DRAM ?

CHAPTER NO. 11

APPLICATION OF DIGITAL ELECTRONICS

11.1 Introduction

Digital electronics operate on digital signals and a digital circuit is an integral component of digital electronics. It is used in various electronic devices for different purposes. The main purpose of this circuit is the transmission of power to the different components of the circuit. It can perform different types of Boolean operations. The Digital circuit is one such circuit where the signal exists in one of the two states which are 0 and 1. We use this kind of circuit to make logic gates and transistors. These circuits consume relatively less power as compared to analog circuits. Some of the important components of a digital circuit include registers, transistors, battery and many more.

11.2 Digital Circuit

A digital circuit is a type of circuit that operates on different logic gates. The logic gates are used to differentiate between various power signals. It can be used to transmit the power signal to the different parts of the electronic device through various gates. This will help to create an output signal which is directly proportional to the energy level of the input signals. Most of these circuits consist of analog components. They have a more complex design as compared to that of analog circuits.

11.3 Digital Circuit Design

Digital circuits can be designed using the various components which are generally used in the circuit. Here, we have shown a simple circuit consisting of a few important components. The following circuit has a diode, a resistor, an inductor, a capacitor, and a power supply.

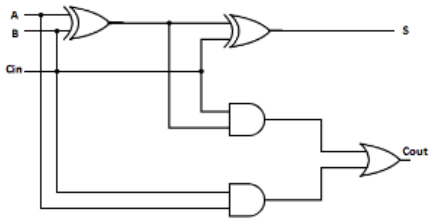


Figure 11.1: Digital Circuit design

The following circuit is that of a full-adder which can be used to add the bits of the circuit. The circuit performs a Boolean operation on the input bits to produce the result. The circuit consists of two XOR gates, two AND gates, and an OR gate.

11.4 Applications of Digital Electronics

There are many applications of digital electronics in which some of them are discussed here.

1. Digital circuits are a part of all the important electronic devices.
2. It can be used for designing the display of your watch or a countdown timer.
3. We use digital circuits in complex processes like Rocket Science and Quantum Computing.
4. Digital circuits are also used in traffic lights and automatic glass doors in offices and restaurants.
5. It used in Moving display
6. It used in Frequency counters
7. It used in Digital clock

11.4.1 Moving Message Display

A moving message display is ideal to get your message (advertisements, greetings, etc) across in an eye-catching way.

You can make an LCD show a brief moving message by interfacing it to a microcontroller. Here's an AVR-based moving message display that uses a 16×2 LCD display incorporating HD44780. The 16×2 LCD can display 16 characters per line and there are two such lines.

11.4.1.1 Moving message display circuit

Fig. 11.2 shows the circuit for AVR ATmega16-based moving message display on an LCD. It consists of an ATmega16 microcontroller, a 16×2 LCD, an SPI6 connector and a power supply section.

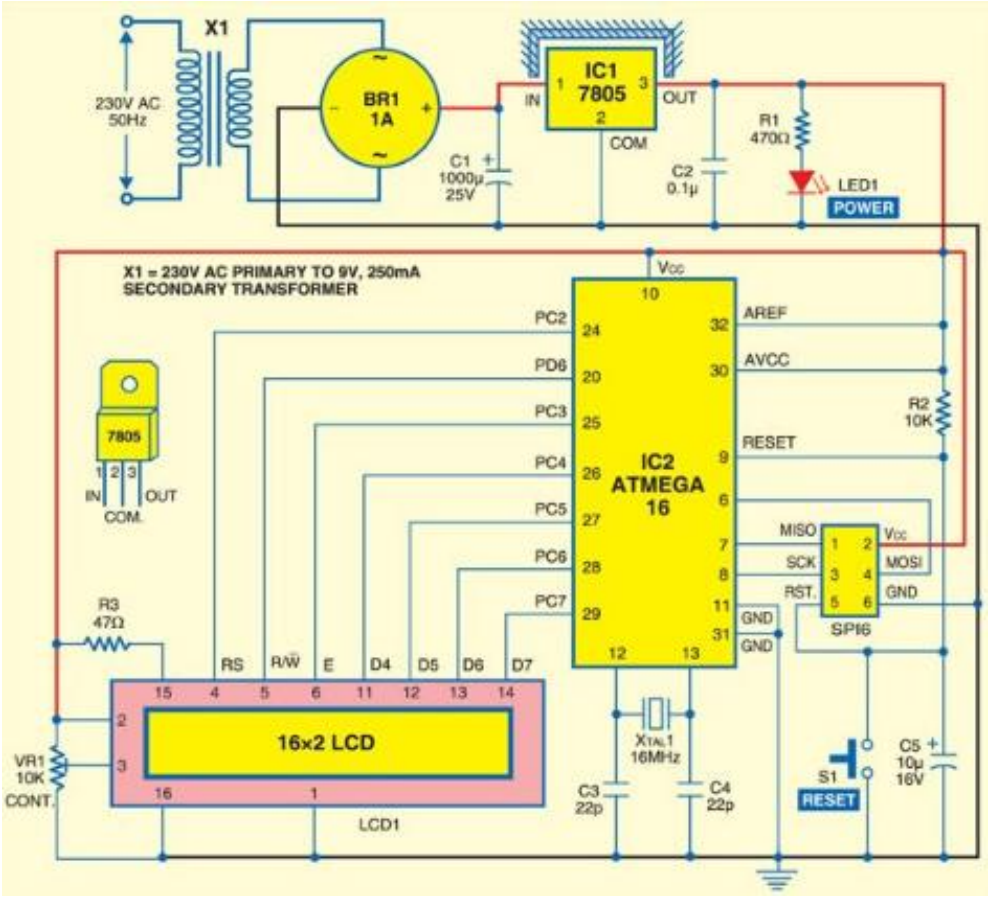


Figure 11.2: Circuit diagram of moving display project

To derive the power supply for the circuit, 230V AC mains is stepped down by a 9V, 250mA secondary transformer, rectified by bridge rectifier module BR1A and filtered by capacitor C1. The voltage is regulated by a 7805 regulator. LED1 glows to indicate the presence of power in the circuit. The regulated 5V DC powers the entire circuit including SPI6 connector.

Port-C pins PC4 through PC7 of the microcontroller (IC2) are connected to data lines D4 through D7 of the LCD. The LCD control lines—read/write (R/W), register-select (RS) and enable (E)—are connected to PD6, PC2 and PC3 of IC2, respectively.

11.4.1.2 Why AVR microcontroller?

AVR is faster and more powerful than 8051 microcontroller, yet reasonably cheaper and in-circuit programmable. Most AVR development software are free as these are Open Source. Moreover, discussions and tutorials on the

AVR family of processors are available on the Internet.
ATmega16 is a high-performance, low-power 8-bit AVR microcontroller. It has 16 kB of in-system self-programmable flash, 1 kB of internal SRAM, 512 bytes of EEPROM, 32×8 general-purpose working registers and JTAG Interface (which supports programming of flash, EEPROM, fuse and lock bits).

11.4.1.3 On-chip peripheral features

1. Two 8-bit timers/counters with separate pre-scaler and compare modes
2. One 16-bit timer/counter with separate pre-scaler, comparator and capture modes.
3. Four pulse-width-modulation channels
4. channel, 10-bit analogue-to-digital converter
5. Byte-oriented two-wire serial interface
6. Programmable serial USART
7. Master/slave serial peripheral interface
8. Programmable watchdog timer with separate on-chip oscillator

11.4.1.4 LCD display module

The project uses a Hitachi HD44780-controlled LCD module. The HD44780 controller requires three control lines and four or eight input/output (I/O) lines for the data bus. The user may choose to operate the LCD with a 4-bit or 8-bit data bus. If a 4-bit data bus is used, the LCD will require a total of seven data lines—three lines for sending control signals to the LCD and four lines for the data bus. If an 8-bit data bus is used, the LCD will require a total of eleven data lines—three control lines and eight lines for the data bus.

11.4.2 Frequency Counter

A frequency counter is a test instrument that measures frequency accurately. You can find its application in many radio frequencies (RF), to measure the frequency of repetitive signals. These days, digital frequency counters have made their ways to various circuitries.
The basic working principle of a frequency counter is that it counts the number of time signal passes a trigger point-mostly a voltage trigger point in a particular period of time.
The trigger points can be set to the desired value, if not; it is usually a zero-

crossing point. For example, if you want to count the number of signals, and you have set a time point at second, then the counter will count the number of times the signal crosses the trigger point. If it crosses 50 times, you can say that the frequency of the signal is 50 cycles per second or 50 Hz.

11.4.2.1 Applications of Frequency Counter

The frequency counter finds application in various instruments that are dependent on the frequency. The frequency, however, has to be a steady one, to get accurate results. These are used to measure frequency in the transmitter carrier, the oscillator in the circuit, analog, and digital signals, etc. In short, you can use the frequency counter whenever there is a requirement for you to measure the frequency of a repetitive signal.

11.4.2.2 Frequency Counter Using Arduino.

This frequency counter using Arduino is based on the UNO version and can count up to 40 KHz. A 16×2 LCD display is used for displaying the frequency count. The circuit has minimum external components and directly counts the frequency. Any way the amplitude of the input frequency must not be greater than 5V. If you want to measure signals over than 5V, additional limiting circuits have to be added and i will show it some other time. Now just do it with 5V signals.

The frequency to be counted is connected to digital pin 12 of the Arduino. Pulse In () function is used here for counting the frequency connected to pin 12. Pulse In () function counts the number of pulses (HIGH or LOW) coming to a particular pin of the Arduino. The general syntax of this function is pulse IN (pin, value, time) where the pin is the name of the pin, the value is either HIGH or LOW and time is the time for which the function to wait for a pulse. The function returns zero if there is no valid pulse within the specified time. The pulse IN () function can count pulses with a time period ranging from 10 µS to 3 minutes. A circuit diagram of the frequency counter using Arduino is given below.

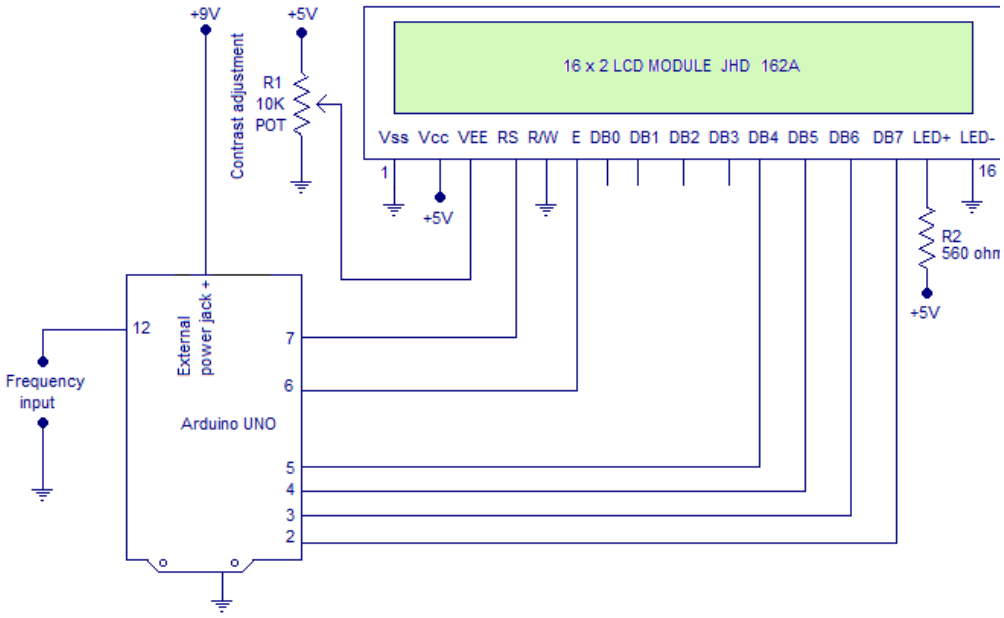


Figure 11.2: Circuit diagram of Frequency counter

Potentiometer R1 is used to adjust the contrast of the LCD screen. Resistor R2 limits the current through the back light LED. In the program, the high time and low time of the input signal are measured using separate pulse IN() functions. Then the high and low times are added together to get the total time period of the signal. Frequency is just 1/time period in seconds. The pulse IN () function returns the time period in microseconds. The total time period in microseconds first divided by 1000. Then 1000 is divided by the result to get the frequency in hertz.The program is shown below.

```
#include <LiquidCrystal.h>
int input=12;
int high_time;
int low_time;
float time_period;
float frequency;
LiquidCrystal lcd(7, 6, 5, 4, 3, 2);
void setup()
{
pinMode(input,INPUT);
lcd.begin(16, 2);
```

```
}
void loop()
{
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Frequency Meter");
high_time=pulseIn(input,HIGH);
low_time=pulseIn(input,LOW);
time_period=high_time+low_time;
time_period=time_period/1000;
frequency=1000/time_period;
lcd.setCursor(0,1);
lcd.print(frequency);
lcd.print(" Hz");
delay(500);
}
```

11.4.2.3 Conclusion

Frequency counters are one of the essential test instruments with a wide range of applications. You can easily create one using Arduino with the program and circuit, we have discussed in this project-based article. You can always experiment with the circuit and build your own frequency counter Hope this article clears your basics about frequency and frequency counters.

11.4.3 Digital Clock

A digital clock is a type of clock that displays the time digitally (i.e. in numerals or other symbols), as opposed to an analogue clock. Digital clocks are often associated with electronic drives, but the "digital" description refers only to the display, not to the drive mechanism.

Alarm Clocks are very useful devices in today's busy life. They are designed to make a signal / alarm at a specific time. The use of digital alarm clocks has increased over years with development in electronics.

The advantage of digital alarm clocks over analogue alarm clocks is that they require less power, the time can be set or reset easily and displays the time in digits. Design of a simple digital alarm clock is explained here.

Using this digital alarm clock, time can be displayed in 24 HR format using an LED display and alarm can be set to a specific time. Alternatively, the digital alarm clock circuit can also be used to turn ON/OFF and electrical appliance

after a specific time.

11.4.3.1 Digital Alarm Circuit Diagram

The below is the circuit diagram of project.

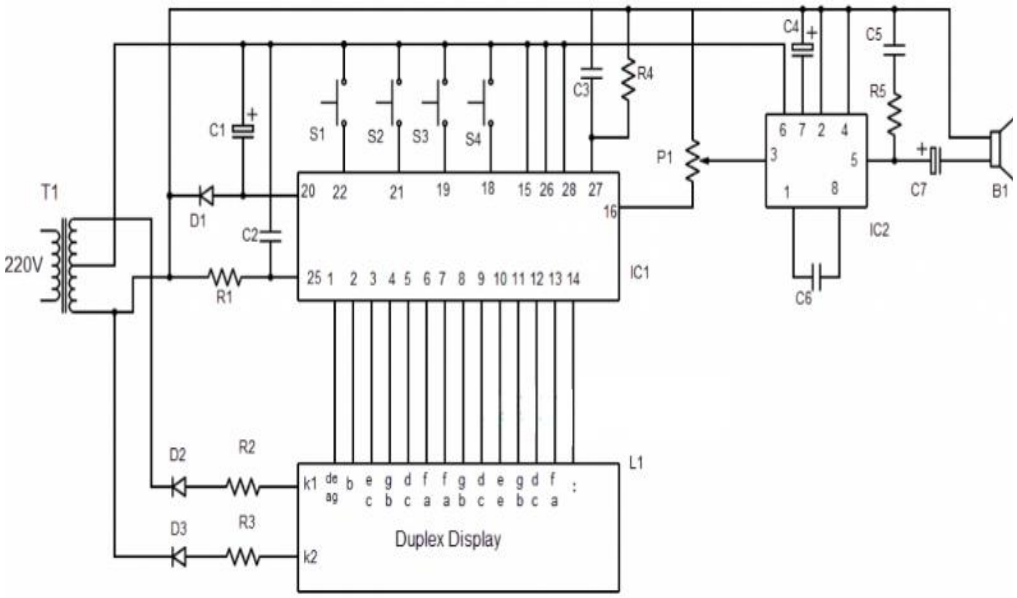


Figure 11.3: Circuit diagram of Digital clock

11.4.3.2 Digital Clock Components

1. IC1 – LM 8560
2. IC2 – LM 386
3. Duplex LED Display
4. T1 – 0-12V / 500mA Transformer
5. D1, D2, D3 – IN4001
6. R1 – 100K
7. R2, R3 – 120
8. R4 – 91K
9. R5 – 10
10. P1 – 10K Potentiometer
11. C1 – 470µF
12. C2, C3 – 10nF
13. C4 – 10µF
14. C5, C6 – 100nF
15. C7 – 100µF
16. B1 – Buzzer

17. S1, S2, S3, S4 – Switches

11.4.3.3 Digital Alarm Circuit Design

LM8560

The heart of the digital alarm clock is the IC LM8560. It is an alarm equipped digital clock IC. The drivers required to drive a duplex LED display to show the time are in-built to the IC. The functions of the IC LM8560 are display of current time, alarm support, snooze function and supports 12 HR and 24 HR time format.

LM386

It is a low voltage audio power amplifier. It is used in the circuit to drive the alarm buzzer. The alarm output of the LM8560 is given to power amplifier through a potentiometer.

Duplex LED Display

In the digital alarm clock, to display the information like current time, alarm notification etc. we use a duplex LED display. There is no need for any extra drivers to drive the display unit as all the required drivers are built in to the digital clock IC.

Push Button Switches (S1 – S4)

These four switches are used to set time, set alarm and turn off the alarm. When setting time and alarm in the digital alarm clock, both Hours and Minutes can be set separately using these switches.

Diodes (D1, D2, and D3)

Diode D1 in combination with the capacitor C1 will produce a DC voltage that is supplied to the IC1 (LM8560). Diodes D2 and D3 act as switch signal generator to the cathode of the duplex display. They work in relation to the input of the LM8560.

Buzzer (B1)

The main function of the digital alarm clock is to signal an alarm at specified time. A buzzer is used for this purpose which is driven by an audio power amplifier.

Transformer (T1)

A transformer is used to step down the 220-240 V AC supply to 12 V AC supply.

11.4.3.4 Digital Alarm Circuit Working

A digital alarm clock is a very useful device as it can display the current time as well as function as an alarming device. The circuit shown above can

act as a simple digital alarm clock without requirement of any programming.

The components required and the complexity of the circuit are less. The working of the above shown digital alarm clock is as follows.

As explained in the component description, the heart of the digital alarm clock circuit is the digital clock IC LM8560. The input from the mains is given to the transformer which converts the input voltage to 12V AC.

A diode (D1) and a capacitor (C1) are used to convert this AC voltage to DC and supplied to the Vdd (Pin 20) of the IC1. IC1 (LM8560) works at a frequency of 50/60 Hz. This signal can be supplied from the mains supply using the resistor R1 to the Pin 25 of the IC1.

In relation to the input of the IC1, corresponding inputs to the cathode of the display are given through the diodes (D2 and D3). After switching on the power, we need to set the time. This is the current time to be displayed. In order to set the current time, we use the switches S1 and S2.

Switch S1 is used to set Hours and S2 is used to set Minutes. LM8560 automatically triggers the display connected to it and the time will be displayed on the duplex LED display.

The duplex display is a 4-bit seven segment LED display, where two bits are for displaying the Hours and two bits are for displaying Minutes and the Hour and Minute bits are separated by a colon.

Once the time is set, we can set the alarm using the switches S1, S2 and S3. In order to differentiate between the setting of current time and alarm time in the digital alarm clock, we use the switch S3 which internally triggers the alarm display of the LM8560.

In order to set the alarm, we need to hold the switch S3 and set the Hour and Minutes of the alarm time by using switches S1 and S2 respectively. Once the alarm is set successfully, the digital clock IC waits for the alarm time and once the alarm time is reached, an alarm output at the Pin 3 is generated.

This can be connected to a small buzzer directly for alarm output. But in order to get special alarm tones from the digital alarm clock, we can use special alarm signals. But to drive these special buzzers, we need an audio power amplifier. Hence, the IC LM386, which is a power audio amplifier, is used in the digital alarm clock circuit.

The alarm output of the IC1 (LM8560) is given to the input (Pin 3) of the

IC2 (LM386) through a potentiometer P1. The potentiometer is used to control the volume and pressure of the alarm signal from the buzzer.

Once, the digital alarm clock starts the alarm signal, we need to stop this alarm. For this, we use the switch S4. The switch S4 can also be used to cancel the alarm any time before triggering.

This digital alarm clock can also be provided with snooze and repeat alarm facilities. Additional Pins of the IC1 (LM8560) have to be used in the digital alarm clock circuit in order to use these functionalities.

Part I-Multiple Choice Questions (MCQ)

1. A digital circuit that can store only one bit is a
(a) Register (b) NOT gate (c) Flip Flop (d) X-OR

2. Which of the following cannot be used as a unit for frequency?
(a)HZ (b) BPM (c) RPM (d) S

3. Which of the following device used for measurement of frequency?
(a) Voltmeter (b) Ammeter (c) Stroboscope (d) None

4. A __ enables us to display a BCD code on a seven-segment display.
(a) Encoder (b) Decoder (c) MUX (d) DE-MUX

5. The device used to display the result is
(a)Seven segment display (b) LCD (c) TV (d) a & b

6. Arrangement of memory cell is called
(a)Read (b) Write (c) Array (d) Word

7. Which of the following is non-saturating?
(a) TTL (b) CMOS (c) EXL (d) BOTH A & B

8. A device which converts BCD to seven segments is called
(a)Encoder (b) Decoder (c) MUX (d) None

9. The basic storage element in a digital system is
(a) Flip-flop (b) counter (c) multiplexer (d) encoder

10. At which frequency the digital data can be applied to a gate?
(a) Run-time (b) Propagation (c) Operating (d) ~AC

ANSWER KEY

1.	C	2.	d	3.	D	4.	c	5.	d
6.	C	7.	c	8.	B	9.	a	10.	c

PART-II SAMPLE SHORT QUESTIONS

1. Define digital electronics?
2. What is digital clock?
3. Define the term counter?
4. Define the term frequency counter?
5. Draw the circuit diagram of moving display unit?
6. Draw the block diagram of frequency counter?

7. Advantages of frequency counter?
8. Draw the block diagram of digital clock?
9. What are the applications of digital clock?
10. What are the features of digital clock?
11. Why we use frequency counter?
12. How digital circuits are made?
13. What are the applications of digital circuit?
14. How seven segment display works?
15. How frequency counter works?
16. How digital clock works?
17. What is the working of 555 timers in digital clock?
18. Define digital electronics?
19. Define analog electronics?
20. Define the term frequency?

Part-III Long Questions

1. Define digital electronics also briefly explain the applications of digital circuits?
2. What is frequency counter? Explain the topic with circuit diagram?
3. Write down the construction and working of frequency counter?
4. Write down the construction and working of digital clock?
5. Write down the construction and working of moving display project?